

National Engineering Degree

Academic year 2025/2026



Computer Science and Information Systems

Software Engineering and Data Science

Final-Year Engineering Project Report

Discipline: Web development

Enterprise Knowledge Intelligence Platform based on Artificial Intelligence

*Plateforme d'Intelligence de Connaissance d'Entreprise basée sur
l'Intelligence Artificielle*

Prepared by	Adam Farhat
Host organization	HyprCraft Solutions
Industrial supervisor	mohamed aziz rezgui
Academic supervisor	Jacem Mhenni
Defense date	July 24, 2026
Jury chair	Haifa Mohdhi
Examiner	Hajer Maraoui

Dedication

I dedicate this work to my parents, whose unconditional love, support, and sacrifices have been the foundation of all my achievements. Their encouragement has been a constant source of strength throughout my academic journey.

To my family and friends, I express my sincere gratitude for their continuous support, patience, and understanding during both the difficult and rewarding moments of this work.

I also dedicate this project to my own efforts and perseverance, which allowed me to overcome challenges and stay committed to my goals.

Finally, I extend my deepest appreciation to my professors, whose guidance, expertise, and dedication have greatly contributed to my learning and to the successful completion of this project.

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to the success of this final-year engineering project.

My thanks go first to my industrial supervisor at HyprCraft Solutions, for providing a meaningful problem statement, regular feedback, and access to the organizational context that shaped the requirements of the Knowledge Platform.

I am equally grateful to my academic supervisor at ESPRIM, Jacem Mhenni, for methodological guidance, critical review of the technical choices, and support in aligning the deliverable with engineering-degree expectations.

I also thank the examination jury for their time and constructive evaluation, as well as the ESPRIM faculty who prepared us for this capstone experience.

Finally, I acknowledge my colleagues and peers whose discussions, testing, and code reviews improved the robustness of the implemented solution.

Abstract

Organizations today generate vast volumes of internal documentation — policies, technical guides, reports, and procedures — that remain largely inaccessible due to fragmented storage, inadequate search tools, and the absence of structured access governance. Employees, particularly new hires, waste significant time locating information, asking colleagues, or making decisions without reliable knowledge backing.

This report presents the design and implementation of the **Knowledge Platform**, an enterprise-grade document management and AI-assisted question-answering system built for teams organised by departments, with role-based access control. The platform is developed as a TypeScript monorepo comprising a **Next.js 15** web application, a **Node.js/Express** REST and Server-Sent Events API, **PostgreSQL with pgvector** for hybrid retrieval, **Redis and BullMQ** for asynchronous document ingestion, and **Google Gemini** as the sole external AI provider.

The core technical contribution is a **hybrid Retrieval-Augmented Generation (RAG) pipeline** combining dense vector search and BM25-style full-text retrieval, merged through weighted Reciprocal Rank Fusion (RRF), followed by LLM-based reranking and streaming answer generation with source citations. The system enforces department-scoped document visibility, a three-role hierarchy (Administrator, Manager, Employee), and a feedback memory loop that uses negative user ratings to improve future answers semantically.

Validation is grounded in **74 automated integration tests** across ten Vitest suites, a RAG evaluation harness with automated judge metrics, and a fully reproducible Docker Compose deployment. The project was managed using an **Agile Scrum** methodology structured across five sprints.

Keywords: Retrieval-Augmented Generation, Enterprise Search, Vector Database,

Access Control, Full-Stack Web Application, Large Language Models

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
Table of contents	viii
List of figures	x
List of tables	xi
List of abbreviations and acronyms	xiii
General introduction	1
1 State of the Art	3
1.1 Scientific and Technical Context	3
1.1.1 The Enterprise Knowledge Lifecycle Problem	3
1.1.2 Limitations of Traditional Document Systems	4
1.2 Information Retrieval: From Keywords to Semantics	4
1.2.1 Classical Retrieval Models	4
1.2.2 Dense Retrieval and Semantic Search	5
1.2.3 Hybrid Retrieval Systems	5
1.3 Retrieval-Augmented Generation (RAG)	5
1.3.1 Concept and Architecture	5
1.3.2 Advantages of RAG	6
1.3.3 Limitations in Enterprise Context	6
1.3.4 Evaluation of RAG Systems	6

1.4	Access Control and Knowledge Governance	6
1.4.1	Role-Based Access Control (RBAC)	7
1.4.2	Challenges in Knowledge Systems	7
1.5	Enterprise Knowledge Systems and Existing Solutions	7
1.5.1	Shared Drives and Document Management Systems	7
1.5.2	Vector Databases and RAG Frameworks	8
1.5.3	Commercial AI Copilots	8
1.5.4	Knowledge Base Platforms	9
1.5.5	Enterprise AI Search Systems (Glean)	9
1.6	Comparative Analysis	9
1.7	Project Positioning	10
1.7.1	Key Contribution	11
1.8	Chapter Conclusion	11
2	Requirements and Methodology	12
2.1	Analysis of the Existing Situation and Usage Context	12
2.1.1	Host Organisation Context	12
2.1.2	System Actors	13
2.1.3	Global Use-Case Model	14
2.2	Functional and Non-Functional Requirements	15
2.2.1	Functional Requirements	15
2.2.2	Non-Functional Requirements	19
2.2.3	Access Control Model	19
2.3	Specifications and Performance Indicators	20
2.4	Selected Methodology	21
2.4.1	Agile Scrum Framework	21
2.4.2	Sprint Plan	23
2.4.3	Development and Modelling Tools	30
2.4.4	Data Science Orientation	30
2.5	Chapter Conclusion	31
3	Design and Architecture	32
3.1	Overall Solution Architecture	32

3.2	Logical Subsystem Decomposition	34
3.3	Domain Data Model	34
3.4	Technology Choice Justification	35
3.5	Dynamic Interaction Design by Sprint	36
3.5.1	Sprint 1: Authentication and access control	37
3.5.2	Sprint 2: Document library and ingest	50
3.5.3	Sprint 3: Search, RAG and conversations	60
3.5.4	Sprint 4: Governance and notifications	68
3.5.5	Sprint 5: Operations, testing and deployment	84
3.6	Chapter Conclusion	87
4	Implementation	88
4.1	Hardware and Software Environment	88
4.2	Implementation Steps by Sprint	89
4.2.1	Sprint 1: Authentication and access control	89
4.2.2	Sprint 2: Document library and ingest pipeline	89
4.2.3	Sprint 3: Search, RAG and conversations	89
4.2.4	Sprint 4: Governance and notifications	90
4.2.5	Sprint 5: Operations, testing and deployment	90
4.3	Difficulties Encountered and Solutions	92
4.4	Produced Technical Deliverables	92
4.5	Chapter Conclusion	93
5	Validation	94
5.1	Validation plan	94
5.2	Protocols and experimental conditions	94
5.3	Presentation of Results	95
5.3.1	Integration Test Results by Sprint Domain	96
5.3.2	RAG evaluation results	97
5.3.3	Manual UX validation	97
5.4	Critical discussion	97
	General conclusion and perspectives	99

References	101
A Complete system diagrams	103
A.1 Structural diagrams	103
A.2 Sequence diagram index (by sprint)	105

List of Figures

2.1	Global use-case diagram of the Knowledge Platform	14
2.2	Scrum sprint timeline — iterative delivery across five sprints	24
3.1	Overall platform architecture	33
3.2	Global domain class diagram (persisted entities and main relationships) .	35
3.3	Sprint 1 use-case diagram — Authentication and access control	38
3.4	Sprint 1 domain class diagram — Authentication and access control . . .	39
3.5	User login	40
3.6	Access token refresh and session rotation	42
3.7	User logout	43
3.8	Forgot and reset password	44
3.9	Change password with session invalidation	45
3.10	Profile update	46
3.11	Feature restriction enforcement	47
3.12	Current user session payload	48
3.13	Register information page (no self-service signup)	49
3.14	Restricted feature explanation page	49
3.15	Sprint 2 use-case diagram — Document library and ingest	50
3.16	Sprint 2 domain class diagram — Document library and ingest	51
3.17	Document upload	52
3.18	Asynchronous ingest worker	53
3.19	Reprocess document version	54
3.20	Document file download	55
3.21	Document list	56
3.22	Document detail	57
3.23	Favorites and recents	58
3.24	Document metadata update	59

3.25	New document version	60
3.26	Sprint 3 use-case diagram — Search, RAG and conversations	61
3.27	Sprint 3 domain class diagram — Search, RAG and conversations	62
3.28	Semantic vector search	63
3.29	Hybrid RAG Ask with SSE streaming	64
3.30	Conversation and messages	65
3.31	Answer feedback	66
3.32	Conversation CRUD supplement	67
3.33	Admin feedback statistics	68
3.34	Sprint 4 use-case diagram — Governance and notifications	69
3.35	Sprint 4 domain class diagram — Governance and notifications	70
3.36	Automatic notification on document events	71
3.37	Manual announcement	72
3.38	Notification inbox	73
3.39	Manager department overview	74
3.40	Admin create user	75
3.41	Admin merge departments	76
3.42	Bulk department access	76
3.43	Admin create department	77
3.44	Single department access grant/revoke	78
3.45	Admin KPI statistics	79
3.46	Admin activity log and CSV export	80
3.47	Document audit trail	81
3.48	Documents CSV export	82
3.49	Bulk document delete	83
3.50	Admin user patch and lock	84
3.51	Sprint 5 use-case diagram — Operations, testing and deployment	85
3.52	Sprint 5 domain class diagram — Operations, testing and deployment	85
3.53	API health probe	86
A.1	Overall system architecture	103
A.2	Global use-case diagram	104
A.3	Domain class diagram	105

List of Tables

1.1	Comparative analysis of enterprise knowledge management solutions . . .	10
2.1	System actors and their primary capabilities	13
2.2	Functional requirements with acceptance criteria	15
2.3	Functional requirements traceability summary	18
2.4	Non-functional requirements	19
2.5	Key performance indicators	21
2.6	Agile Scrum sprint plan	23
2.7	Sprint 1 backlog: authentication and access control	25
2.8	Sprint 2 backlog: document library and ingest	26
2.9	Sprint 3 backlog: search, RAG and conversations	27
2.10	Sprint 4 backlog: governance and notifications	28
2.11	Sprint 5 backlog: operations, testing and deployment	29
2.12	Development and modelling tools	30
3.1	Subsystem responsibilities	34
3.2	Decision-support matrix for platform architecture (scores: 1 = weak, 3 = strong)	35
3.3	Requirements-to-design traceability (excerpt)	36
4.1	Development and deployment stack	88
4.2	Index of implementation sequence diagrams (Chapter 3, §3.5)	91
4.3	Implementation challenges and resolutions	92
5.1	KPI validation results	95
5.2	Validation test summary	96
A.1	Sequence diagram catalog indexed by sprint	106

List of abbreviations and acronyms

Acronym	Definition
AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Matching 25 (lexical ranking function)
BullMQ	Redis-backed job queue library used for document ingest
CORP	Cross-Origin Resource Policy (browser security header)
CSP	Content Security Policy
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DTO	Data Transfer Object
FR	Functional Requirement
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbor index)
HTTP	Hypertext Transfer Protocol
HyDE	Hypothetical Document Embeddings (retrieval augmentation technique)
IDE	Integrated Development Environment
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
MIME	Multipurpose Internet Mail Extensions (file type identification)
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping (Prisma)
PFE	Final-Year Project (Projet de Fin d'Études)
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
SSE	Server-Sent Events
TTL	Time To Live (cache expiration)
UML	Unified Modeling Language

General introduction

In recent years, organizations have undergone an accelerated digital transformation, leading to an exponential increase in the production of digital documents such as technical reports, internal procedures, contracts, and operational guidelines. While this information is essential for daily decision-making, it is often dispersed across multiple systems, shared drives, and communication channels, making it difficult to access efficiently.

This fragmentation of knowledge creates a significant operational challenge. Employees frequently struggle to locate relevant information, rely on informal communication channels, or recreate existing knowledge, which results in time loss, inconsistency, and reduced productivity. This issue becomes even more critical in engineering environments where accurate and timely access to information is essential.

Beyond simple document storage, organizations now require intelligent systems capable of organizing knowledge, controlling access according to organizational structure, and enabling efficient retrieval of relevant information. At the same time, the increasing use of artificial intelligence introduces new opportunities for improving how users interact with internal knowledge bases, particularly through natural language interfaces.

This final-year project was conducted during an internship at HyprCraft Solutions. It aims to explore and address the challenges of enterprise knowledge management by designing and implementing a platform that centralizes organizational documents and improves information accessibility through intelligent search and interaction mechanisms.

The main problem addressed in this work can be stated as follows: *How can an engineering organization structure its internal knowledge, ensure secure and role-based access to information, and enable efficient retrieval of reliable answers from its documentation?*

To address this problem, the project focuses on the following objectives:

1. design a secure platform supporting multiple user roles with controlled access to organizational documents;
2. improve information retrieval from heterogeneous document sources;
3. enable more efficient access to knowledge through intelligent assistance mechanisms;
4. ensure system reliability through testing and validation approaches;
5. provide a reproducible and deployable software solution.

The scope of this project includes the design and implementation of a full-stack web application for document management and knowledge access. It covers authentication, document organization, and intelligent retrieval functionalities, while ensuring structured access based on organizational roles and departments.

However, certain aspects remain outside the scope of this work, including mobile application development, large-scale distributed infrastructure deployment, and external enterprise authentication systems.

This report is organized as follows. Chapter 1 presents the state of the art and positions the project within existing solutions. Chapter 2 describes the requirements analysis and methodology. Chapter 3 details the system design and architecture. Chapter 4 presents the implementation process. Chapter 5 covers testing and validation. Finally, the conclusion summarizes the contributions and discusses future work.

CHAPTER 1

State of the Art and Project Positioning

This chapter surveys the scientific and industrial landscape relevant to enterprise knowledge management, information retrieval, and Retrieval-Augmented Generation. It analyses existing solutions comparatively and positions the Knowledge Platform within this context. Implementation details, requirements, and validation are deliberately out of scope here; they are addressed in subsequent chapters.

1.1 Scientific and Technical Context

1.1.1 The Enterprise Knowledge Lifecycle Problem

Modern organisations face a fundamental challenge in managing knowledge at scale. On one hand, they continuously generate large volumes of heterogeneous digital assets, including technical documentation, procedures, contracts, internal reports, spreadsheets, emails, and operational guidelines. On the other hand, these assets are distributed across multiple disconnected systems such as shared drives, email platforms, messaging tools, and departmental repositories.

This fragmentation creates a structural inefficiency that goes beyond retrieval. Enterprise knowledge is difficult not only to find, but also to:

- collect in a structured and consistent manner;

- centralize across heterogeneous tools and departments;
- organize according to business logic and ownership;
- govern through access control policies;
- retrieve in a meaningful and context-aware way.

As a result, employees often rely on informal communication or personal experience instead of documented knowledge. Studies show that knowledge workers spend a significant portion of their time searching for information rather than performing productive tasks [1].

1.1.2 Limitations of Traditional Document Systems

Traditional enterprise tools such as Google Drive, OneDrive, and Microsoft SharePoint were primarily designed for storage and collaboration rather than intelligent knowledge management.

While they provide reliable file storage and basic access control, they suffer from key limitations:

- lack of semantic understanding of content;
- keyword-based search that ignores meaning and context;
- weak knowledge structuring and duplication of information;
- limited ability to answer questions directly;
- coarse-grained permission models not aligned with organizational logic.

These systems function as passive repositories rather than active knowledge systems.

1.2 Information Retrieval: From Keywords to Semantics

1.2.1 Classical Retrieval Models

Traditional information retrieval relies on lexical matching approaches such as TF-IDF and BM25 [2]. These methods rank documents based on term frequency and inverse document frequency, making them effective for exact keyword matching.

However, they fail to capture semantic similarity. Two expressions with identical meaning but different wording may not match, which limits their effectiveness in enterprise contexts.

1.2.2 Dense Retrieval and Semantic Search

Dense retrieval methods address this limitation by encoding queries and documents into continuous vector representations [3, 4]. This enables semantic matching between conceptually similar texts even when lexical overlap is low.

Despite their advantages, dense retrieval models also have limitations:

- weaker performance on rare or technical terms;
- sensitivity to embedding model quality;
- reduced interpretability of ranking decisions.

1.2.3 Hybrid Retrieval Systems

To overcome these limitations, modern systems adopt hybrid retrieval strategies that combine lexical and semantic approaches [5, 6]. Results are typically merged using ranking fusion techniques such as Reciprocal Rank Fusion (RRF), which balances precision and recall.

This hybrid approach is now widely considered the standard for enterprise search systems.

1.3 Retrieval-Augmented Generation (RAG)

1.3.1 Concept and Architecture

Retrieval-Augmented Generation (RAG) [7] enhances traditional retrieval systems by integrating Large Language Models (LLMs). Instead of returning documents, the system retrieves relevant passages and uses them as context for answer generation.

A typical RAG pipeline consists of:

1. a retriever that selects relevant document chunks;

2. an optional reranker that refines retrieval quality;
3. a generator that produces a grounded natural language response.

1.3.2 Advantages of RAG

RAG systems provide several advantages over standalone LLMs:

- factual grounding in external documents;
- reduced hallucination through retrieval constraints;
- ability to update knowledge without retraining models;
- improved traceability through source citations.

1.3.3 Limitations in Enterprise Context

Despite their advantages, RAG systems introduce new challenges:

- strong dependency on retrieval quality;
- sensitivity to chunking and indexing strategies;
- difficulty in systematic evaluation;
- lack of native access control mechanisms;
- complexity of production deployment.

1.3.4 Evaluation of RAG Systems

Evaluating RAG systems remains a complex task, as classical IR metrics do not capture answer quality or factual grounding. Recent research proposes LLM-based evaluation frameworks that assess faithfulness, relevance, and completeness of generated answers [8]. These methods enable automated detection of hallucinations and quality regressions in production systems.

1.4 Access Control and Knowledge Governance

1.4.1 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) [9, 10] is the standard model for managing permissions in enterprise systems. Users are assigned to roles, and roles are associated with permissions over resources.

This model simplifies administration and supports organizational scalability.

1.4.2 Challenges in Knowledge Systems

In knowledge platforms, access control must be enforced beyond simple document visibility. It must govern:

- which documents can be retrieved during search;
- which sources can influence generated answers;
- cross-department boundaries and hierarchical structures;
- auditability of access and usage history.

A critical requirement is that access control must be applied at the retrieval layer to prevent information leakage in generated responses.

1.5 Enterprise Knowledge Systems and Existing Solutions

1.5.1 Shared Drives and Document Management Systems

Systems such as Google Drive, OneDrive, and Microsoft SharePoint [11] provide foundational document storage and collaboration capabilities.

Strengths:

- reliable storage and synchronization;
- basic access control mechanisms;
- widespread enterprise adoption.

Limitations:

- lack of semantic search;
- no question-answering capabilities;
- weak knowledge structuring;
- folder-based organization not aligned with knowledge flows.

1.5.2 Vector Databases and RAG Frameworks

Tools such as Pinecone, Weaviate, LangChain, and LlamaIndex [12–14] provide infrastructure for building retrieval systems.

Strengths:

- scalable semantic retrieval;
- modular RAG pipeline construction;
- flexible architecture for developers.

Limitations:

- no complete end-user system;
- no document lifecycle management;
- no access control or governance layer;
- significant engineering effort required for production use.

1.5.3 Commercial AI Copilots

Solutions such as Microsoft Copilot and ChatGPT Enterprise [15] provide conversational interfaces over enterprise data.

Strengths:

- natural language interaction;
- strong ecosystem integration;
- low adoption barrier.

Limitations:

- limited transparency in retrieval mechanisms;
- restricted control over embedding and ranking strategies;
- vendor lock-in;

- limited customization of access control logic.

1.5.4 Knowledge Base Platforms

Tools such as Guru and Tettra focus on structured knowledge management.

Strengths:

- organized documentation workflows;
- curated and validated content;
- team-oriented knowledge sharing.

Limitations:

- manual content creation required;
- no automated ingestion pipelines;
- limited or no AI-based retrieval;
- not suitable for large unstructured corpora.

1.5.5 Enterprise AI Search Systems (Glean)

Enterprise solutions such as Glean represent the most advanced commercial attempt at unified knowledge search.

Strengths:

- unified search across multiple enterprise data sources;
- natural language question answering with citations;
- strong user experience and integration capabilities.

Limitations:

- proprietary and closed architecture;
- no transparency in retrieval pipeline;
- no self-hosting or full customization;
- limited support for feedback-driven optimization.

1.6 Comparative Analysis

Table 1.1 – Comparative analysis of enterprise knowledge management solutions

Solution	Strengths	Limitations	Relevance to this work
Shared Drives (Drive, Share-Point)	Reliable storage and basic access control	No semantic search, no QA, weak structuring	Baseline storage layer only
Vector DB + RAG frameworks	Strong retrieval and modular pipelines	No full system or governance layer	Core retrieval component only
Commercial Copilots	Ease of use and enterprise integration	Black-box systems, limited control	Demonstrates demand but lacks transparency
Knowledge Base Tools	Structured documentation workflows	Manual maintenance, no scalable ingestion	Not suitable for large corpora
Glean	Unified enterprise AI search with citations	Closed system, no customization or transparency	Closest industrial solution but not open
Proposed System	Hybrid RAG + RBAC + full lifecycle + feedback loop	External LLM dependency	Full enterprise knowledge lifecycle solution

1.7 Project Positioning

The analysis reveals a structural gap in existing solutions: no system simultaneously addresses document lifecycle management, centralized governance, semantic retrieval, and controlled AI-based question answering in a unified and transparent architecture.

The **Knowledge Platform** addresses this gap through:

- centralized document ingestion and organization;
- role-based and department-scoped access control;
- hybrid lexical and semantic retrieval;
- Retrieval-Augmented Generation with source grounding;
- feedback-driven improvement mechanisms;
- full production-grade system architecture.

Unlike existing tools that focus on isolated components, this work proposes a complete and integrated enterprise knowledge lifecycle system.

1.7.1 Key Contribution

This work contributes a unified approach that ensures:

- knowledge is properly collected and centralized;
- access is securely governed and auditable;
- retrieval is hybrid and semantically aware;
- answers are grounded and traceable;
- the system is fully reproducible and production-ready.

1.8 Chapter Conclusion

The state-of-the-art analysis highlighted the limits of traditional document platforms and motivated a hybrid retrieval and RAG-based approach adapted to enterprise governance constraints. By comparing existing solutions and defining the project positioning, this chapter justified the need for an integrated knowledge platform combining lifecycle management, controlled access, semantic search, and grounded AI assistance. Chapter 2 translates these insights into functional and non-functional requirements and defines the Agile Scrum methodology adopted for delivery.

CHAPTER 2

Requirements Analysis, Specifications and Methodology

This chapter translates the gaps identified in Chapter 1 into a verifiable specification for the Knowledge Platform. It analyses the host-organisation context, defines functional and non-functional requirements with acceptance criteria, and establishes the Agile Scrum methodology and five-sprint delivery plan that structure the design (Chapter 3), implementation (Chapter 4), and validation (Chapter 5) chapters.

2.1 Analysis of the Existing Situation and Usage Context

2.1.1 Host Organisation Context

HyprCraft Solutions is a technology company whose teams produce and consume significant volumes of internal documentation: technical specifications, onboarding guides, project reports, API references, and operational procedures. Prior to this project, knowledge was distributed across disconnected tools — shared drives, email threads, and informal messaging channels — with no unified governance model and no mechanism for intelligent retrieval.

The key pain points identified through analysis of the existing situation were:

- **Fragmented storage:** documents resided in multiple systems with no single point of access, making comprehensive search impossible.
- **No access governance:** there was no systematic control over which teams or roles could access which documents, leading to either over-sharing or information silos.
- **Keyword-only search:** retrieval was limited to filename and keyword matching, returning documents rather than answers.
- **Onboarding friction:** new employees had no reliable way to find answers to operational questions, creating repeated interruptions to experienced colleagues.
- **No feedback mechanism:** when information found through search was incorrect or outdated, there was no way to signal this or prevent the same error from recurring.

2.1.2 System Actors

Four actor groups interact with the Knowledge Platform, as summarized in Table 2.1 and detailed in the global use-case diagram (Subsection 2.1.3, Figure 2.1):

Table 2.1 – System actors and their primary capabilities

Actor	Primary capabilities
Employee	Browse and search the document library, perform semantic search and RAG question-answering, manage personal profile, maintain conversation history, and rate assistant answers.
Manager	All Employee capabilities, plus access to the manager dashboard showing managed departments and their members, and the ability to send announcements to managed departments.
Administrator	Full platform control: user lifecycle management, department CRUD and merge, per-user restrictions, bulk operations, audit log access, KPI dashboards, activity exports, and platform-wide notifications.
System	Autonomous actor: executes the BullMQ ingest worker (extract, chunk, embed), emits automatic notifications on document and role events, and runs the RAG evaluation harness.

2.1.3 Global Use-Case Model

Figure 2.1 presents the global use-case model of the Knowledge Platform. It identifies primary actors (**Guest**, **Employee**, **Manager**, and **Administrator**), with the three authenticated roles generalizing an abstract **Authenticated user** actor. Use cases are grouped into authentication, document management, search and AI, notifications, and governance domains. Secondary actors (**Email**, **Gemini**, and **BullMQ worker**) model external interactions through «notify», «invoke», and «enqueue» dependencies.

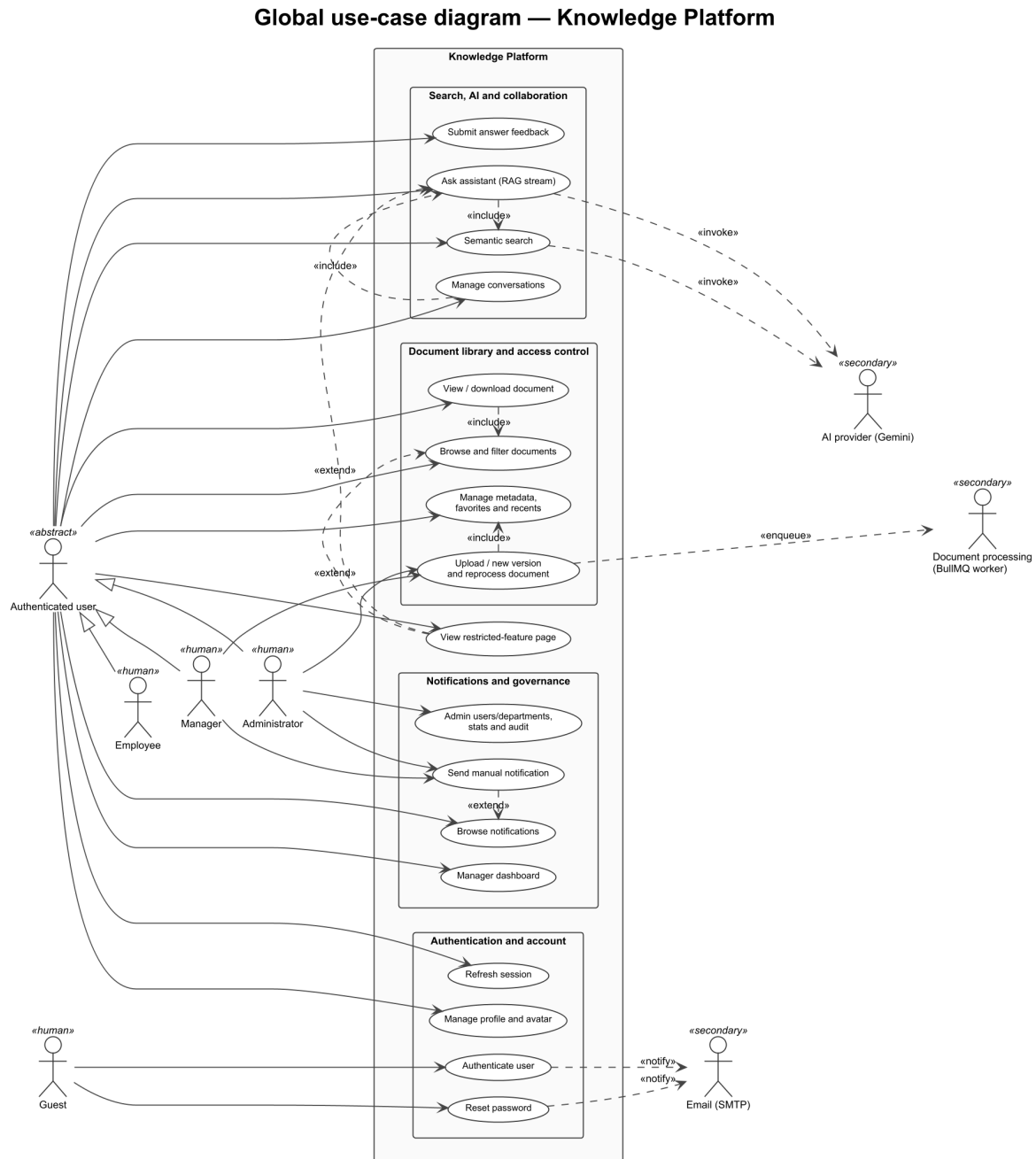


Figure 2.1 – Global use-case diagram of the Knowledge Platform

The use-case packages map directly to the functional requirement domains in Table 2.2: authentication (FR-01, FR-02, FR-12), document management (FR-03 to FR-05), search and AI (FR-06 to FR-08), notifications (FR-09), and governance (FR-10, FR-11).

2.2 Functional and Non-Functional Requirements

The requirements were derived from an analysis of user workflows and system architecture, ensuring alignment between functional needs and system design.

2.2.1 Functional Requirements

The system defines a set of functional requirements organized around **authentication** (FR-01, FR-02), **document management** (FR-03 to FR-05), **search and AI capabilities** (FR-06 to FR-08), **notifications** (FR-09), **administration** (FR-10, FR-11), and **system-level feature controls** (FR-12). Access to all functionalities is governed by a hybrid authorization model combining Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), and per-user restriction flags (see Subsection 2.2.3).

Table 2.2 – Functional requirements with acceptance criteria

ID	Requirement	Acceptance criterion	Module / API
FR-01	JWT authentication with HttpOnly refresh cookie rotation	Login returns access token; refresh rotates session atomically; replay revokes family	POST/auth/login POST/auth/refresh
FR-02	User profile management including avatar upload	Platform-hosted avatar served with correct CORP header; cross-origin display verified	PATCH/auth/profile GET/avatars/:id
FR-03	Document upload with multi-format extraction and async ingest	File stored on disk; ingest job enqueued; version status reaches READY	POST/documents/upload BullMQ worker

ID	Requirement	Acceptance criterion	Module / API
FR-04	Document visibility control (ALL / DEPARTMENT / PRIVATE)	Unauthorised access returns 404 (not 403) to prevent enumeration	<code>canReadDocument</code> <code>documentAccess.ts</code>
FR-05	Document metadata: tags, favorites, recent, archive	Tag autocomplete, favorite toggle, recent view, archive/restore all persisted	<code>documents.ts</code>
FR-06	Semantic vector search over department-scoped document corpus	Vector similarity results ranked correctly; access-filtered; respects restrictions	<code>POST/search/semantic</code>
FR-07	Hybrid RAG Ask with SSE streaming and source citations	SSE events emitted in order: <code>sources</code> , <code>token</code> , <code>done</code> ; citations reference real chunks	<code>POST/search/ask</code> <code>ragCompletion.ts</code>
FR-08	Conversation history with per-message feedback	Threads persisted; thumbs up/down stored; feedback stats available to admin	<code>/conversations/*</code>
FR-09	Automatic and manual notification system with attachments	Department-scoped events emitted on document create/update/delete; manual send available to managers/admins	<code>/notifications/*</code> <code>notificationService.ts</code>
FR-10	Full administrative control: user/department CRUD, bulk operations, audit	Last-admin safety enforced; CSV imports escaped; audit log exportable	<code>/admin/*</code>
FR-11	Manager dashboard scoped to manageable departments	Only departments from <code>manageableDepartmentIds</code> returned; member list correct	<code>/manager/*</code>

ID	Requirement	Acceptance criterion	Module / API
FR-12	Per-user feature restriction flags	Flags block login, document access, and AI queries via middleware; restricted page displayed	<code>requireDocLibraryAccess</code> <code>requireUseAiQueries</code>

Table 2.3 summarises the mapping from each functional requirement to its primary implementation module and development sprint.

Table 2.3 – Functional requirements traceability summary

ID	Domain	Primary module / service	Sprint
FR-01	Authentication	<code>auth.ts</code> (JWT, refresh families)	S1
FR-02	Profile management	<code>auth.ts</code> , <code>avatarsPublic.ts</code>	S1
FR-03	Document ingest	BullMQ ingest worker, <code>documents.ts</code>	S2
FR-04	Access control	<code>documentAccess.ts</code> , visibility filters	S2
FR-05	Document organisation	<code>documents.ts</code> (tags, favorites, archive)	S2
FR-06	Semantic search	<code>search.ts</code> , pgvector retrieval	S3
FR-07	RAG question answering	<code>ragCompletion.ts</code> (hybrid RAG pipeline)	S3
FR-08	Conversations	<code>conversations.ts</code>	S3
FR-09	Notifications	<code>notificationService.ts</code> , <code>notifications.ts</code>	S4
FR-10	Administration	<code>admin.ts</code>	S4
FR-11	Manager dashboard	<code>manager.ts</code>	S4
FR-12	Feature restrictions	Auth middleware, <code>requireUseAiQueries</code>	S1

Chapter 3 extends this summary with design elements and validation test references (Table 3.3).

2.2.2 Non-Functional Requirements

Table 2.4 – Non-functional requirements

ID	Requirement	Acceptance criterion
NFR-01	Health monitoring endpoint	GET <code>/health</code> returns 200 <code>{status: "ok"}</code> when PostgreSQL and Redis are reachable; 503 with per-check detail otherwise
NFR-02	Security headers and rate limiting	Helmet CSP enforced on all responses; login limited to 15 req/15 min/IP; Ask limited to 12 req/min/IP
NFR-03	Single-flight refresh token (web client)	Parallel requests from the same tab do not trigger multiple <code>/auth/refresh</code> calls; the in-flight promise is shared
NFR-04	Docker Compose reproducibility	<code>docker compose up --build</code> starts all four services (postgres, redis, api, web) in a healthy state; migrations run automatically
NFR-05	Integration test coverage	All Vitest/Supertest suites pass on a real PostgreSQL + Redis instance with seeded data
NFR-06	RAG evaluation harness	<code>npm run eval:rag:quick</code> executes without error and writes a JSON report to <code>eval-reports/</code>

2.2.3 Access Control Model

Building on the RBAC concepts introduced in Chapter 1, the platform enforces a **hybrid authorization model** that operates at three complementary layers. This subsection specifies how the theoretical foundations are applied to the requirements defined above.

- **Role-Based Access Control (RBAC)**: every user holds one platform role
- ADMIN, MANAGER, or EMPLOYEE — that determines coarse-grained capabilities.

Administrators manage users, departments, and platform configuration (FR-10); managers access department-scoped dashboards and announcements (FR-11); employees use the document library, search, and AI assistant within their permitted scope.

- **Attribute-Based Access Control (ABAC)**: permissions are further constrained by attributes such as department membership and document visibility (`ALL`, `DEPARTMENT`, `PRIVATE`). Retrieval and RAG pipelines apply these filters before returning results, ensuring that semantic search and generated answers never expose unauthorised content (FR-04, FR-06, FR-07).
- **Per-user feature restriction flags**: administrators can disable specific capabilities for individual users — for example, blocking document library access or AI queries — via middleware that intercepts requests before business logic executes (FR-12).

This layered model ensures fine-grained control over system capabilities while maintaining scalability and security across authentication, document management, search, and AI workflows.

2.3 Specifications and Performance Indicators

The following Key Performance Indicators (KPIs) link requirements to the validation chapter (Chapter 5):

Table 2.5 – Key performance indicators

KPI	Measurement method	Target
Ingest reliability	Percentage of uploaded documents reaching READY status without FAILED	$\geq 95\%$
Integration test pass rate	<code>npm run test:api</code> output: passed / total	74 / 74 (100%)
RAG topic classification (no API key)	<code>eval:rag:quick</code> optimizer pass rate	$\geq 15/34$ (degraded mode)
RAG judge scores (with API key)	Faithfulness and relevance from automated judge	Documented in eval report
Security: re-fresh replay	Reuse of rotated token triggers family revocation	Verified by auth test suite
Security: avatar CORP	Cross-Origin-Resource-Policy: <code>cross-origin</code> header present on avatar responses	Verified by avatars test

2.4 Selected Methodology

2.4.1 Agile Scrum Framework

The project was managed using the **Agile Scrum** methodology, chosen for its iterative structure, its emphasis on working deliverables at the end of each sprint, and its adaptability to evolving requirements — all well-suited to a complex full-stack product developed by a single engineer over a bounded internship period.

2.4.1.1 Scrum Roles

- **Product Owner:** mohamed aziz rezgui (HyprCraft Solutions), responsible for validating feature priorities and acceptance criteria.

- **Scrum Master / Developer:** Adam Farhat, responsible for sprint planning, execution, daily self-review, and retrospective.
- **Academic Advisor:** Jacem Mhenni (ESPRIM), providing methodological and scientific guidance at sprint boundaries.

2.4.1.2 Scrum Artefacts

- **Product Backlog:** the complete list of functional and non-functional requirements (FR-01 to FR-12, NFR-01 to NFR-06) prioritised by business value and technical dependency.
- **Sprint Backlog:** the subset of backlog items committed to in each sprint, decomposed into implementation tasks.
- **Increment:** a working, tested vertical slice of the platform delivered at the end of each sprint.
- **Definition of Done:** a requirement is considered done when (a) the feature is implemented, (b) at least one integration test covers the happy path and one error path, and (c) the corresponding API route or module is manually validated.

2.4.1.3 Sprint Structure

Each sprint followed a fixed rhythm:

1. **Sprint Planning:** review backlog, select items, decompose into tasks, estimate effort.
2. **Development:** implement features, write integration tests, perform code review against security and style guidelines.
3. **Sprint Review:** demonstrate the working increment; validate against acceptance criteria.
4. **Sprint Retrospective:** identify what worked, what blocked, and what to improve in the next sprint.

2.4.2 Sprint Plan

The project was structured across **five sprints**, each delivering a coherent and independently testable platform capability. Detailed interaction design for each sprint appears in Section 3.5 of Chapter 3.

Table 2.6 – Agile Scrum sprint plan

Spr.	Theme	Deliverables	Req.	Des.
S1	Auth., users, RBAC	JWT login/logout/refresh (HttpOnly cookies), refresh-token family revocation, password policy, session cap, per-user restrictions, profile/avatar, RBAC middleware	FR-01, FR-02, FR-12 NFR-02, NFR-03	3.5.1
S2	Document library & ingest	Multi-format upload (PDF, Office, HTML, XML, ZIP), BullMQ worker (extract→chunk→embed→persist), versioning, tags, favorites, recents, archive, visibility, audit log	FR-03, FR-04, FR-05	3.5.2
S3	Hybrid RAG & Ask	Query optimiser, hybrid retrieval (pgvector + BM25 + RRF), LLM reranking, neighbour expansion, SSE answers with citations, confidence scoring, critique, claim verification, follow-ups, feedback loop, conversations	FR-06, FR-07, FR-08 NFR-06	3.5.3
S4	Admin, manager, notifications	Admin CRUD (users, departments, merge, bulk ops, CSV), dept-access management, KPI dashboard, activity/audit exports, manager view, auto/manual notifications	FR-09, FR-10, FR-11	3.5.4
S5	Tests, eval & deploy	74 Vitest/Supertest integration tests, RAG eval CLI (automated judge), Docker Compose prod config, migration pipeline, 54 UML diagrams, README	NFR-01, NFR-04, NFR-05 NFR-06	3.5.5

The **Des.** column references the sprint interaction-design subsections of Section 3.5 (Chapter 3).

2.4.2.1 Sprint Timeline

Figure 2.2 illustrates the sprint sequence defined in Table 2.6 and the iterative delivery of working increments across the project duration. Each sprint builds on the previous one: authentication and access control (S1), document corpus (S2), hybrid RAG (S3), governance (S4), then operational hardening and validation (S5).

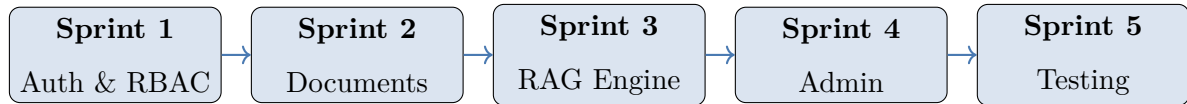


Figure 2.2 – Scrum sprint timeline — iterative delivery across five sprints

2.4.2.2 Sprint Backlogs

Tables 2.7–2.11 detail each sprint backlog at user-story level. The stories refine the scope presented in Table 2.6 and provide planning-level traceability between implementation tasks, acceptance criteria, and effort estimation. Interaction flows for every story are documented in Section 3.5.

Sprint 1 — Authentication and access control.**Table 2.7** – Sprint 1 backlog: authentication and access control

ID	User Story	Acceptance Criteria	SP	Pri.
US-S1-01	As a guest, I want to sign in securely so that I can access authorised platform features.	POST /auth/login issues access token + refresh cookie; invalid credentials rejected; login limiter enforced.	5	High
US-S1-02	As an authenticated user, I want seamless session continuity so that refresh operations are secure and race-condition free.	/auth/refresh rotates token family atomically; replay revokes family; single-flight refresh on web client.	8	High
US-S1-03	As an authenticated user, I want to sign out so that active refresh sessions are revoked.	Logout revokes current refresh session; subsequent refresh attempts fail.	2	Med.
US-S1-04	As a user, I want to view and update my profile, including avatar, so that my account information remains accurate.	PATCH /auth/profile persists updates; avatar upload works; avatar route serves CORP header for cross-origin display.	5	High
US-S1-05	As an administrator, I want role-based and per-user access restrictions so that capabilities can be controlled precisely.	Role middleware enforces ADMIN/MANAGER/EMPLOYEE boundaries; restriction flags block protected features and redirect to restricted page.	8	High
US-S1-06	As a security owner, I want baseline HTTP protections so that authentication endpoints resist abuse.	Helmet headers active globally; login and Ask endpoints rate-limited according to configured thresholds.	3	Med.

Sprint 2 — Document library and ingest.

Table 2.8 – Sprint 2 backlog: document library and ingest

ID	User Story	Acceptance Criteria	SP	Pri.
US-S2-01	As an employee, I want to upload documents in common formats so that they enter the shared library.	<code>POST/documents/upload</code> validates MIME/size, stores file on disk, creates version row, enqueues ingest job.	8	High
US-S2-02	As the system, I want to ingest uploaded files asynchronously so that text is searchable and embeddable.	BullMQ worker extracts text, chunks content, computes 768-d embeddings, persists <code>Chunk</code> rows; version reaches <code>READY</code> .	13	High
US-S2-03	As a security owner, I want visibility rules enforced so that unauthorised users cannot discover restricted documents.	<code>ALL/DEPARTMENT/PRIVATE</code> enforced via <code>canReadDocument</code> ; unauthorised access returns 404.	8	High
US-S2-04	As an employee, I want to browse, search, and open documents so that I can find relevant files quickly.	<code>GET/documents</code> supports filters/pagination; <code>GET/documents/:id</code> returns metadata and active version; download streams file bytes.	5	High
US-S2-05	As an employee, I want favorites and recents so that I can return to frequently used documents.	Favorite toggle and recent-view endpoints persist per user; library home surfaces both lists.	3	Med.
US-S2-06	As a document owner, I want versioning and re-ingestion so that updates and embedding upgrades are supported.	<code>POST/documents/:id/versions</code> creates new version + ingest job; <code>POST.../reprocess</code> forces re-ingest of existing version.	8	Med.
US-S2-07	As a manager, I want to update metadata and retain an audit trail so that document governance is traceable.	<code>PATCH/documents/:id</code> updates title, tags, visibility within scope; changes recorded in per-document audit log.	5	Med.

Sprint 3 — Search, RAG and conversations.**Table 2.9** – Sprint 3 backlog: search, RAG and conversations

ID	User Story	Acceptance Criteria	SP	Pri.
US-S3-01	As an employee, I want semantic search over the corpus so that I can find relevant passages by meaning.	<code>POST/search/semantic</code> embeds query, ranks department-scoped chunks via pgvector, returns snippet previews.	8	High
US-S3-02	As an employee, I want cited answers streamed in real time so that I receive grounded responses to natural-language questions.	<code>POST/search/ask</code> runs hybrid retrieval + RRF + rerank; SSE emits <code>sources</code> , <code>token</code> , done with chunk citations.	13	High
US-S3-03	As an employee, I want persistent conversation threads so that I can review prior questions and answers.	<code>/conversations</code> supports create, list, rename, delete; messages store sources, confidence, and citations.	8	High
US-S3-04	As an employee, I want to rate assistant answers so that poor responses can be identified and improved.	<code>POST.../feedback</code> stores thumbs up/down; admin stats endpoint aggregates sentiment and volume.	5	Med.
US-S3-05	As a data scientist, I want a reproducible RAG evaluation harness so that retrieval quality can be measured over time.	<code>npm run eval:rag:quick</code> and <code>eval:rag</code> write JSON reports under <code>eval-reports/</code> without breaking CI.	8	Med.

Sprint 4 — Governance and notifications.**Table 2.10** – Sprint 4 backlog: governance and notifications

ID	User Story	Acceptance Criteria	SP	Pri.
US-S4-01	As an employee, I want automatic notifications on document events so that I stay informed of library changes.	Document create/update/delete emits department-scoped inbox entries via <code>notificationService.ts</code> .	5	High
US-S4-02	As a manager or admin, I want to send manual announcements so that I can communicate with targeted teams.	<code>POST/notifications/send</code> validates audience scope; optional attachments downloadable by recipients.	5	High
US-S4-03	As an employee, I want a notification inbox so that I can read and dismiss messages.	List, unread count, mark-read, and mark-all-read endpoints; web client polls with badge indicator.	5	Med.
US-S4-04	As a manager, I want a department dashboard so that I can oversee teams and documents I manage.	<code>/manager/*</code> returns only <code>manageableDepartmentIds</code> ; member lists and department documents scoped correctly.	8	High
US-S4-05	As an administrator, I want full user and department administration so that the platform reflects organisational structure.	<code>/admin/*</code> supports user CRUD, department create/merge, last-admin safety, CSV import with formula escaping.	13	High
US-S4-06	As an administrator, I want bulk access management and operational exports so that governance tasks scale efficiently.	Bulk department-access PUT, KPI stats, activity log, document audit, and CSV exports available to admins.	8	Med.

Sprint 5 — Operations, testing and deployment.**Table 2.11** – Sprint 5 backlog: operations, testing and deployment

ID	User Story	Acceptance Criteria	SP	Pri.
US-S5-01	As an operator, I want a health endpoint so that I can verify database and queue connectivity.	<code>GET/health</code> returns 200 when PostgreSQL and Redis are reachable; 503 with per-check detail otherwise.	3	High
US-S5-02	As a developer, I want automated integration tests so that regressions are caught before release.	<code>npm run test:api</code> runs 74 Vitest/Supertest cases across 10 suites on real DB/Redis; target 100% pass.	13	High
US-S5-03	As an operator, I want reproducible containerised deployment so that the platform can be started consistently.	<code>docker compose up --build</code> starts postgres, redis, api, web healthy; migrations run automatically.	8	High
US-S5-04	As a data scientist, I want automated RAG judge evaluation so that answer quality can be tracked with an API key.	<code>npm run eval:rag</code> produces faithfulness/relevance scores in <code>eval-reports/</code> ; quick mode runs without key.	8	Med.
US-S5-05	As a maintainer, I want complete design documentation so that the system is defensible and maintainable.	54 PlantUML diagrams exported; README and architecture guide updated; sequence flows indexed in Appendix A.	5	Med.

2.4.3 Development and Modelling Tools

Table 2.12 – Development and modelling tools

Tool	Usage in project
Prisma Schema DSL	Domain modelling and 29 sequential migrations
PlantUML	54 UML diagrams (3 structural, 5 sprint use-case, 5 sprint class, 41 sequence)
Vitest + Supertest	Integration testing across all API domains
Docker Compose	Reproducible dev and production environment
GitHub Actions	CI pipeline (lint, audit, typecheck, integration tests)
Zod	Runtime request validation for all API inputs

2.4.4 Data Science Orientation

In alignment with the Data Science discipline, the following elements of the project are treated as data-science artefacts:

- **Corpus and embeddings:** the set of ingested documents and their 768-dimensional vector representations in PostgreSQL constitute the experimental dataset on which retrieval quality is measured.
- **Query optimisation:** classifying query type (factual, summary, comparative, procedural, out-of-scope) and rewriting queries for retrieval is analogous to feature engineering in a machine learning pipeline.
- **RRF ensemble:** the weighted fusion of dense and sparse retrieval results is an ensemble method with tunable hyperparameters (`RRF_K`, `RRF_DENSE_WEIGHT`, `RRF_SPARSE_WEIGHT`).
- **Automated judge evaluation:** the RAG evaluation harness uses an LLM-as-judge to measure faithfulness, answer relevance, and context precision, producing reproducible JSON reports — a standard experimental validation workflow for generative AI systems [8].

- **Feedback-driven adaptation:** negative user ratings are semantically compared to new questions using embedding cosine similarity, constituting an online adaptation mechanism without model retraining.

2.5 Chapter Conclusion

This chapter translated the state-of-the-art insights into a verifiable specification by defining functional and non-functional requirements, acceptance criteria, and requirement-to-module traceability. It also formalized the project methodology through a five-sprint Scrum plan and detailed sprint backlogs, linking planning artefacts to implementation and validation objectives. Chapter 3 presents the resulting system design, including sprint-aligned interaction diagrams in Section 3.5.

CHAPTER 3

Design and Architecture

This chapter translates the requirements and sprint plan of Chapter 2 into a concrete system design. The presentation follows two complementary views:

- **Static design** (Sections 3.1–3.4) — platform architecture, subsystem responsibilities, persisted domain model, and technology justification;
- **Dynamic design** (Section 3.5) — sprint-scoped use-case and class models, followed by interaction sequence diagrams for every major API and UX flow.

Implementation details and validation evidence are developed in Chapters 4 and 5.

3.1 Overall Solution Architecture

The Knowledge Platform follows a three-tier logical architecture: browser client, API layer, and data/AI services. Figure 3.1 summarizes component responsibilities and data flows (see also Appendix A.1 for the full exported diagram).

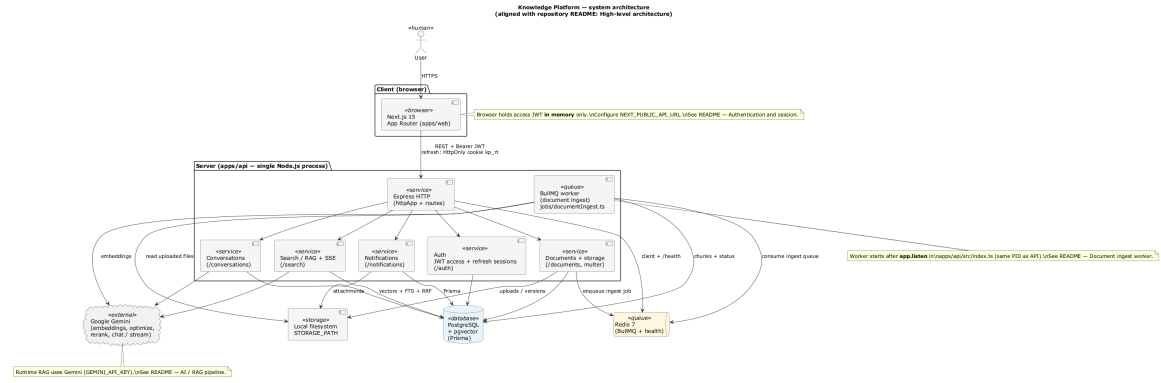


Figure 3.1 – Overall architecture of the Knowledge Platform (PlantUML source: platform-architecture.puml)

The web application (apps/web) keeps access tokens in memory and uses HttpOnly refresh cookies. The API (apps/api) mounts routers for /auth, /documents, /search, /conversations, /admin, /manager, /notifications, and public /avatars. A BullMQ consumer runs in the same Node process after the HTTP server starts.

3.2 Logical Subsystem Decomposition

Table 3.1 – Subsystem responsibilities

Subsystem	Responsibility	Key modules
Authentication	Login, refresh families, restrictions, password reset	<code>auth.ts</code> <code>authClient.ts</code>
Documents	CRUD, versions, visibility, audit, download	<code>documents.ts</code> <code>storage.ts</code>
Ingest worker	Extract, chunk, embed, persist chunks	<code>BullMQworker</code> <code>index.ts</code>
Search / RAG	Hybrid retrieval, SSE answers, cache	<code>search.ts</code> <code>ragCompletion.ts</code>
Conversations	Threads, messages, feedback	<code>conversations.ts</code>
Notifications	Auto events and manual announcements	<code>notifications.ts</code>
Admin / Manager	Governance dashboards	<code>admin.ts</code> <code>manager.ts</code>

3.3 Domain Data Model

The Prisma schema defines 20+ entities including `User`, `Department`, `Document`, `DocumentVersion`, `Chunk` (768-d embedding), `Conversation`, `AnswerFeedback`, and `Notification`. Multi-department access is modeled through `UserDepartmentAccess`. Figure 3.2 presents the principal persisted classes and their relationships across all sprints; sprint-scoped excerpts appear in Section 3.5.

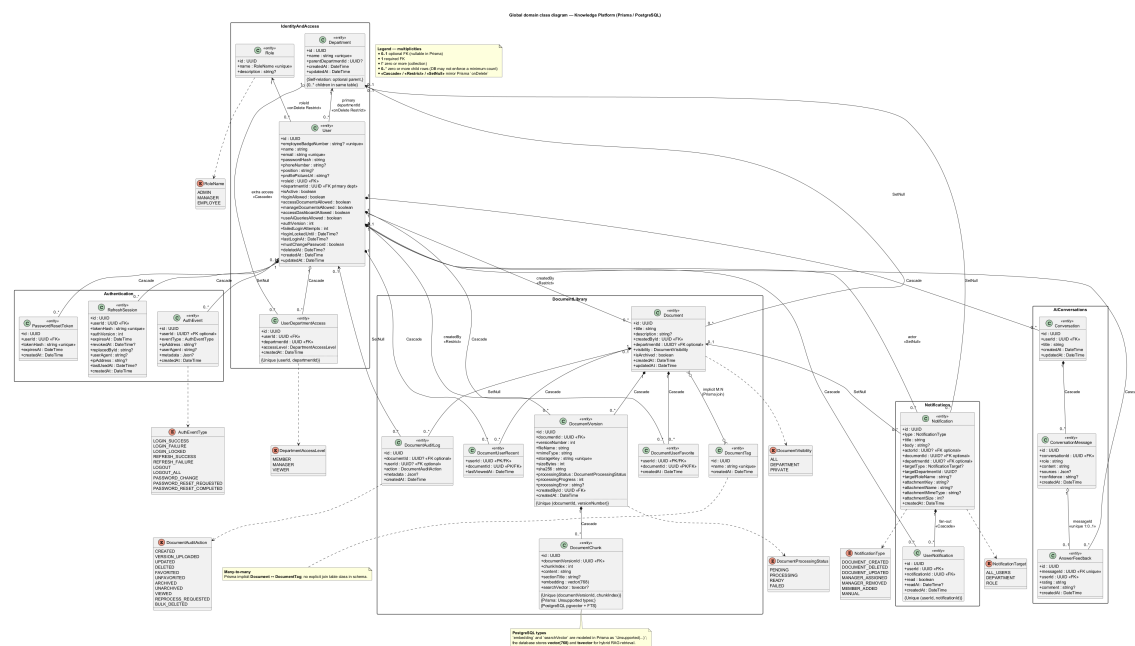


Figure 3.2 – Global domain class diagram (persisted entities and main relationships)

3.4 Technology Choice Justification

Table 3.2 scores architectural options (1 = weak, 3 = strong). The selected stack favours a TypeScript monorepo with Express and Next.js for streaming RAG, operational simplicity, and alignment with project skills.

Table 3.2 – Decision-support matrix for platform architecture (scores: 1 = weak, 3 = strong)

Criterion	Monorepo	Django	Microservice	Rationale
Time to market	3	2	1	Shared TypeScript types; one repository
SSE streaming fit	3	2	2	Native Express streaming for RAG
Team skill alignment	3	2	1	Matches React + Node stack
Operational complexity	3	3	1	Single API process + worker

PostgreSQL + pgvector vs. dedicated vector DB: colocating relational and vector

data simplifies hybrid SQL+vector retrieval, transactional consistency, and department filters in one query layer [16].

Gemini vs. open-weight models: the embedding and chat APIs reduce operational burden while providing strong multilingual embeddings (768 dimensions in this deployment) [17].

In-process BullMQ worker vs. separate worker service: embedding the consumer in the API process reduces deployment complexity for an academic/prototype-scale platform, at the cost of horizontal scaling coupling.

Table 3.3 links representative requirements to the chosen design elements and validation artefacts (see also Table 2.3 in Chapter 2).

Table 3.3 – Requirements-to-design traceability (excerpt)

Req.	Design choice	Implemented element	Validation
FR-01	JWT + re-fresh rotation	<code>auth.ts</code> <code>authClient.ts</code>	<code>auth.integration.test</code>
FR-02	Avatar CORP header	<code>avatarsPublic.ts</code>	<code>avatars.integration.test</code>
FR-03	Async ingest queue	BullMQ + Redis	<code>documents.integration.test</code>
FR-07	Hybrid RAG + SSE	<code>search.ts</code> <code>ragCompletion.ts</code>	<code>search.integration.test</code>
FR-10	Admin RBAC	<code>admin.ts</code>	<code>admin.integration.test</code>
NFR-02	Helmet + rate limits	<code>httpApp.ts</code>	manual + integration

3.5 Dynamic Interaction Design by Sprint

Sections 3.1–3.4 describe **what** the platform is made of (components, data, and technology). This section describes **how** actors and subsystems interact at runtime. For

each sprint in Table 2.6 of Chapter 2, the design is organised in three layers:

1. **Sprint scope** — objectives and mapped functional requirements;
2. **Scoped structural views** — sprint use-case model and sprint domain class model;
3. **Interaction sequence diagrams** — request/response flows between client, API, workers, and external services.

The full diagram catalog is indexed in Appendix A; implementation evidence appears in Chapter 4 and validation in Chapter 5.

3.5.1 Sprint 1: Authentication and access control

Sprint 1 establishes the security perimeter of the platform. It delivers credential-based authentication with refresh-token rotation (FR-01), profile and avatar management (FR-02), per-user feature restrictions (FR-12), and the non-functional controls for HTTP hardening and single-flight token refresh (NFR-02, NFR-03).

3.5.1.1 Scoped structural views

Use-case model.

Sprint 1 use-case diagram — Authentication and access control

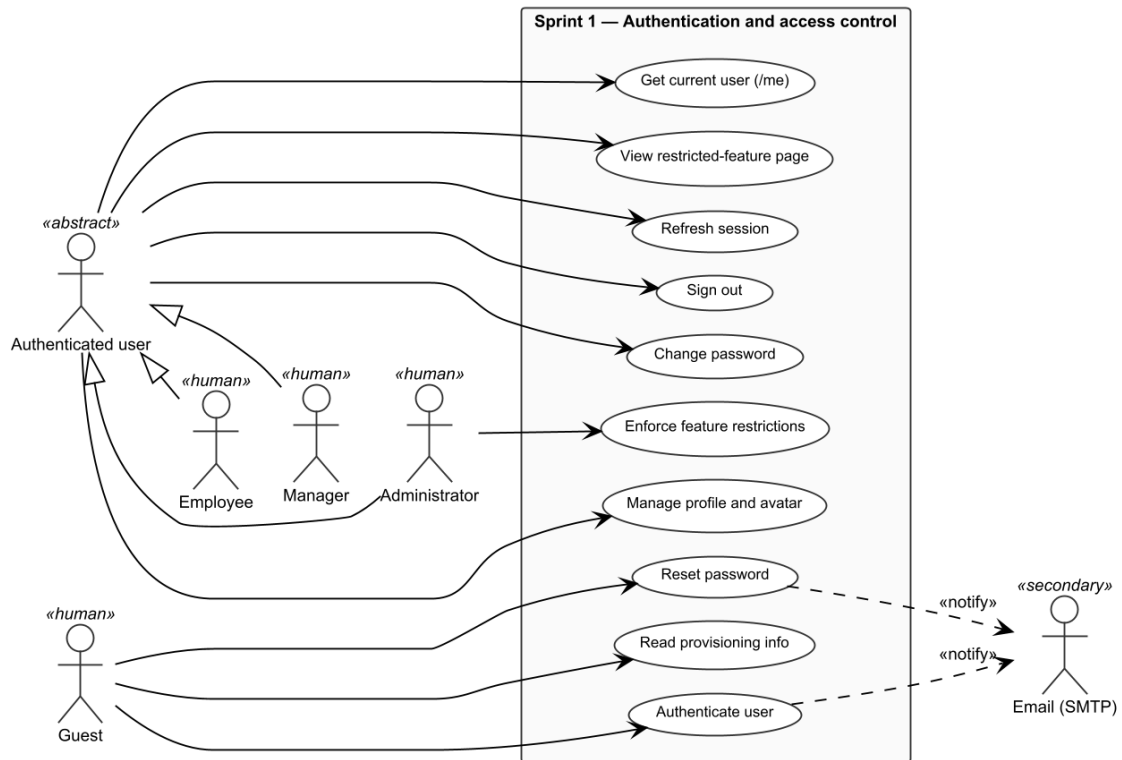


Figure 3.3 – Sprint 1 use-case diagram — Authentication and access control

Domain class model.

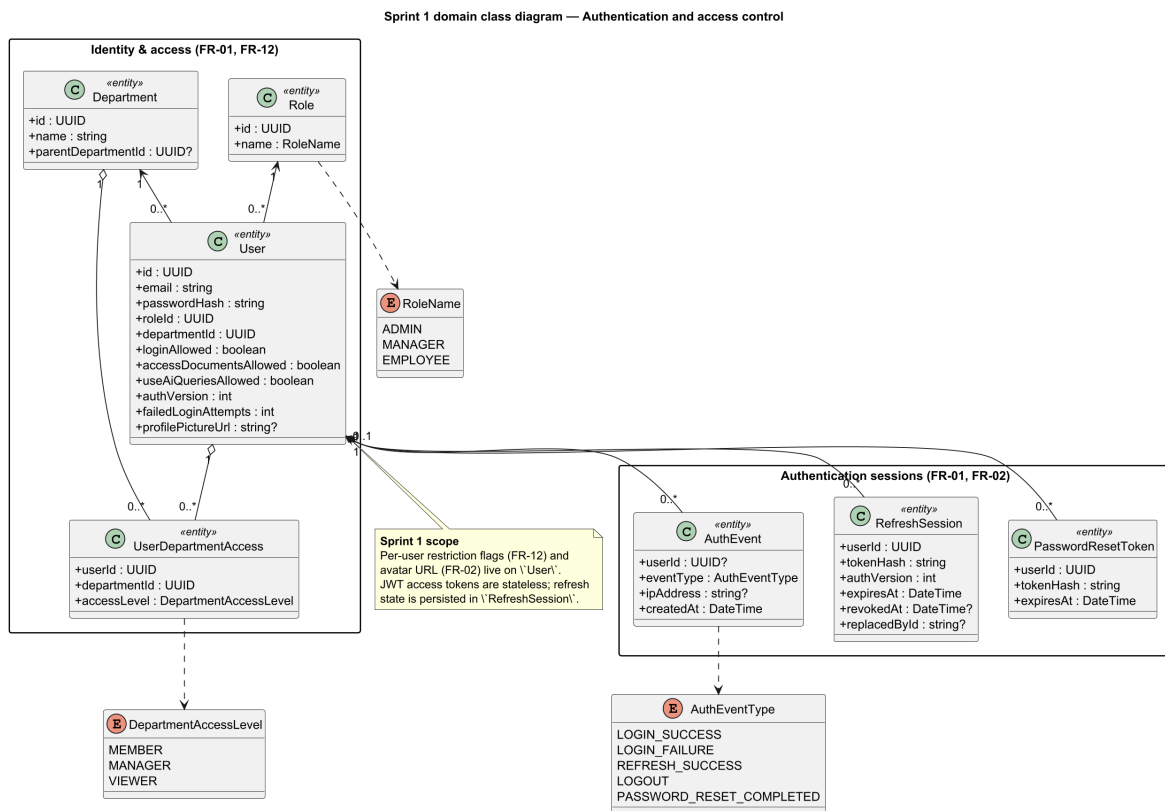


Figure 3.4 – Sprint 1 domain class diagram — Authentication and access control

3.5.1.2 Interaction sequence diagrams

Authentication and session management

User login.

The login flow validates email and password against bcrypt hashes, enforces account lockout after repeated failures, and issues a short-lived access JWT together with an HttpOnly refresh cookie bound to a token family stored in the database.

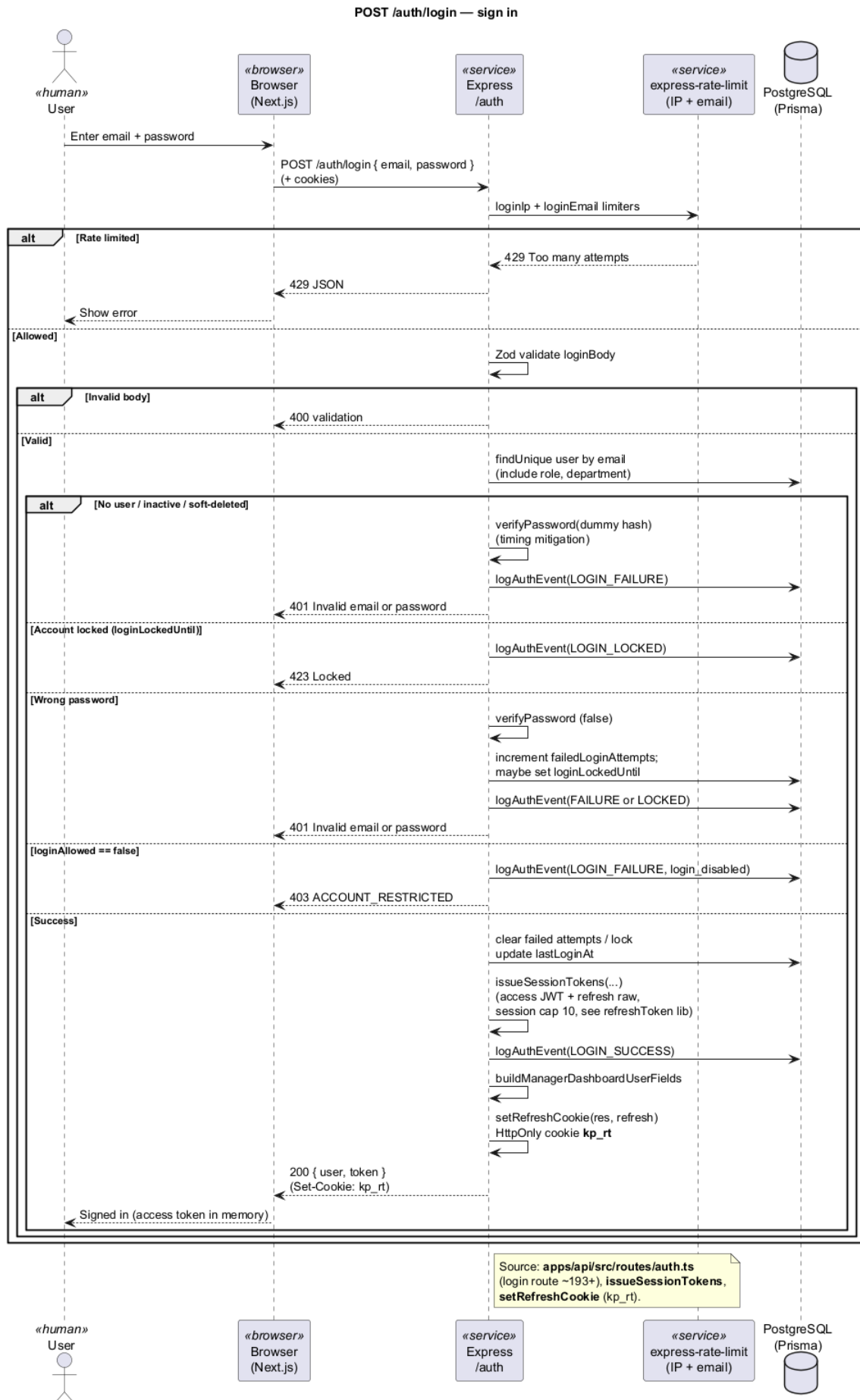
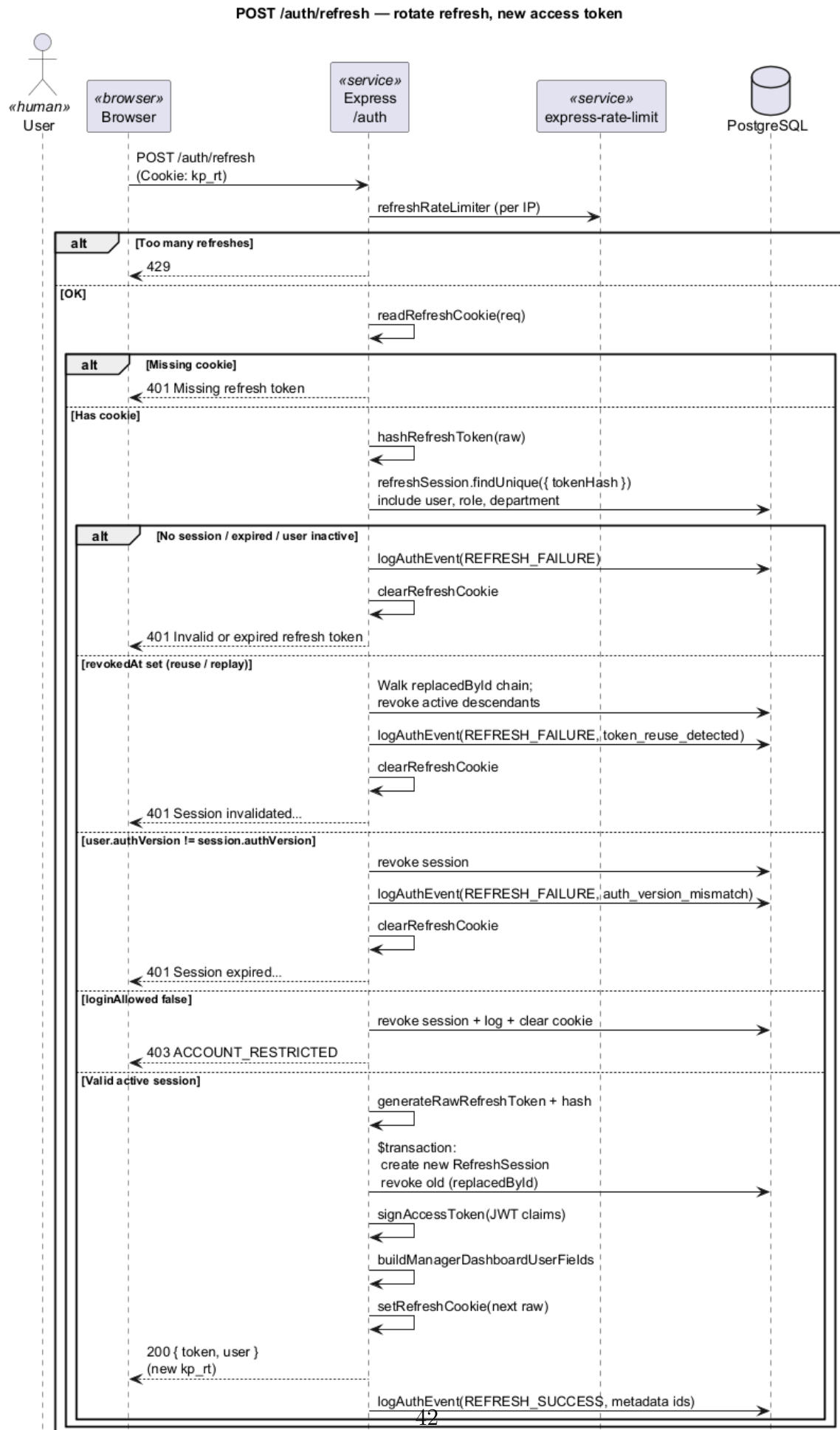


Figure 3.5 – User login (POST /auth/login)

Access token refresh and session rotation.

When the in-memory access token expires, the web client triggers a single in-flight refresh request that rotates the refresh cookie, invalidates the previous family member, and returns a new access token without user interaction.



User logout.

Logout revokes the active refresh-token family server-side and clears the HttpOnly cookie, ensuring that stolen refresh tokens cannot be reused after the user explicitly ends the session.

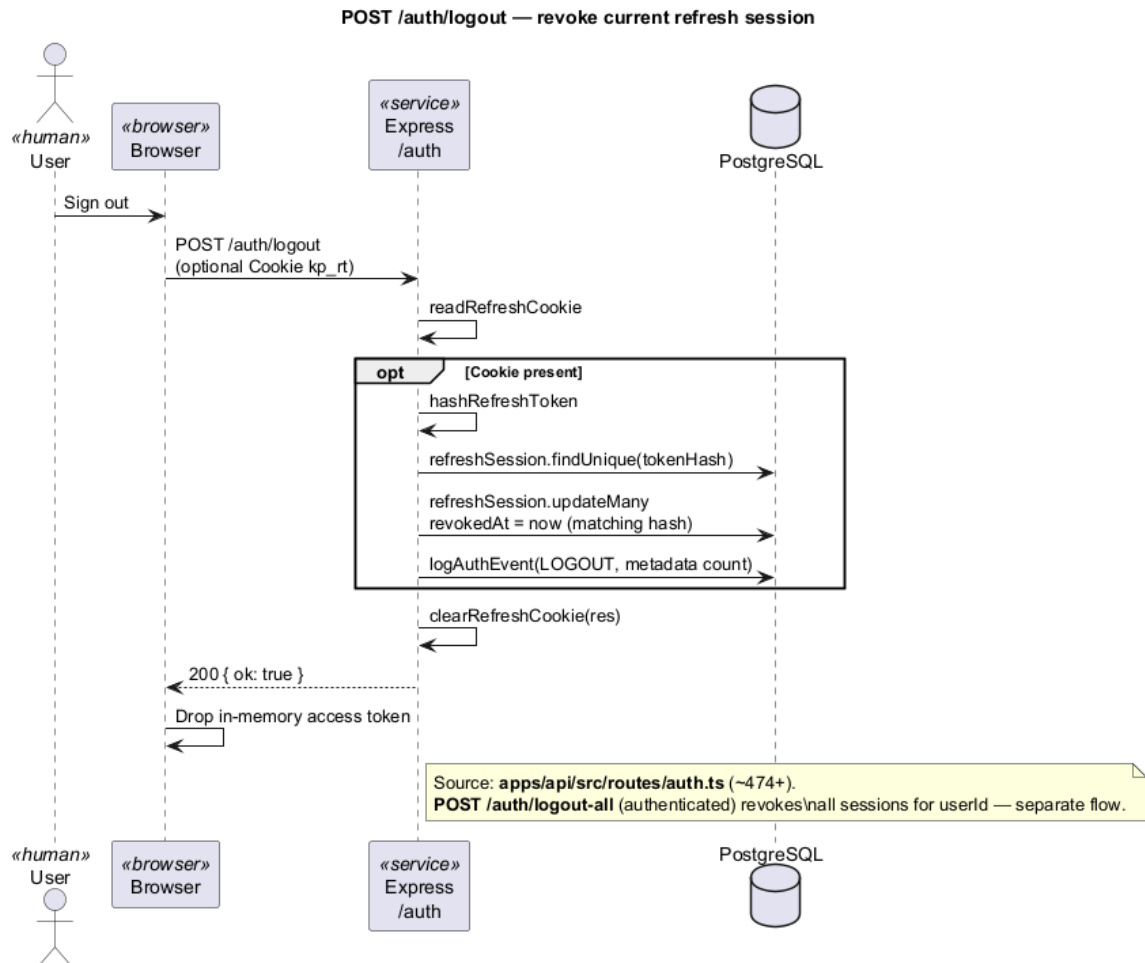


Figure 3.7 – User logout (POST /auth/logout)

Forgot and reset password.

Password recovery sends a time-limited reset link by email; the reset endpoint validates the token, applies the password policy, and invalidates all outstanding refresh families for the affected account.

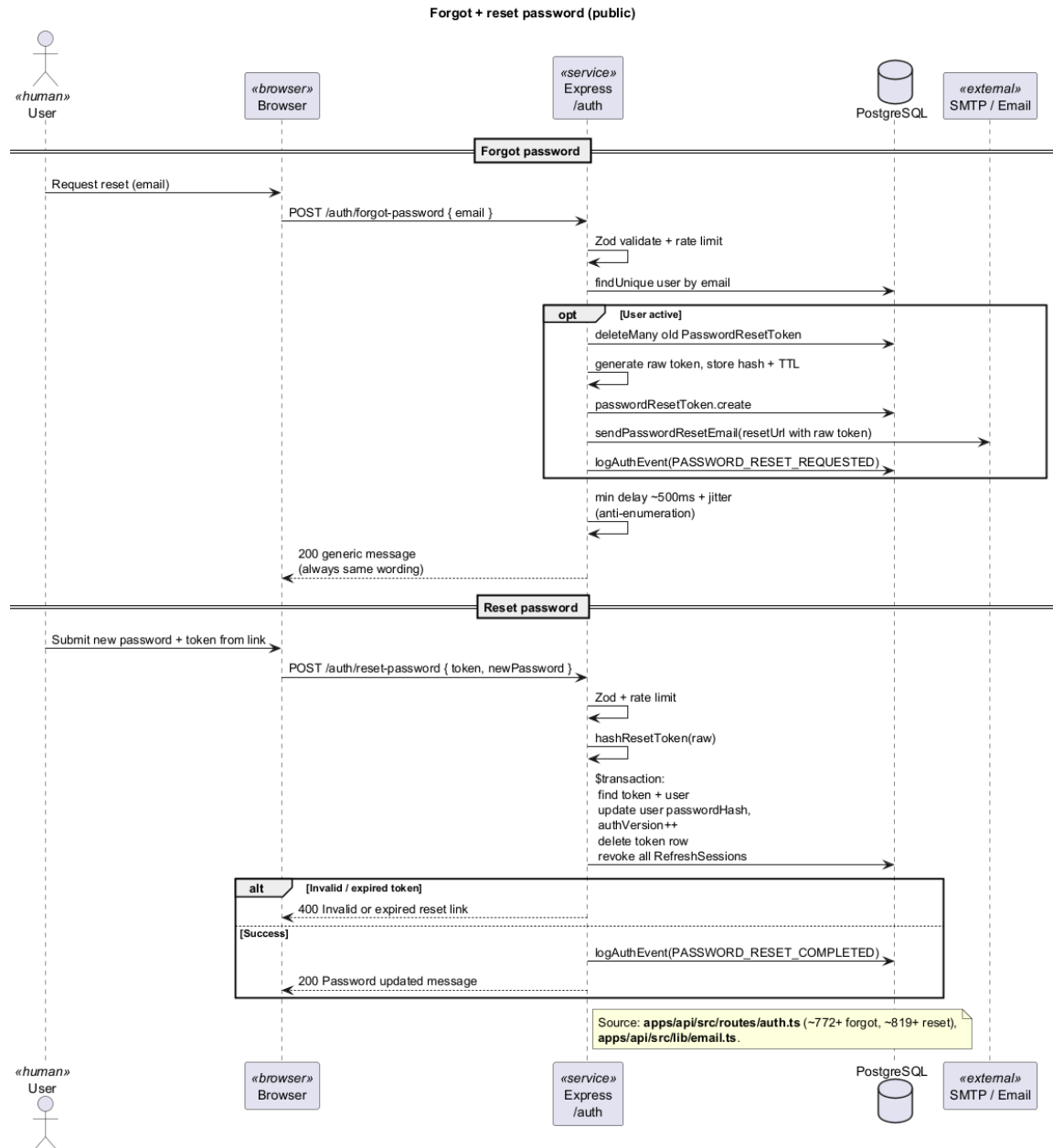


Figure 3.8 – Forgot and reset password (POST /auth/forgot-password, reset-password)

Change password with session invalidation.

Authenticated users may change their password by supplying the current secret; successful rotation updates the hash, enforces policy rules, and terminates other active sessions through family revocation.

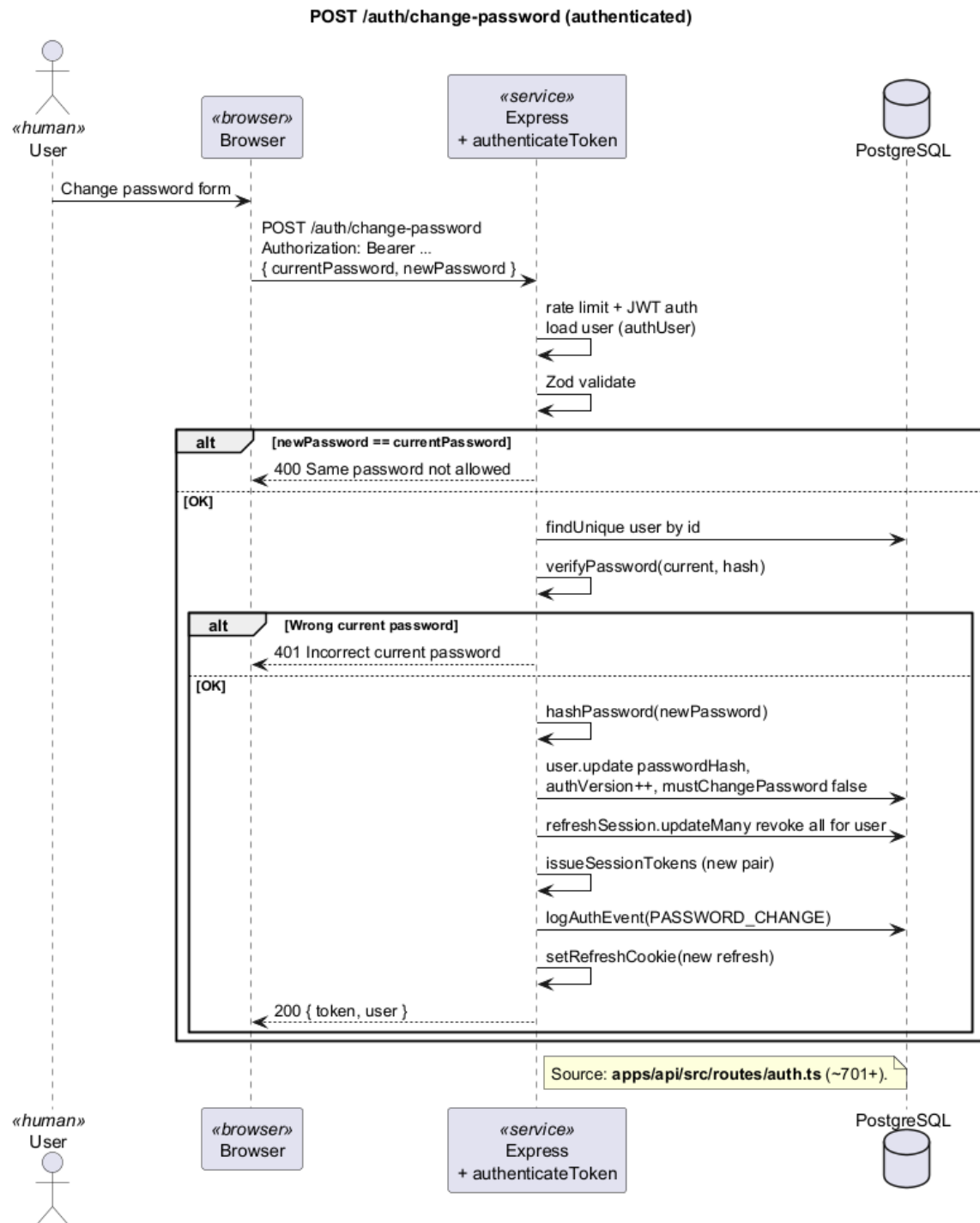


Figure 3.9 – Change password with session invalidation (POST /auth/change-password)

Profile, restrictions and session context

Profile update.

Profile updates accept display name and avatar file uploads; the API persists metadata in Prisma and stores avatar binaries on disk, exposing them through the public `/avatars` route with cross-origin headers.

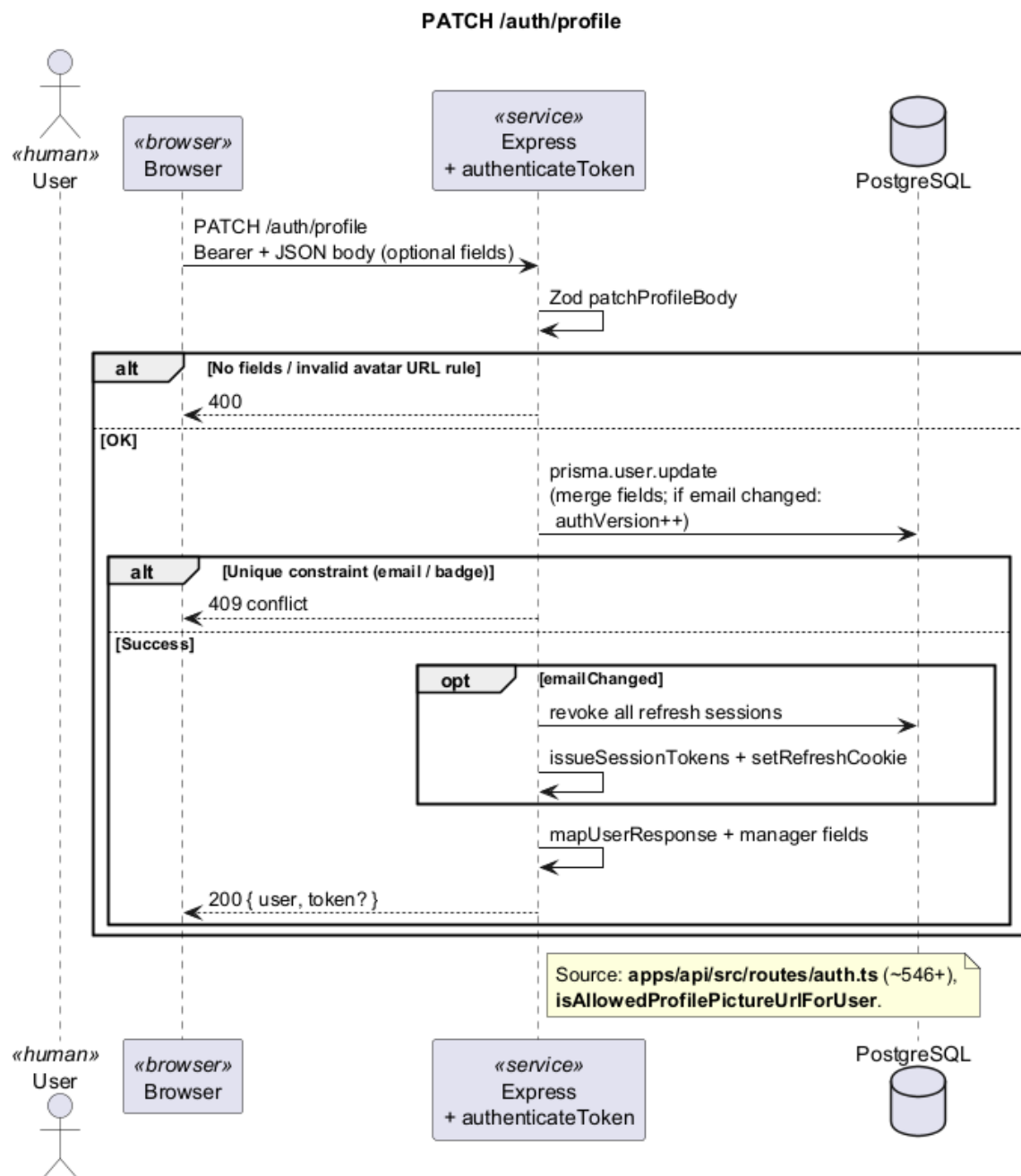


Figure 3.10 – Profile update (PATCH /auth/profile)

Feature restriction enforcement.

Middleware consults per-user feature flags before protected handlers execute; restricted users receive a structured 403 `FEATURE_RESTRICTED` response that the web client maps to a dedicated explanation page.

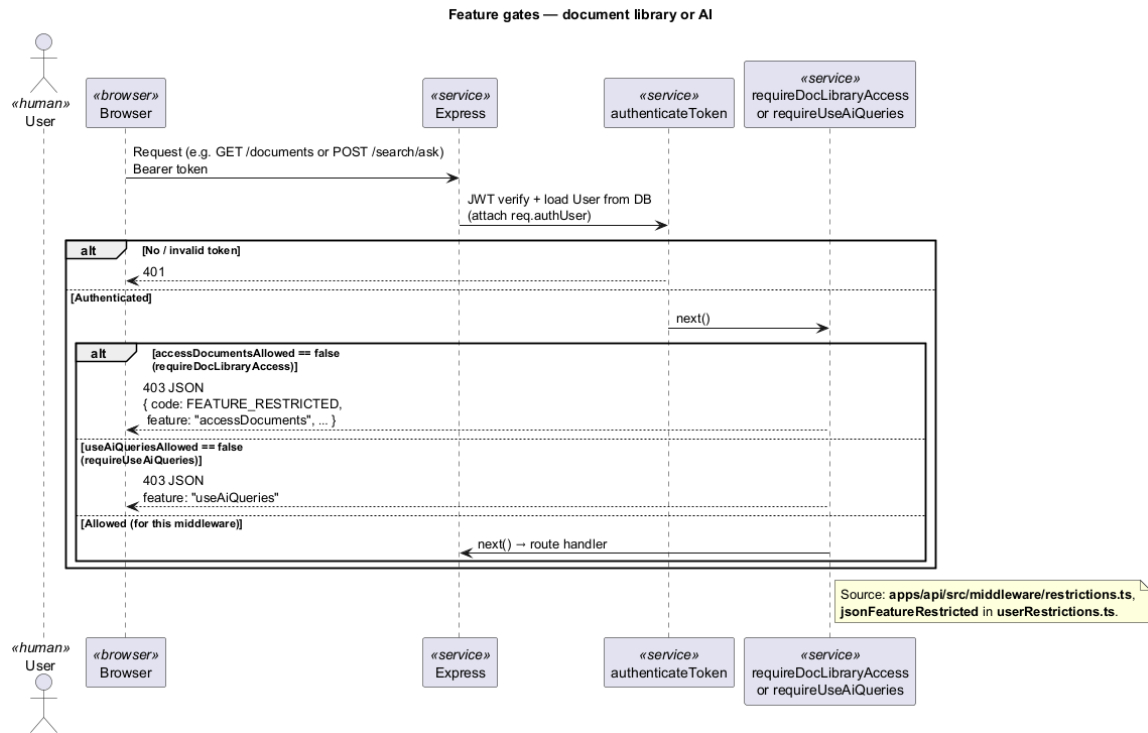


Figure 3.11 – Feature restriction enforcement (403 `FEATURE_RESTRICTED`)

Current user session payload.

The `/me` endpoint returns the authenticated principal, role, department memberships, and active feature restrictions so the client can render navigation and gate UI affordances without additional round trips.

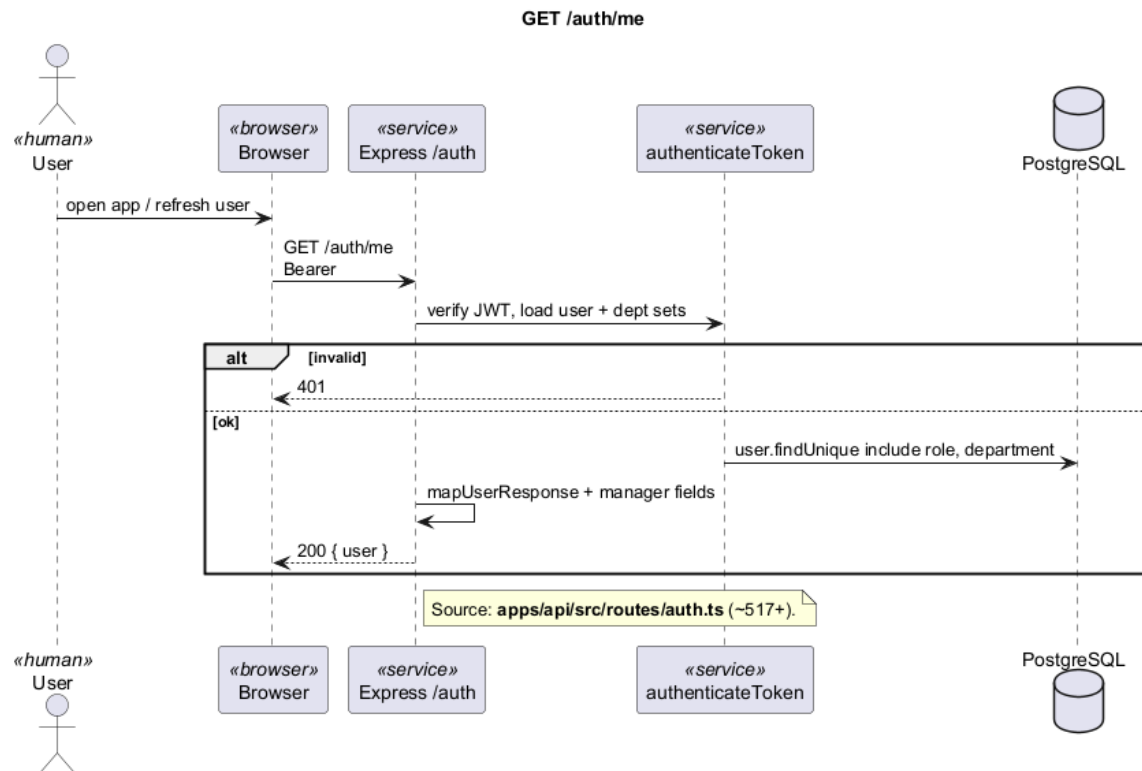


Figure 3.12 – Current user session payload (GET /auth/me)

Guest and restricted UX pages

Register information page (no self-service signup).

Self-service registration is intentionally disabled; the static `/register` page informs visitors that accounts are provisioned by administrators, avoiding orphan users outside the RBAC model.

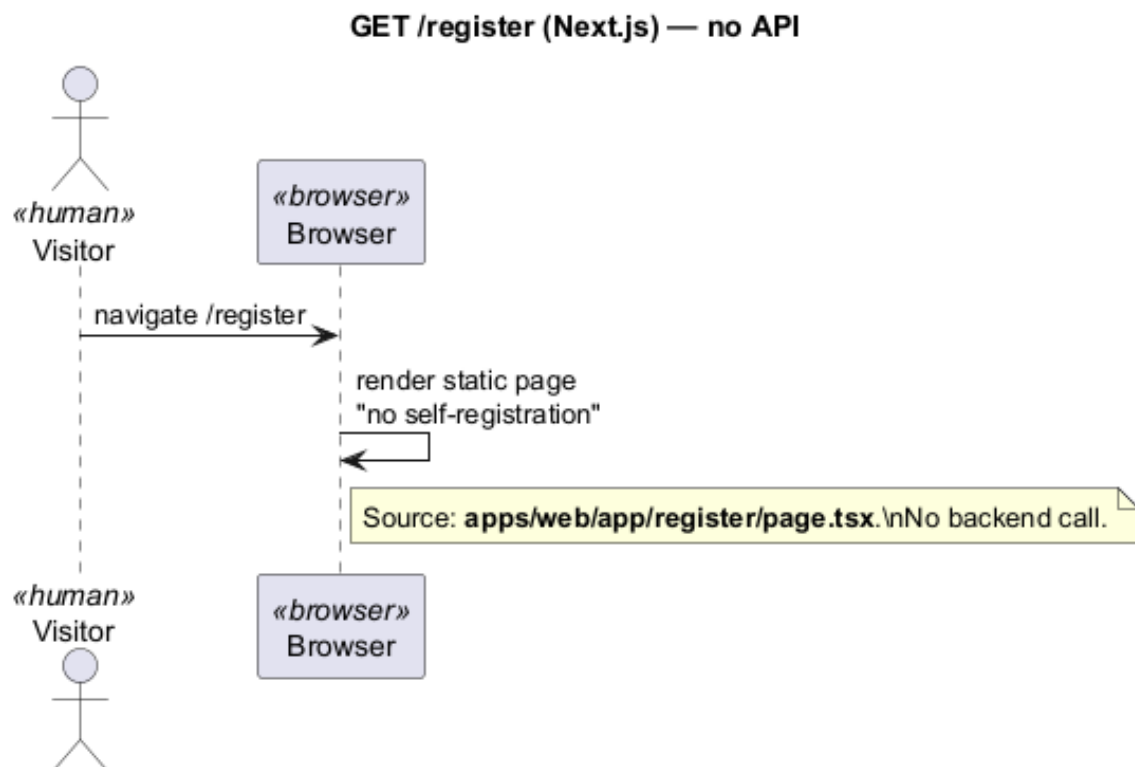


Figure 3.13 – Register information page (no self-service signup) (Next.js /register)

Restricted feature explanation page.

When the API signals a feature restriction, the client redirects to `/restricted`, presenting context-sensitive guidance rather than a generic error banner.

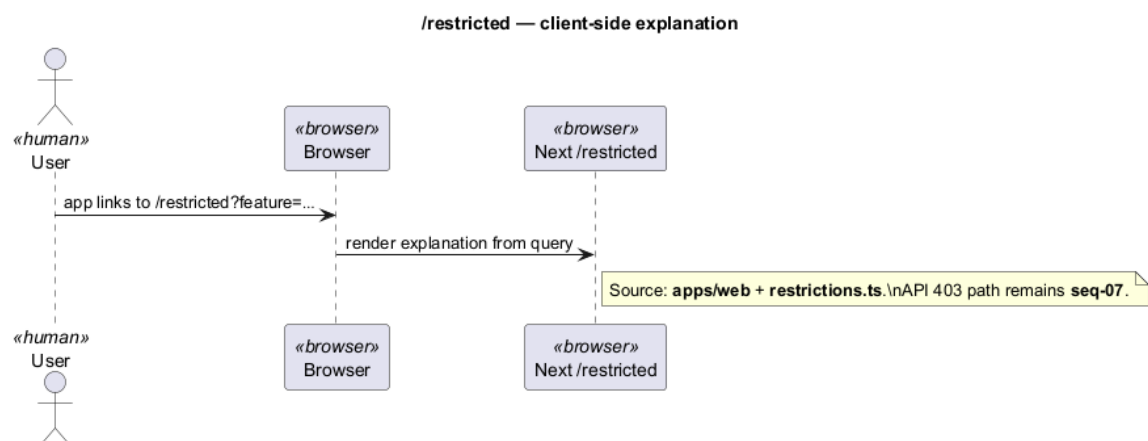


Figure 3.14 – Restricted feature explanation page (`/restricted` UX)

3.5.2 Sprint 2: Document library and ingest

Sprint 2 implements the document corpus lifecycle: multi-format upload and storage (FR-03), department-scoped visibility enforcement (FR-04), and library organisation features such as versioning, tags, favorites, and recents (FR-05). Uploads are accepted synchronously by the API while text extraction, chunking, and embedding run asynchronously in a BullMQ worker.

3.5.2.1 Scoped structural views

Use-case model.

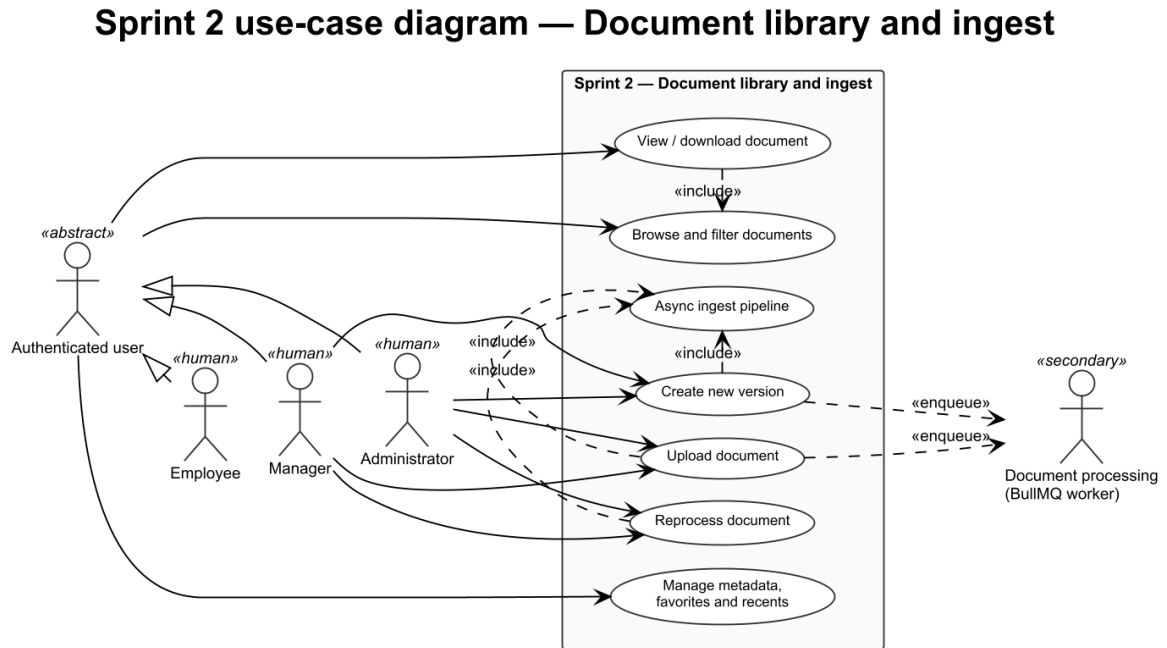


Figure 3.15 – Sprint 2 use-case diagram — Document library and ingest

Domain class model.

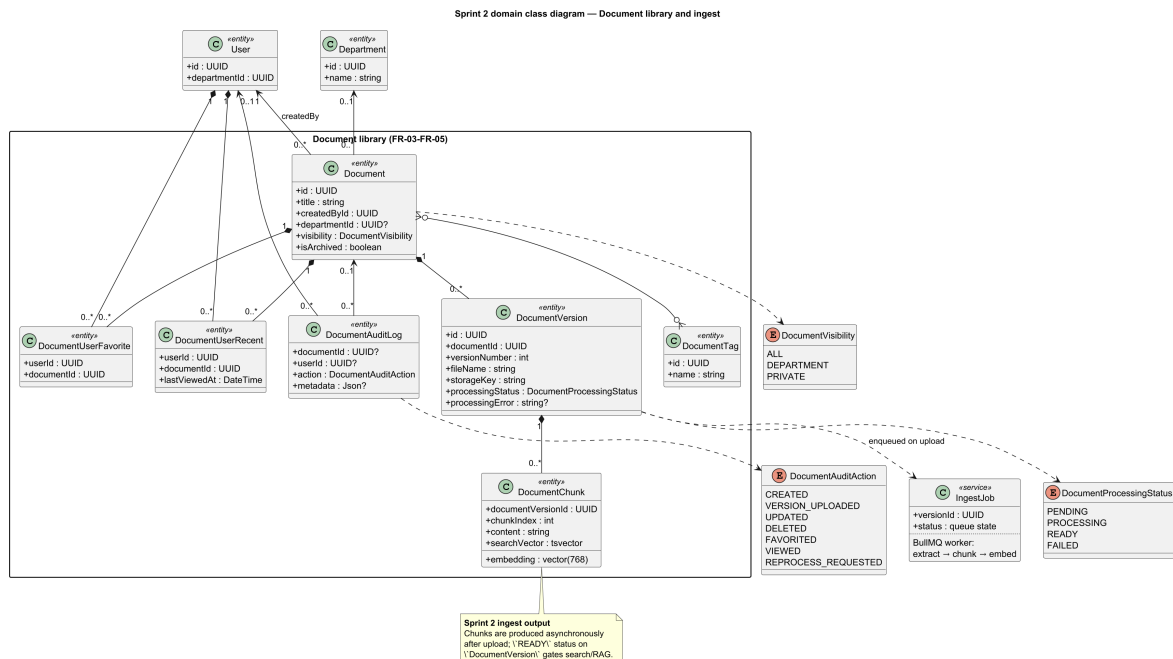


Figure 3.16 – Sprint 2 domain class diagram — Document library and ingest

3.5.2.2 Interaction sequence diagrams

Upload and asynchronous ingest

Document upload.

The upload endpoint validates MIME type and size, writes the binary to disk, creates `Document` and `DocumentVersion` rows, and enqueues an ingest job keyed by version identifier.

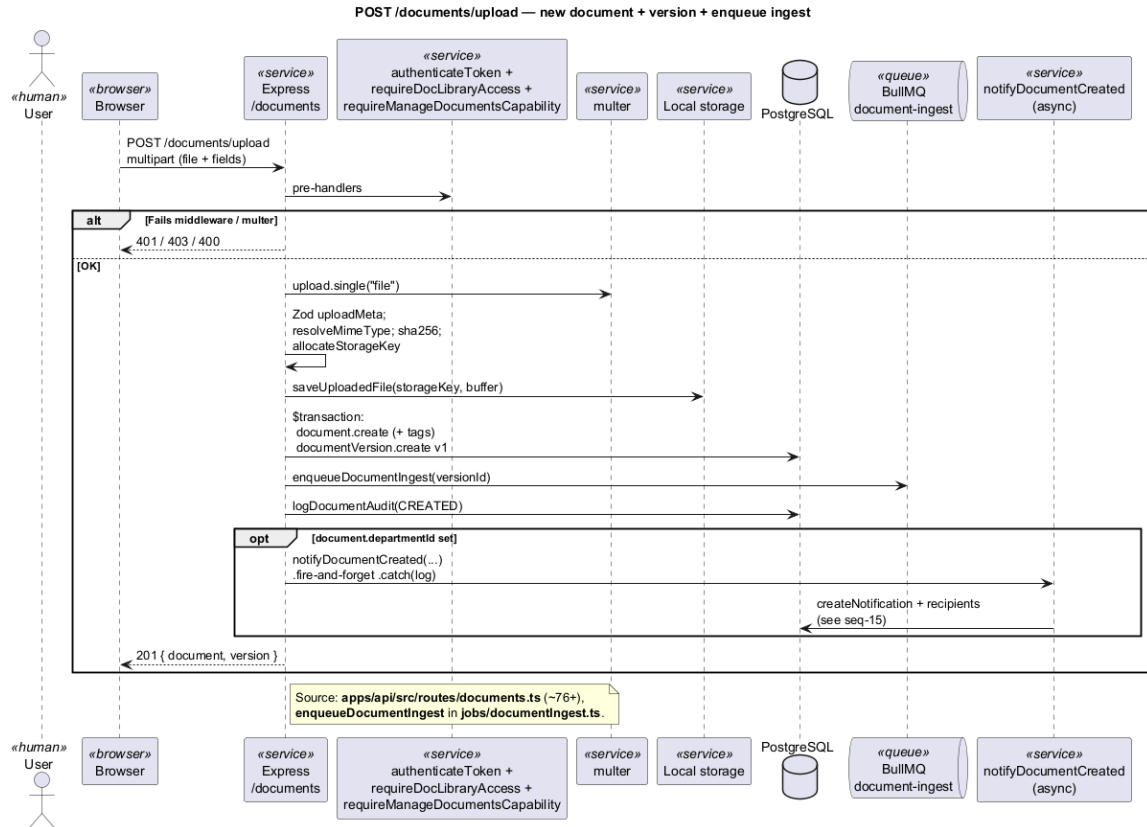


Figure 3.17 – Document upload (POST /documents/upload)

Asynchronous ingest worker.

The BullMQ consumer loads the stored file, extracts text per format, splits content into sentence-aware chunks with table preservation, computes 768-dimensional embeddings, and persists **Chunk** rows indexed for retrieval.

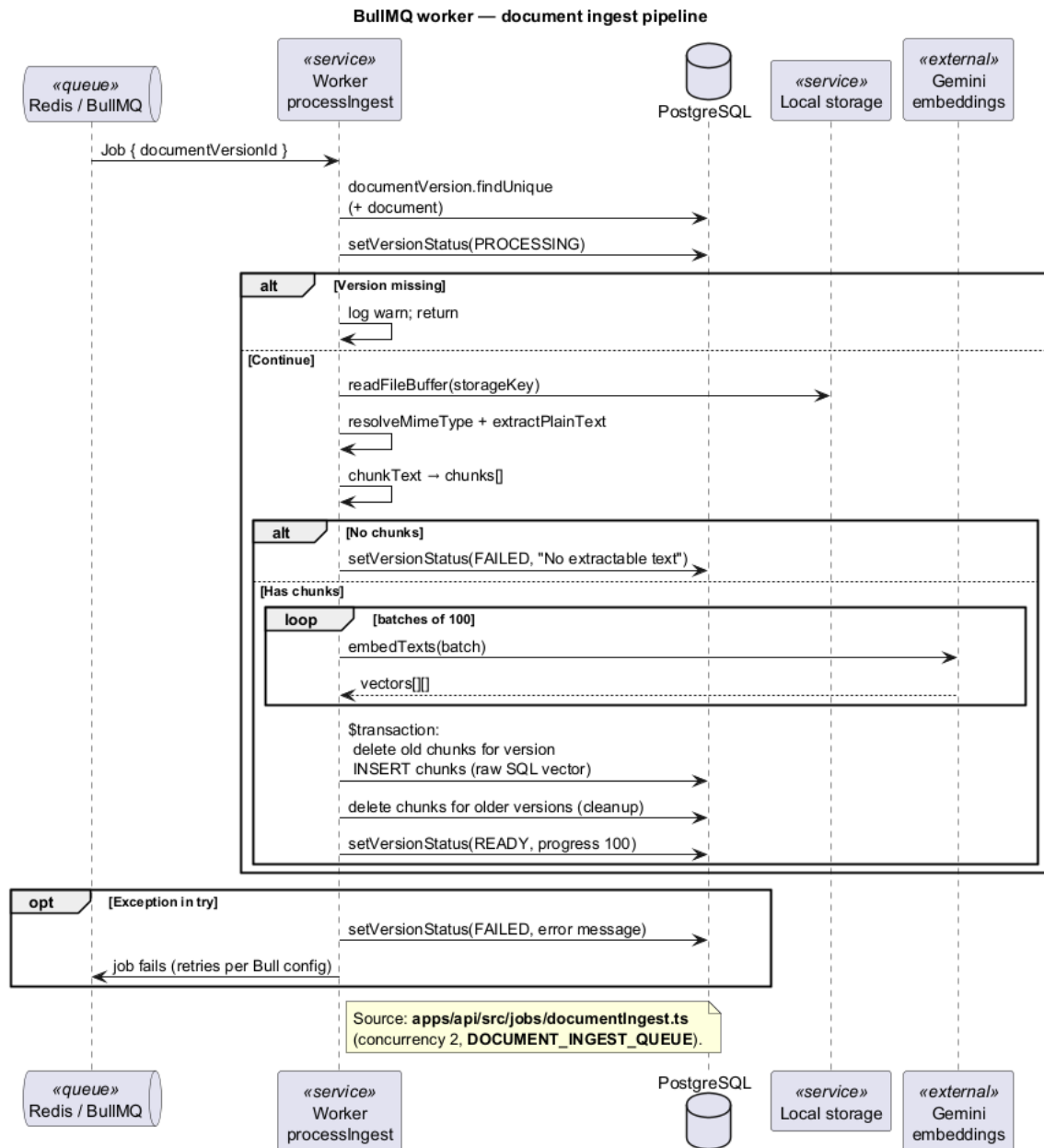


Figure 3.18 – Asynchronous ingest worker (BullMQ `processIngest`)

Reprocess document version.

Administrators can force re-ingestion of an existing version—for example after embedding model upgrades—without re-uploading the binary.

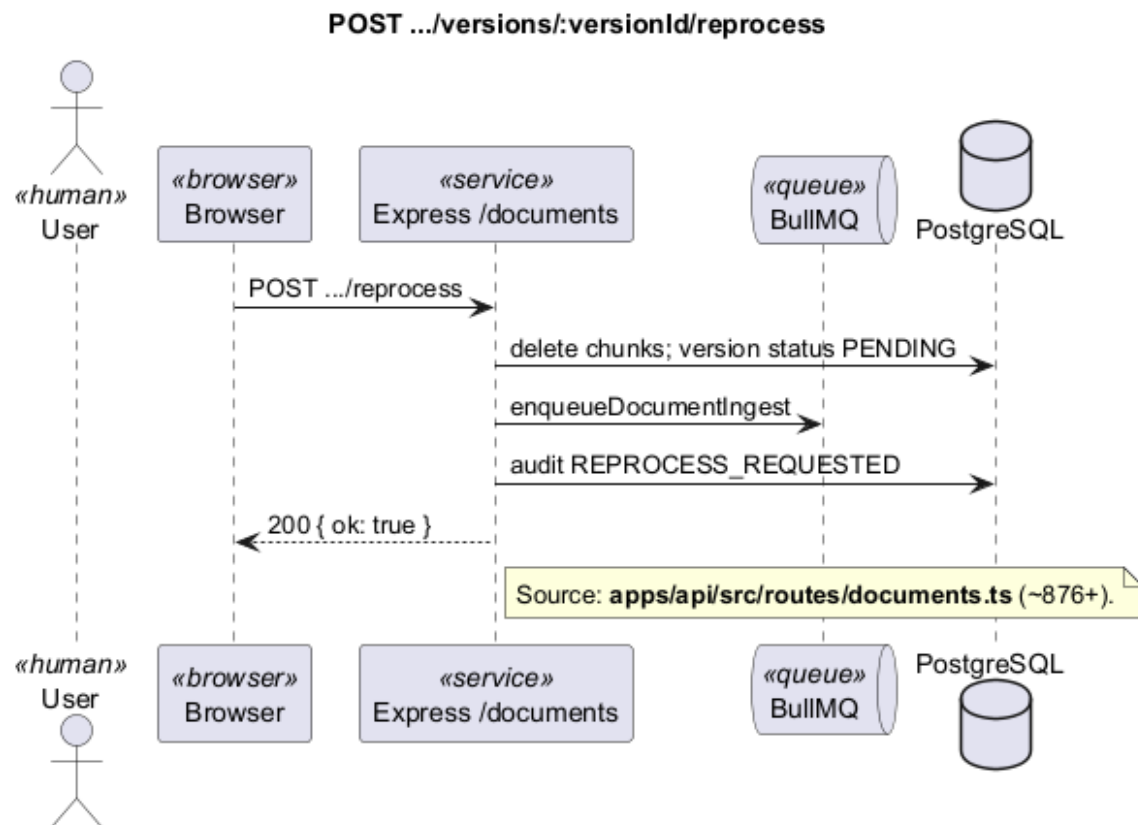


Figure 3.19 – Reprocess document version (POST .../reprocess)

Library access and download

Document file download.

Authorized users download the original file through a version-scoped endpoint that re-checks department visibility before streaming bytes from the storage backend.

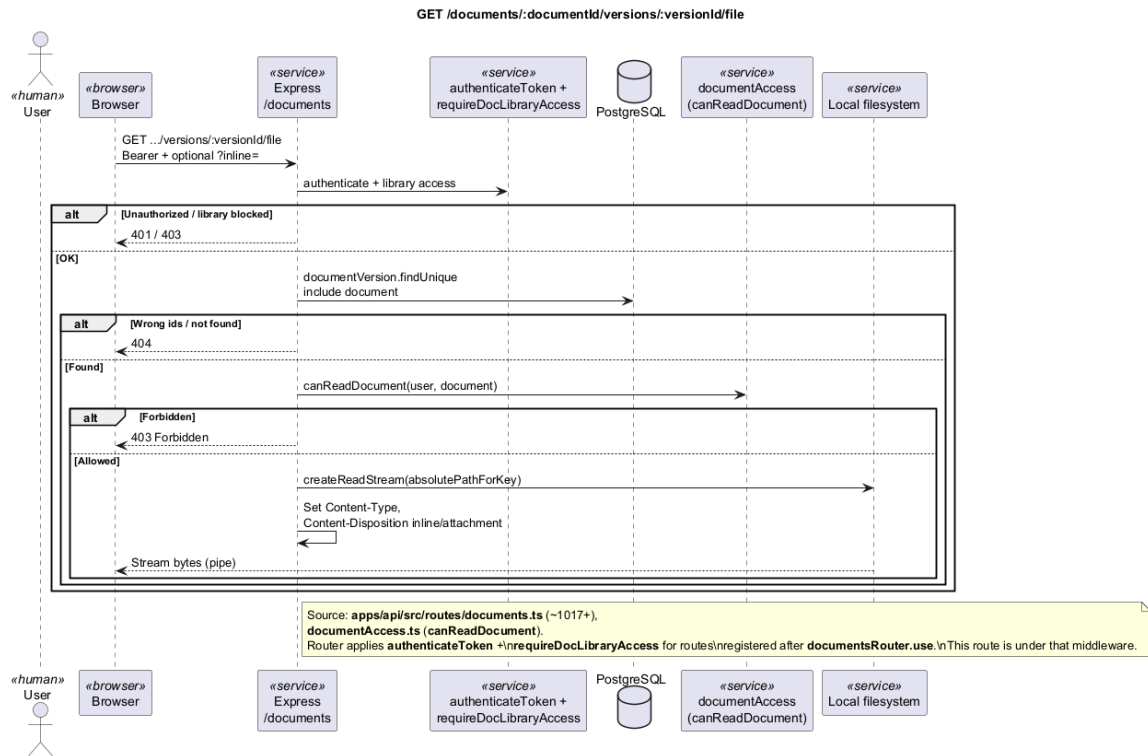


Figure 3.20 – Document file download (GET .../file)

Document list.

The list endpoint applies department filters, search facets, and pagination, returning summary metadata suitable for the library grid without loading chunk payloads.

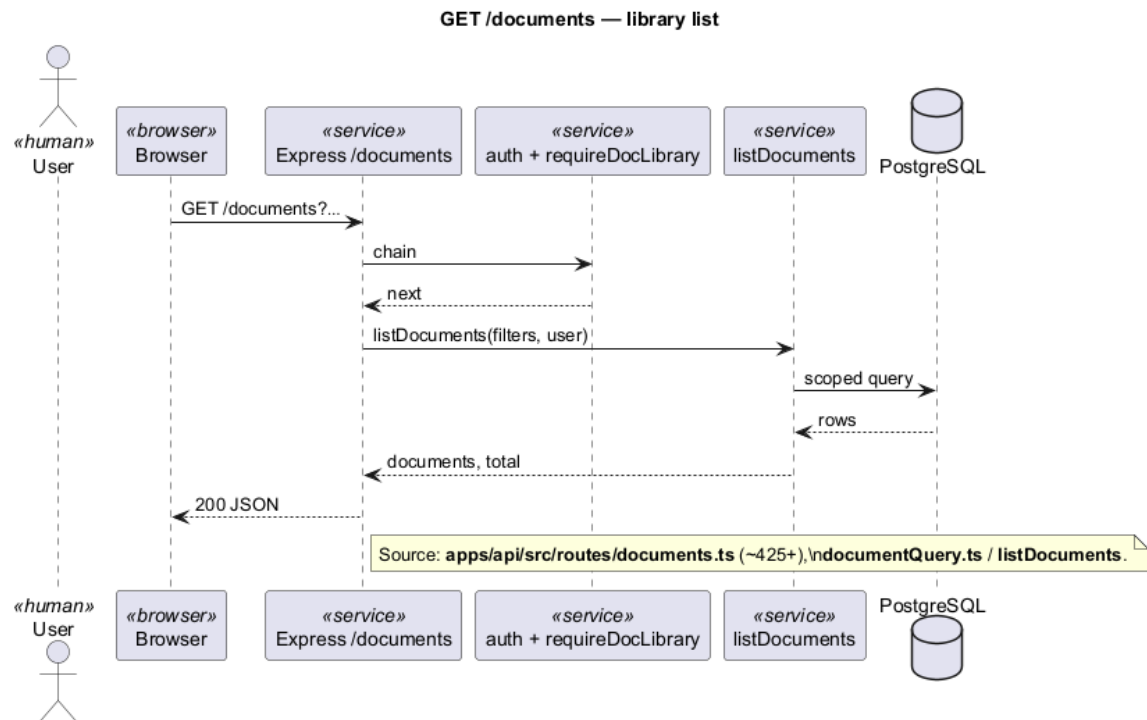


Figure 3.21 – Document list (GET /documents)

Document detail.

Detail retrieval joins document metadata, active version, tags, ingest status, and audit hints, giving the preview pane everything required for inline viewing.

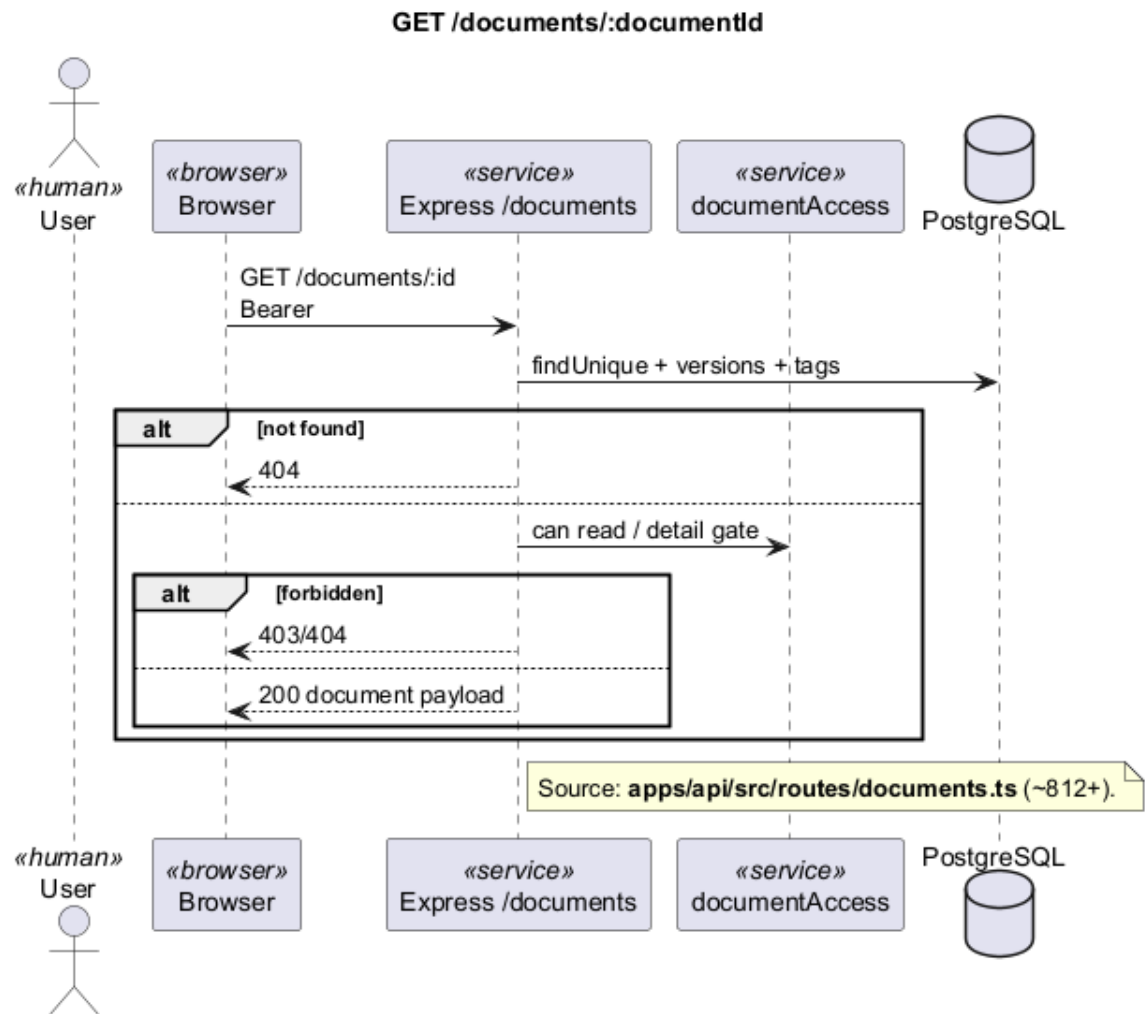


Figure 3.22 – Document detail (`GET /documents/:id`)

Favorites and recents.

Users pin documents to favorites and the API records recent views; both features are scoped to the authenticated principal and surfaced on the library home view.

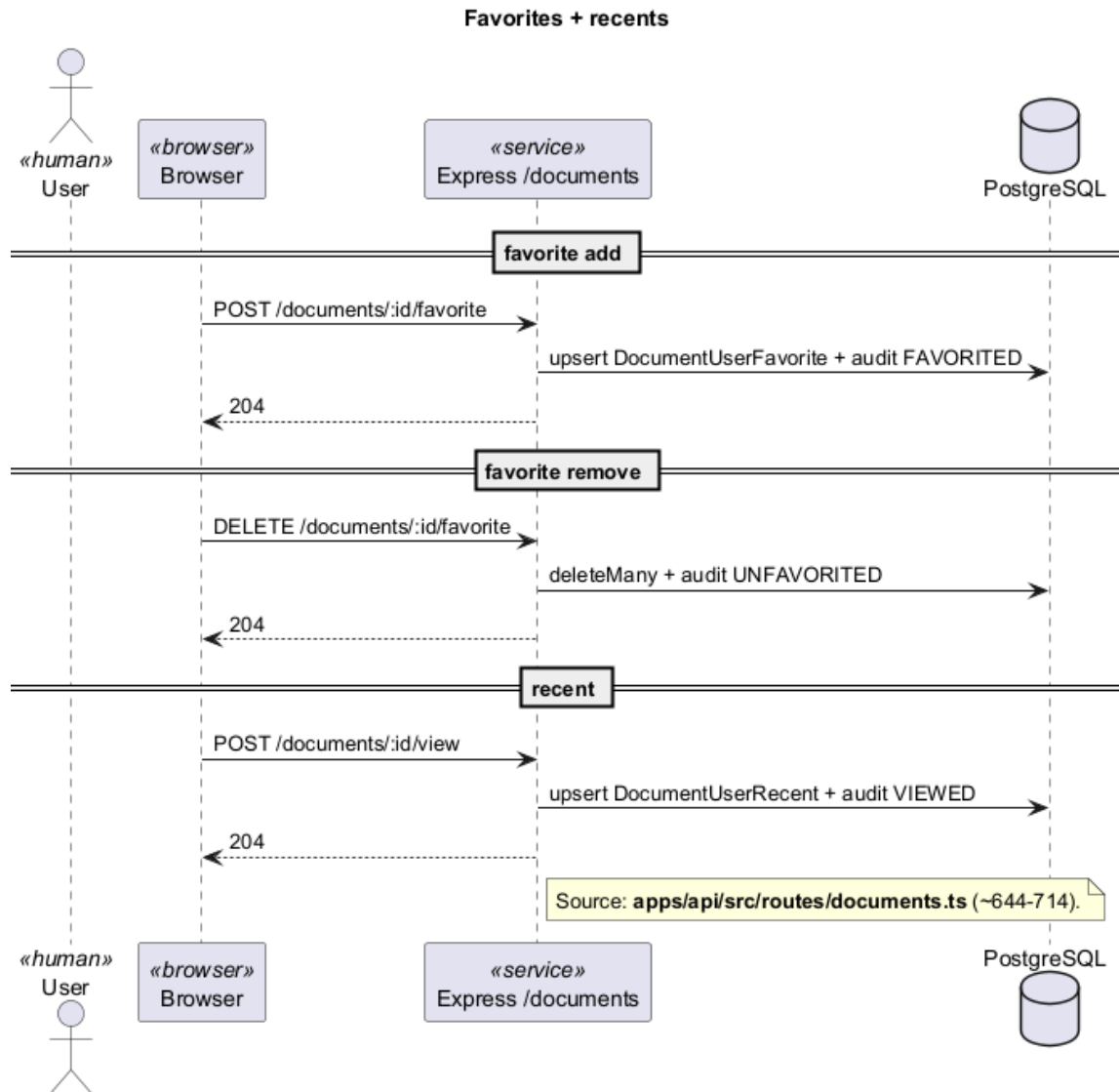


Figure 3.23 – Favorites and recents (POST/DELETE favorite, recent view)

Metadata and versioning

Document metadata update.

Managers and owners may patch title, description, tags, and visibility; changes are validated against writable department scopes and written to the audit trail.

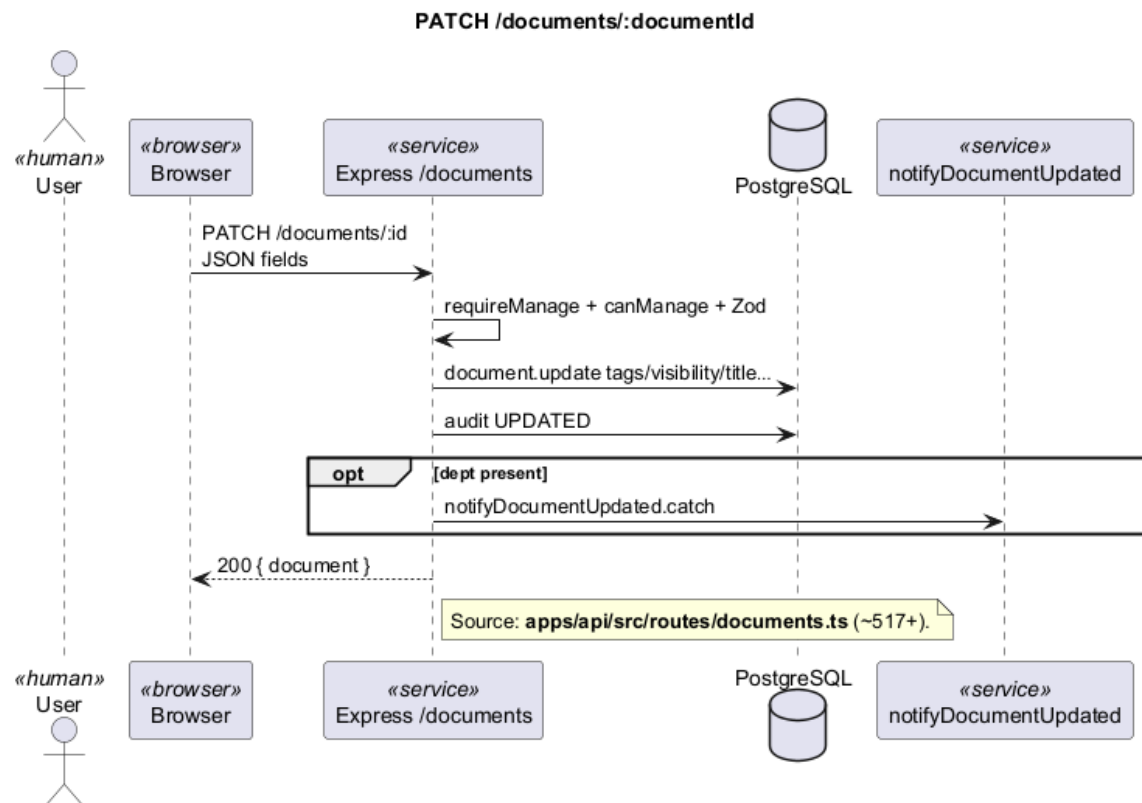


Figure 3.24 – Document metadata update (PATCH /documents/:id)

New document version.

Uploading a replacement file creates a new `DocumentVersion`, triggers a fresh ingest job, and keeps prior versions available for rollback and audit.

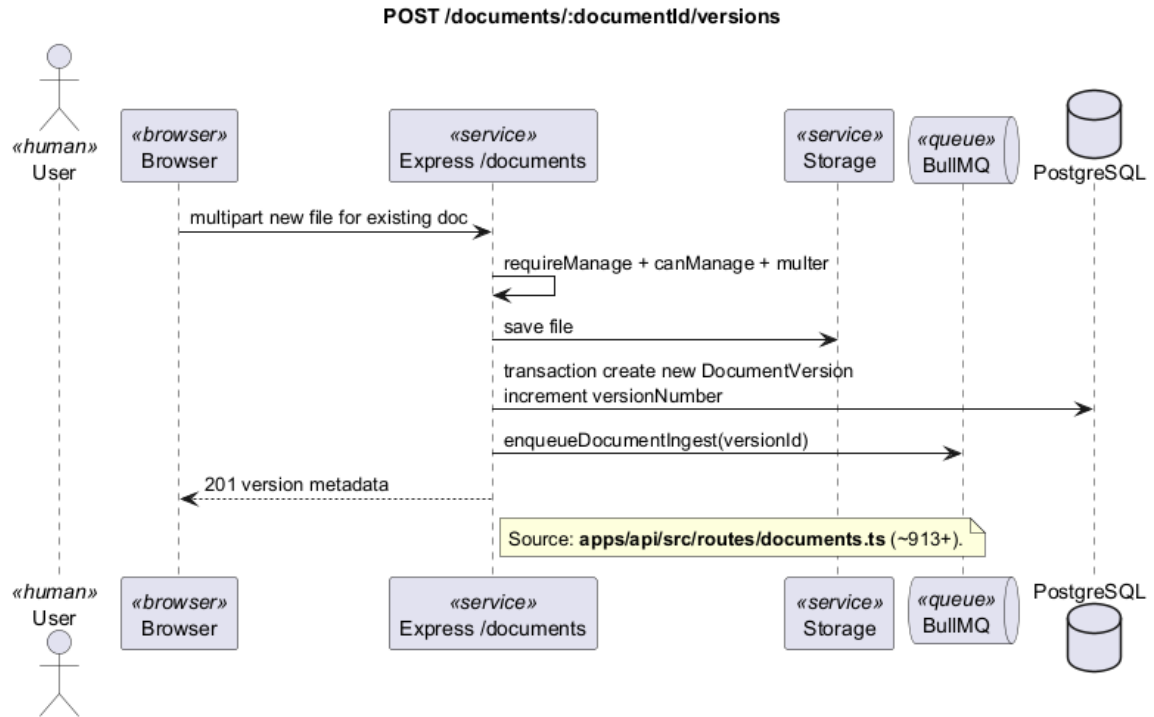


Figure 3.25 – New document version (POST /documents/:id/versions)

3.5.3 Sprint 3: Search, RAG and conversations

Sprint 3 delivers the intelligent retrieval layer: semantic search over embedded chunks (FR-06), hybrid retrieval-augmented generation with streaming answers (FR-07), and persistent conversation threads with feedback (FR-08). The RAG evaluation harness (NFR-06) is introduced in parallel to measure answer quality against golden datasets.

3.5.3.1 Scoped structural views

Use-case model.

Sprint 3 use-case diagram — Search, RAG and conversations

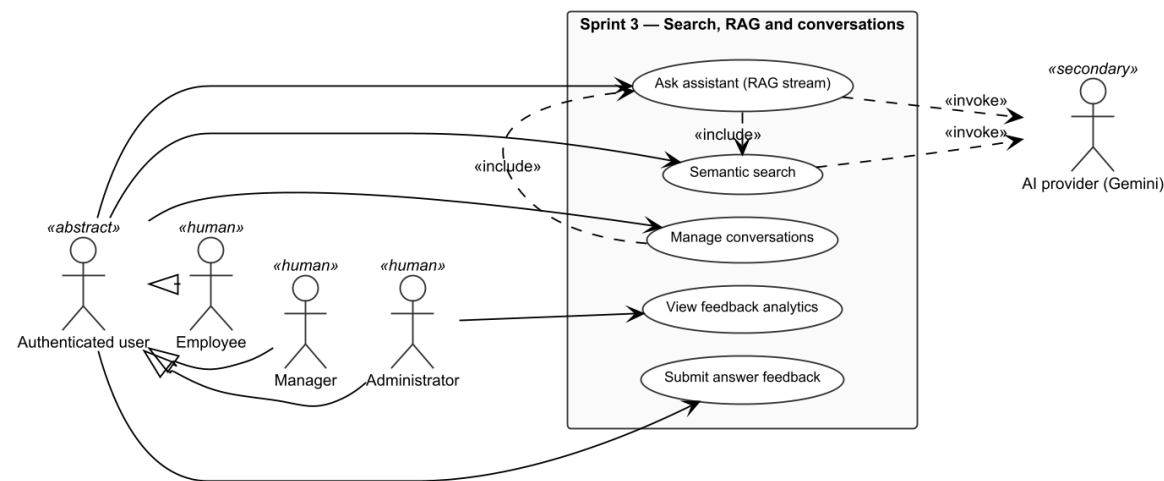


Figure 3.26 – Sprint 3 use-case diagram — Search, RAG and conversations

Domain class model.

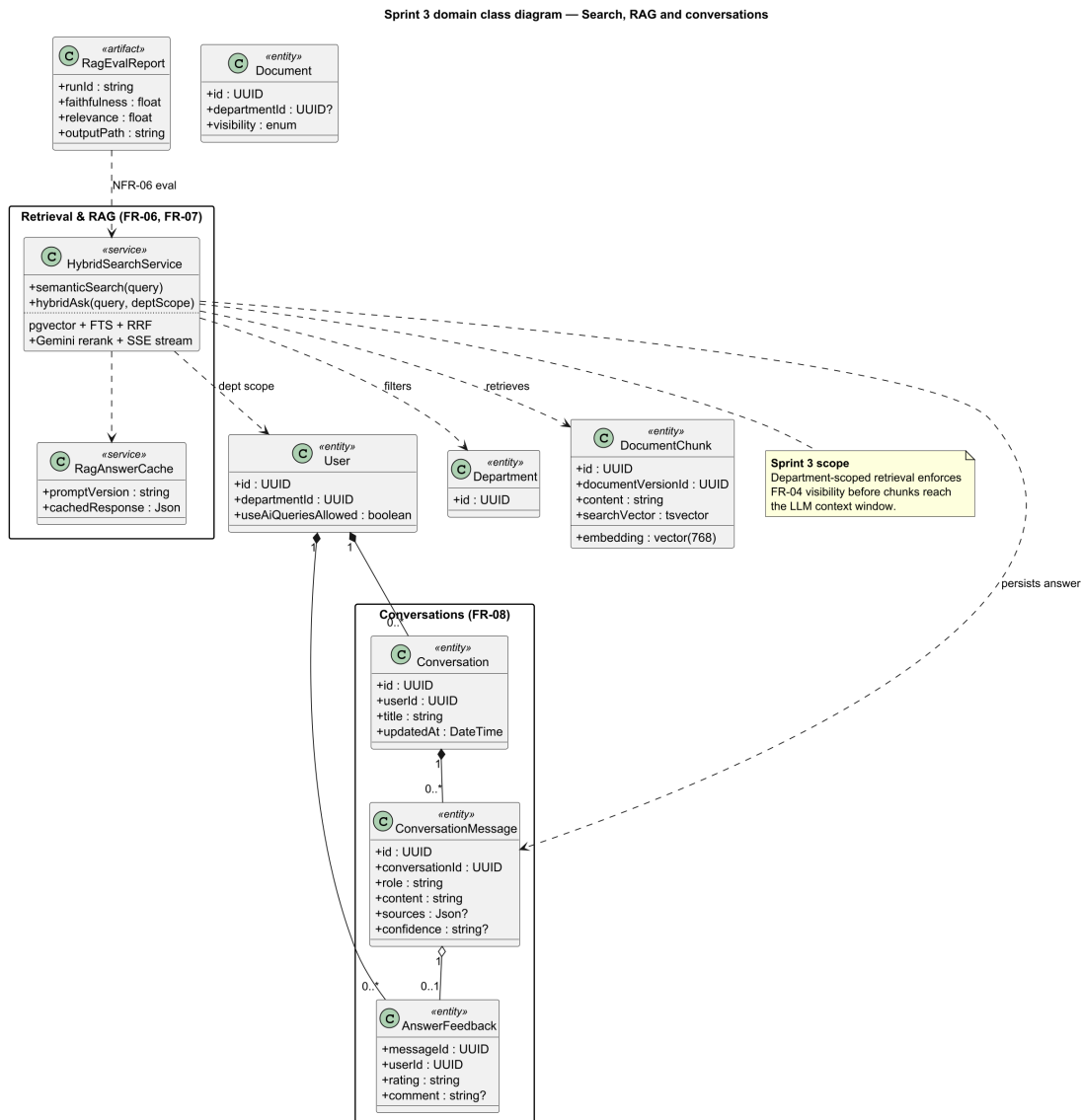


Figure 3.27 – Sprint 3 domain class diagram — Search, RAG and conversations

3.5.3.2 Interaction sequence diagrams

Search and hybrid RAG

Semantic vector search.

The semantic search endpoint embeds the user query, queries pgvector with department filters, and returns ranked chunks with snippet previews for quick exploration outside the chat UI.

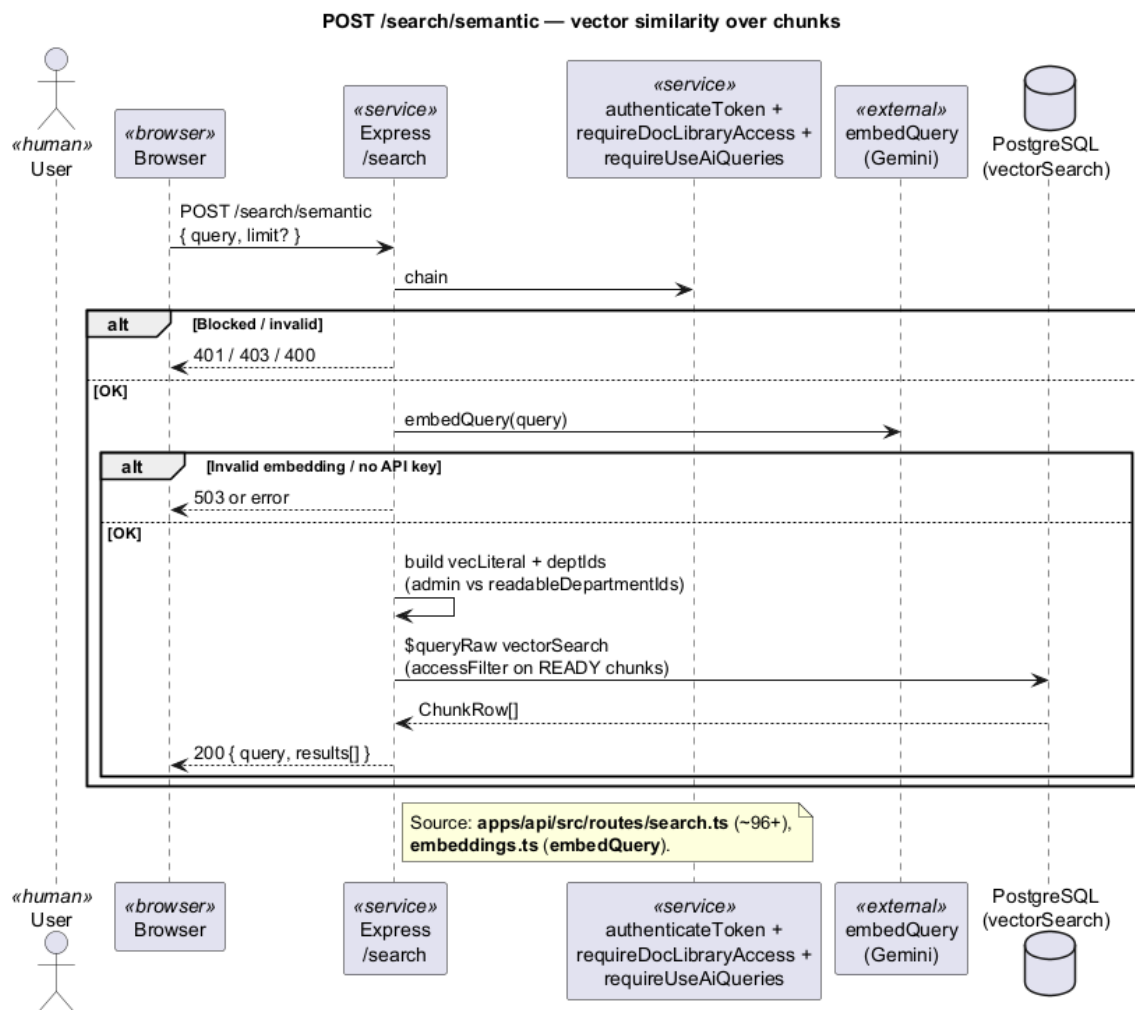


Figure 3.28 – Semantic vector search (POST /search/semantic)

Hybrid RAG Ask with SSE streaming.

The Ask endpoint orchestrates query optimisation, dense and sparse retrieval, reciprocal-rank fusion, LLM reranking, and streams tokens plus source citations to the browser over Server-Sent Events.

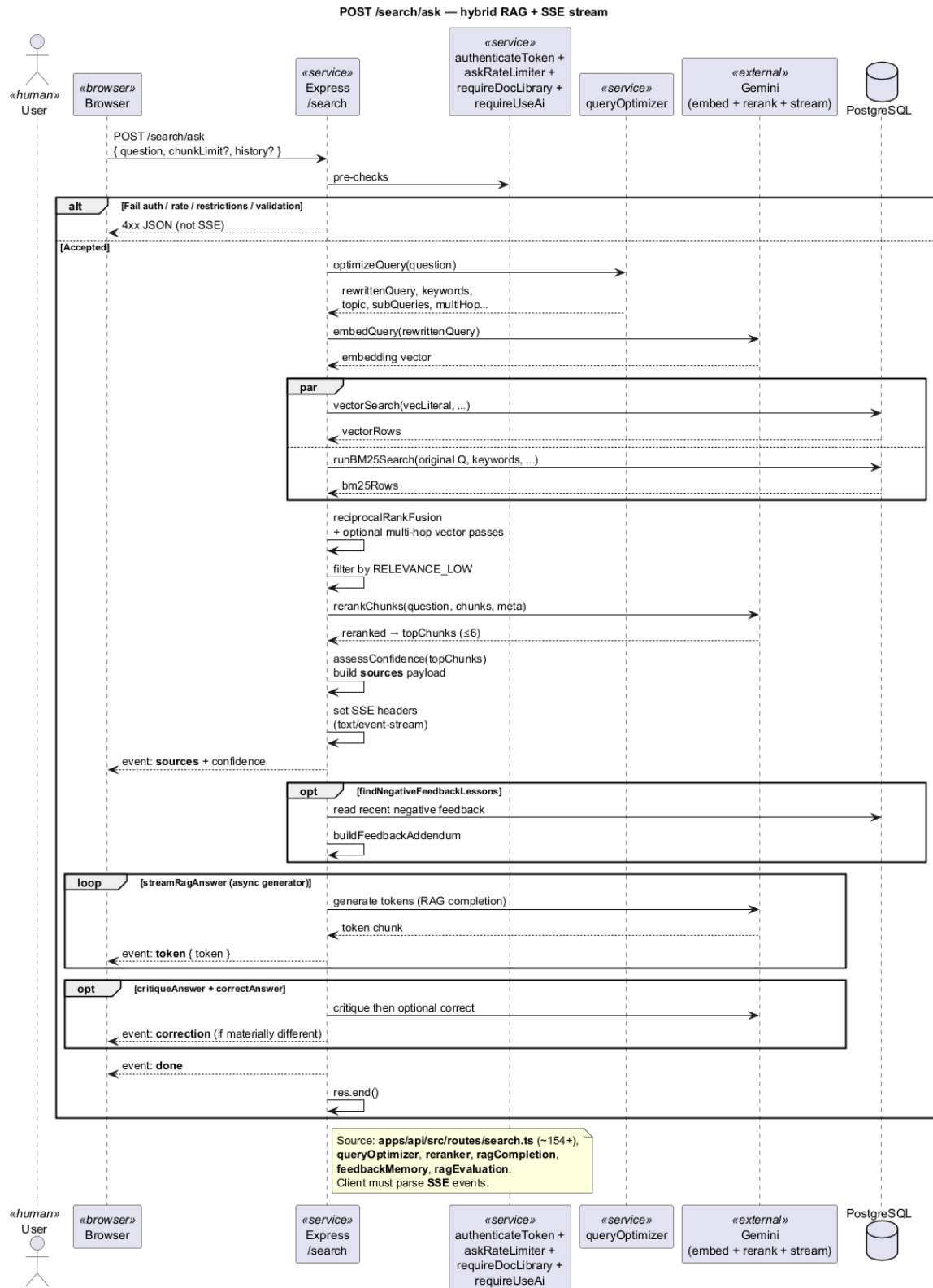


Figure 3.29 – Hybrid RAG Ask with SSE streaming (POST /search/ask)

Conversations and answer quality

Conversation and messages.

Users create conversation threads, append questions, and retrieve message history; each assistant turn stores retrieved sources, confidence metadata, and links back to originating chunks.

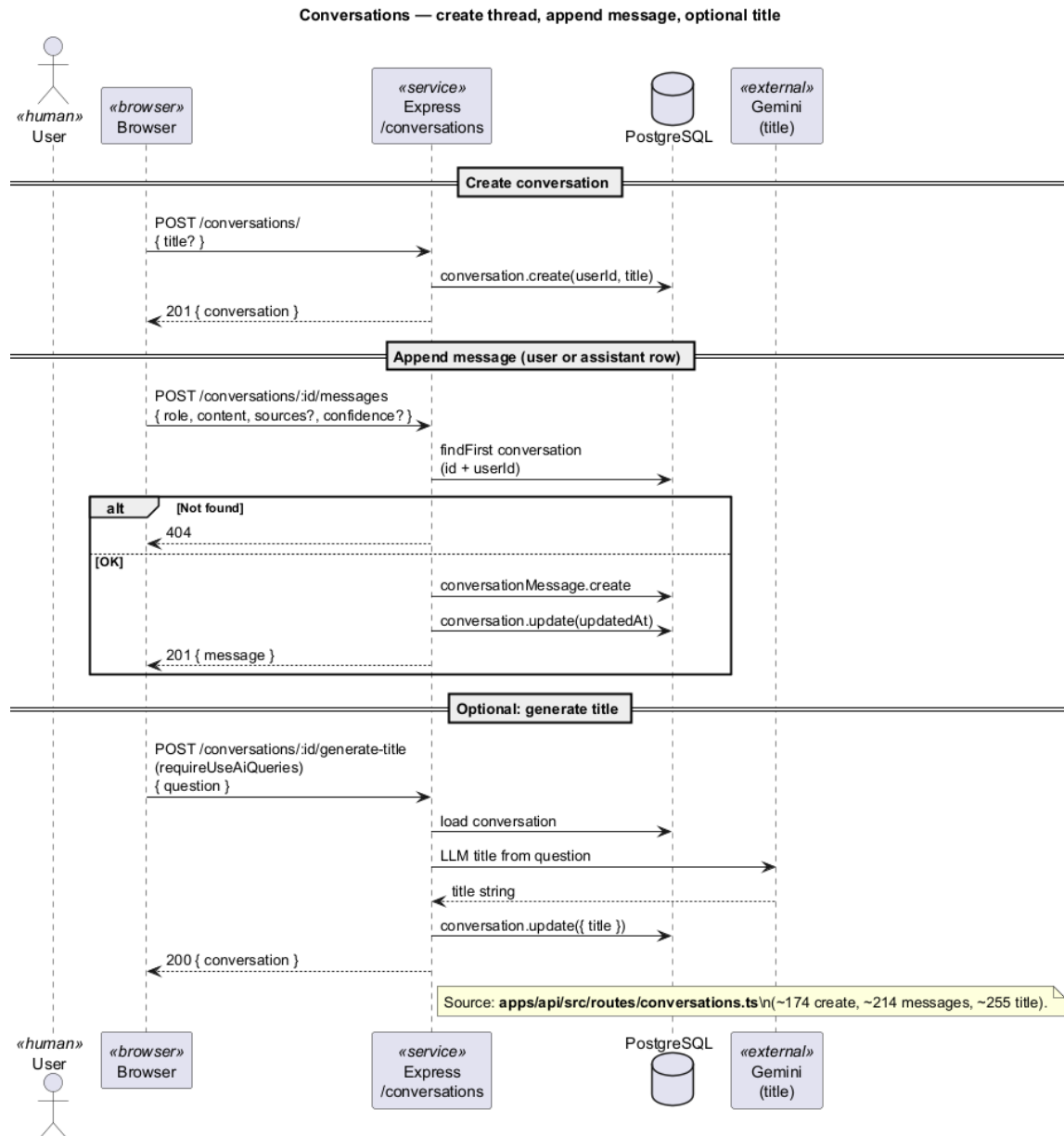


Figure 3.30 – Conversation and messages (/conversations)

Answer feedback.

After each answer, users may submit thumbs-up/down feedback with optional comments; the API persists `AnswerFeedback` rows used later for quality dashboards.

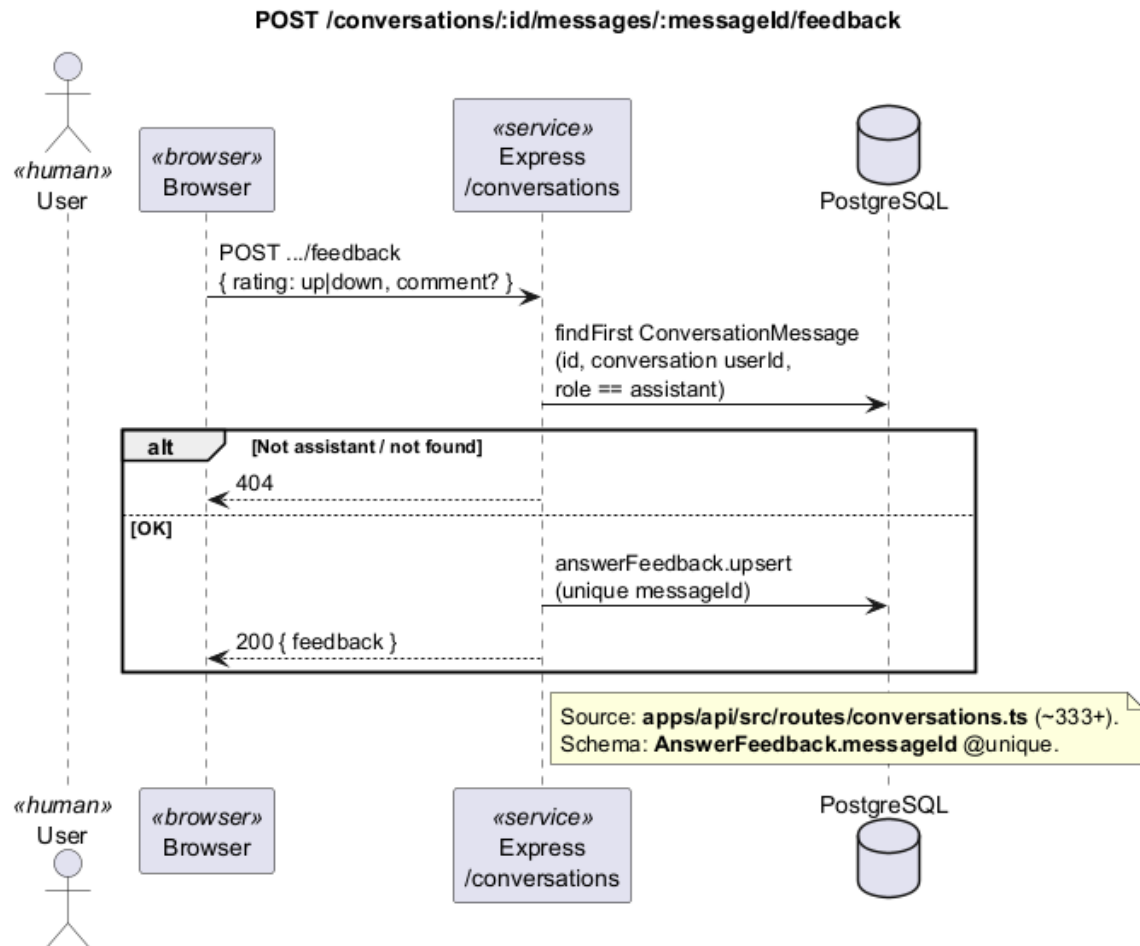


Figure 3.31 – Answer feedback (POST .../feedback)

Conversation CRUD supplement.

Beyond the primary messaging flow, clients list, rename, and delete conversations through supplementary endpoints that keep thread metadata consistent with UI state.

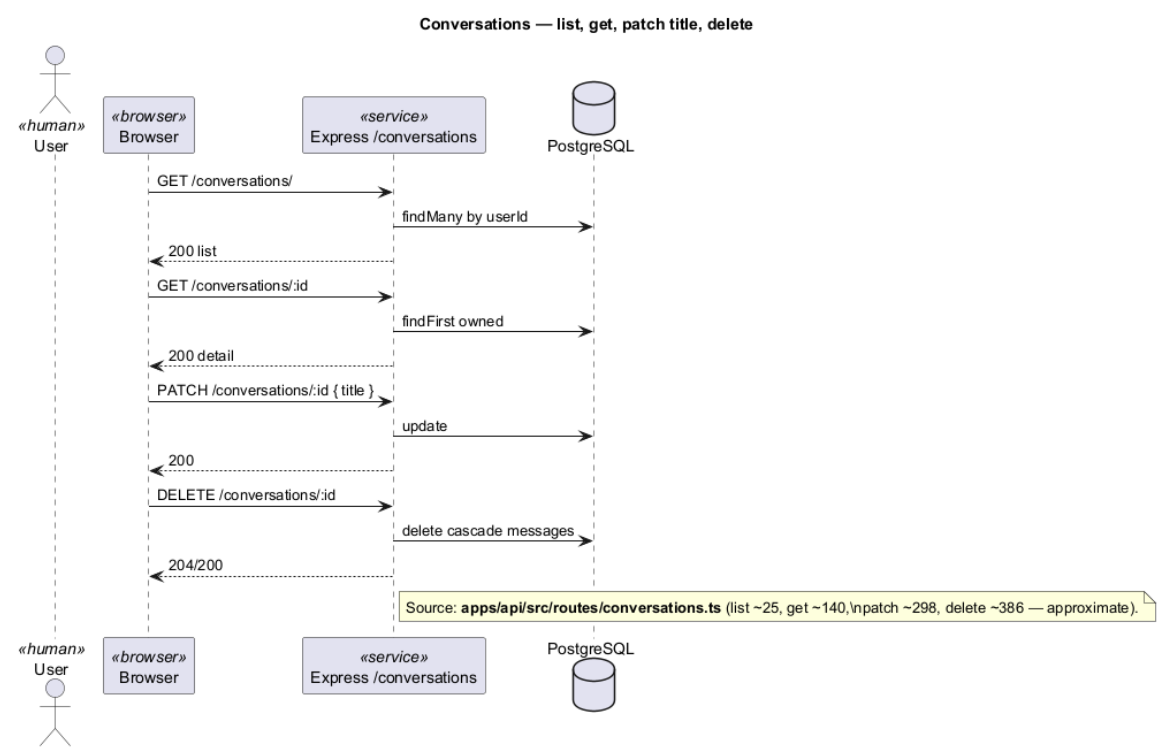


Figure 3.32 – Conversation CRUD supplement (list, patch, delete)

Admin feedback statistics.

Administrators query aggregated feedback metrics—volume, sentiment ratio, and recent comments—to monitor RAG quality trends across departments.

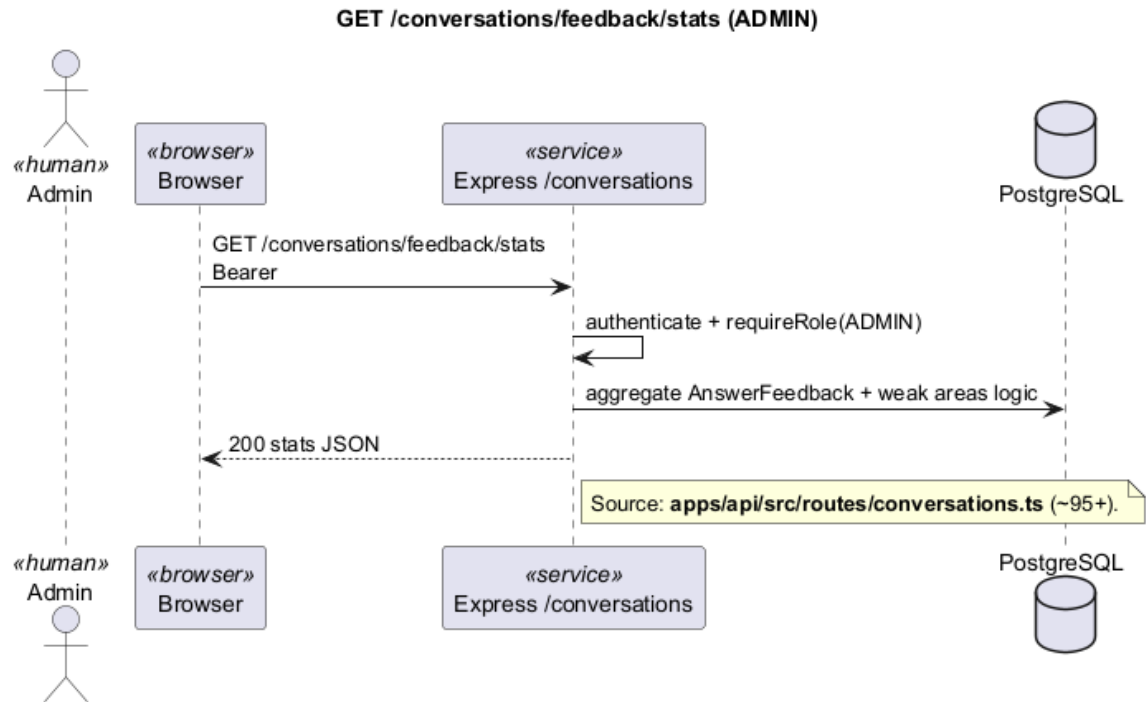


Figure 3.33 – Admin feedback statistics (GET /conversations/feedback/stats)

3.5.4 Sprint 4: Governance and notifications

Sprint 4 covers organisational governance: automatic and manual notifications (FR-09), administrator tooling for users, departments, and bulk operations (FR-10), and the manager department dashboard (FR-11). Notification events are decoupled from document mutations through a service layer, while admin routes enforce elevated RBAC and emit structured audit records for every destructive action.

3.5.4.1 Scoped structural views

Use-case model.

Sprint 4 use-case diagram — Governance and notifications

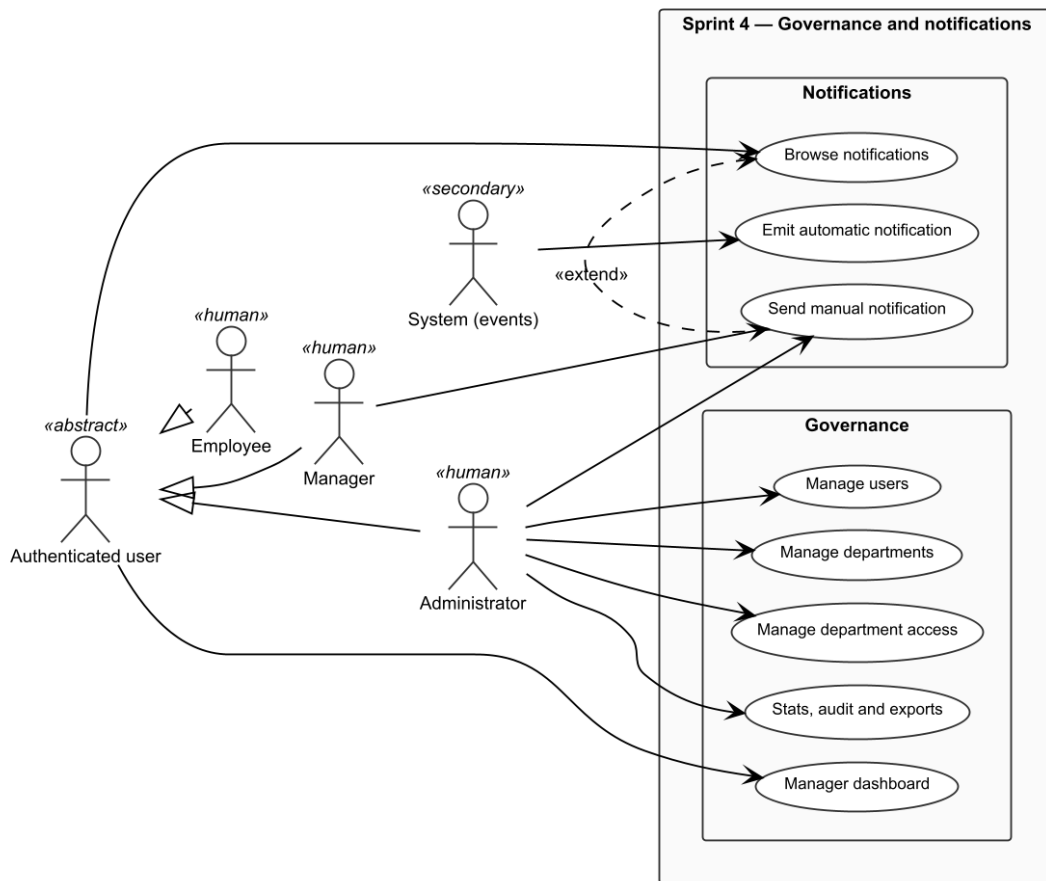


Figure 3.34 – Sprint 4 use-case diagram — Governance and notifications

Domain class model.

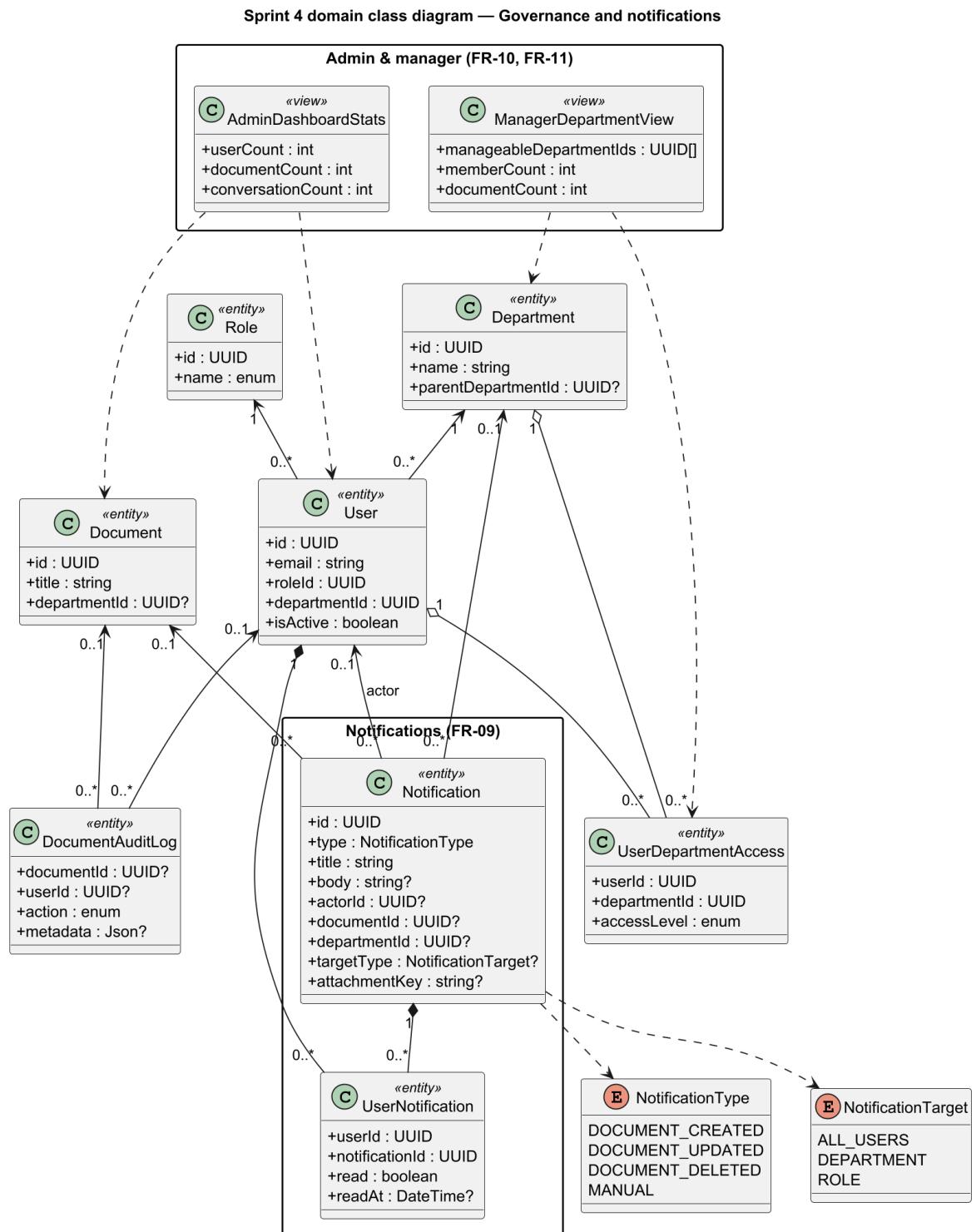


Figure 3.35 – Sprint 4 domain class diagram — Governance and notifications

3.5.4.2 Interaction sequence diagrams

Notification subsystem

Automatic notification on document events.

When documents are created or materially updated, the notification service fans out inbox entries to subscribed department members without blocking the upload response path.

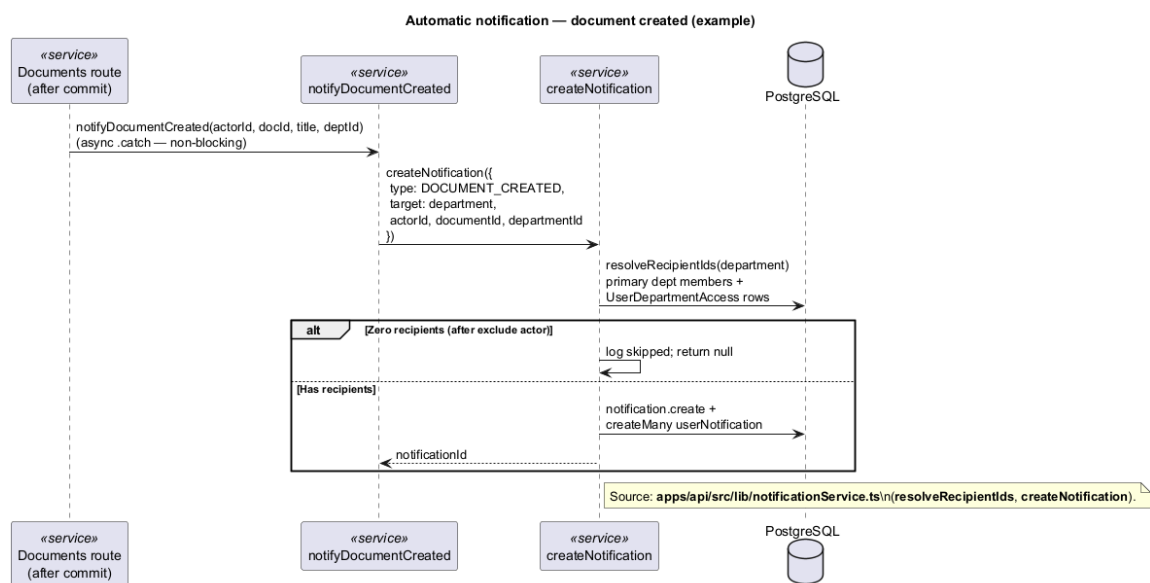


Figure 3.36 – Automatic notification on document events (`notifyDocumentCreated`)

Manual announcement.

Administrators compose announcements with optional attachments; the send endpoint validates audience scope and delivers messages to targeted user inboxes.

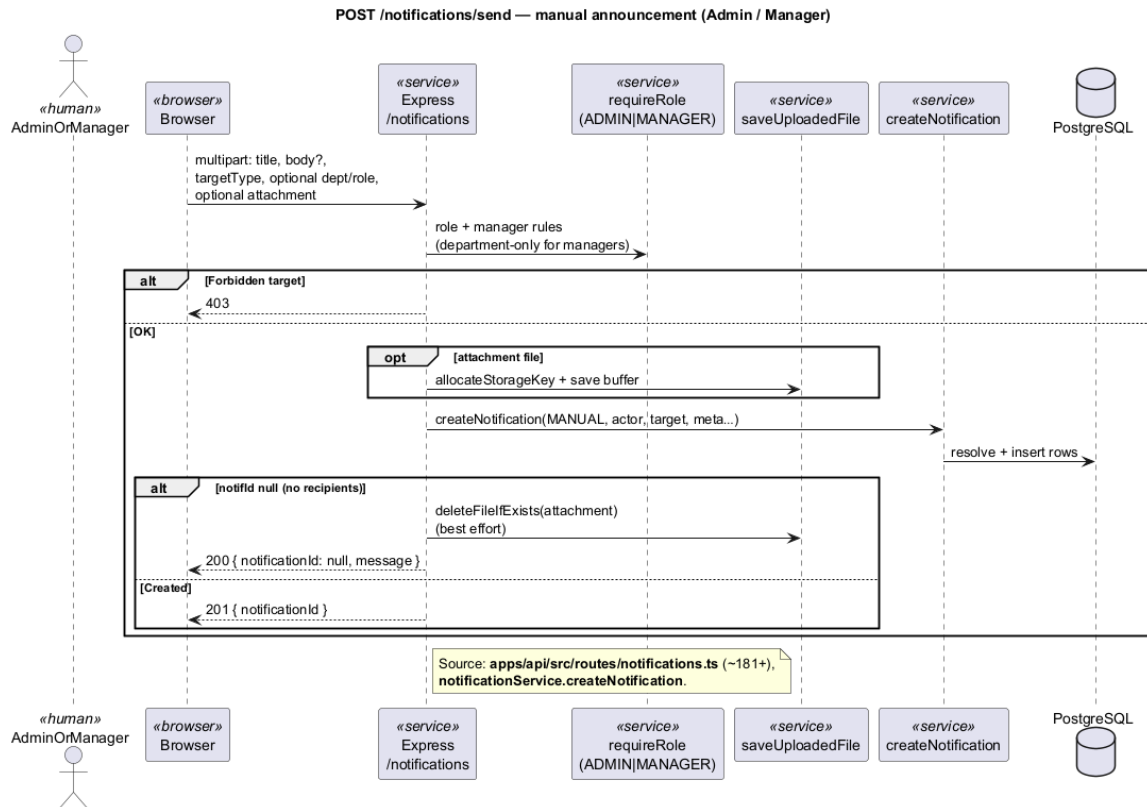


Figure 3.37 – Manual announcement (POST /notifications/send)

Notification inbox.

Users list notifications, query unread counts, mark items as read, and download attachments through REST endpoints polled by the web shell.

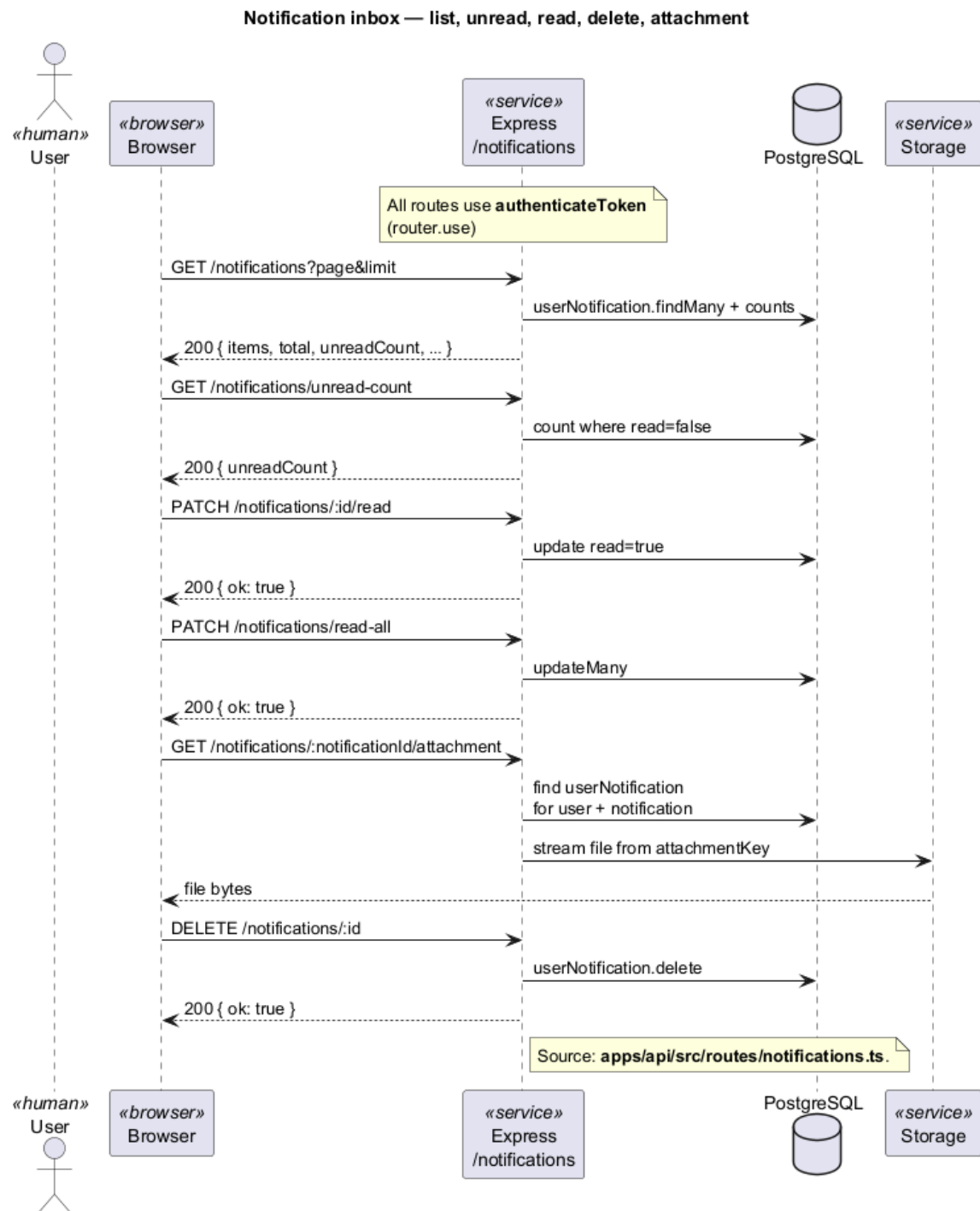


Figure 3.38 – Notification inbox (list, unread, mark read, attachment)

Manager dashboard

Manager department overview.

Managers retrieve department-scoped KPIs—document counts, recent activity, and team membership—through dedicated `/manager` routes filtered to manageable departments.

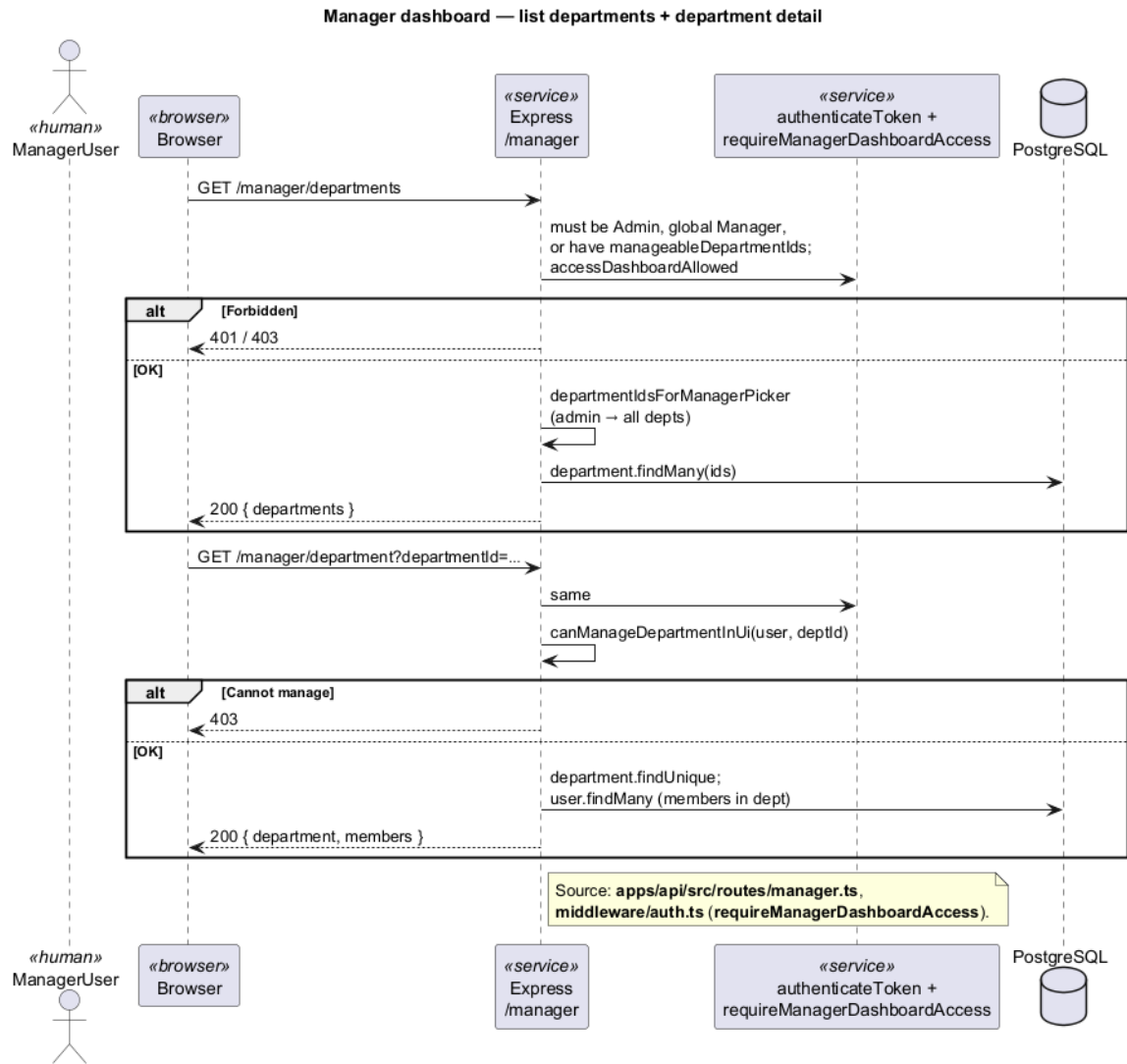


Figure 3.39 – Manager department overview (`GET /manager/*`)

Administrator governance

Admin create user.

Administrators provision accounts with initial roles, department assignments, and optional feature restrictions; credentials are communicated out-of-band per organisational policy.

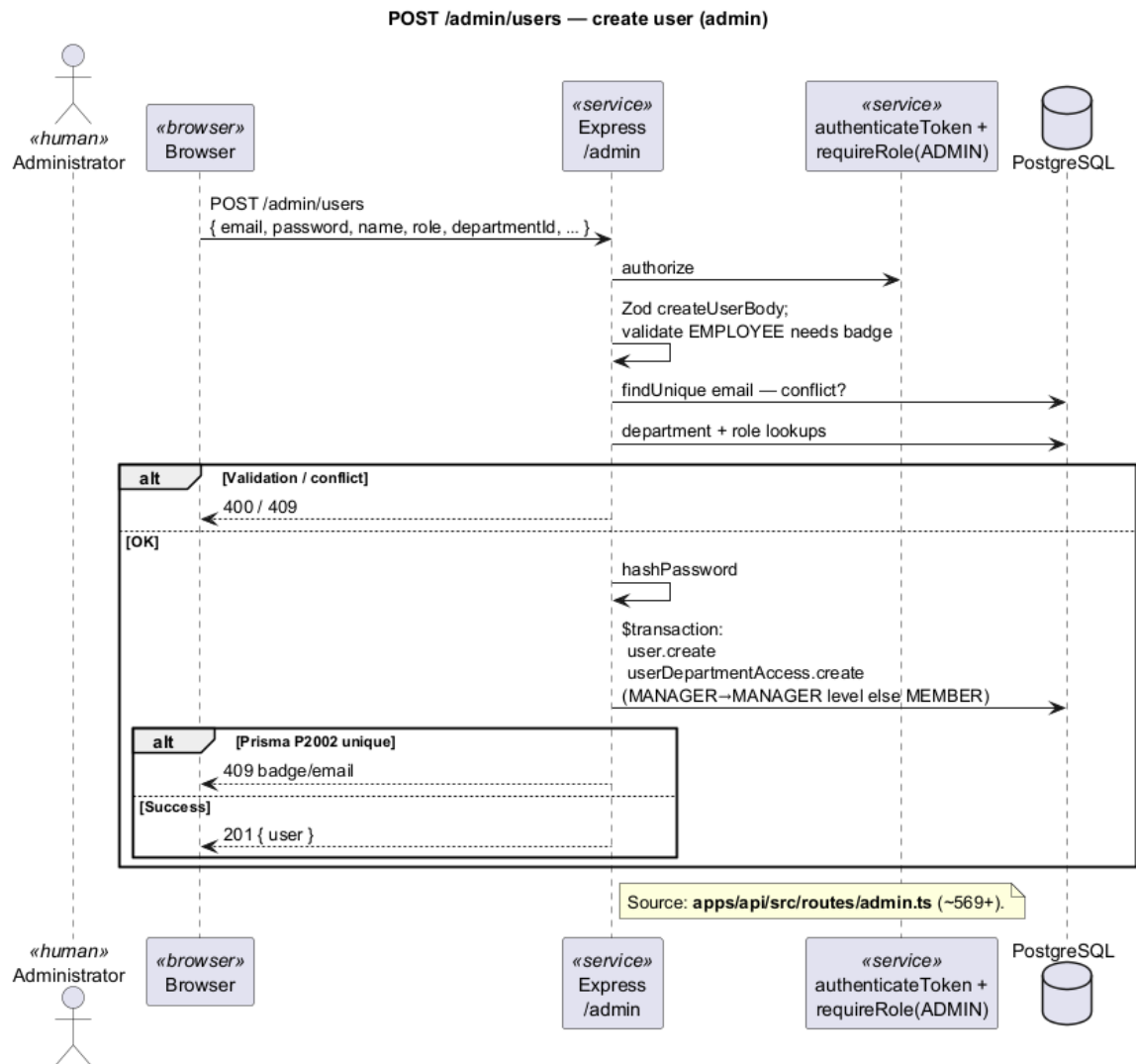


Figure 3.40 – Admin create user (POST /admin/users)

Admin merge departments.

Department merge consolidates documents, access grants, and memberships from a source department into a target, preserving referential integrity inside a transaction.

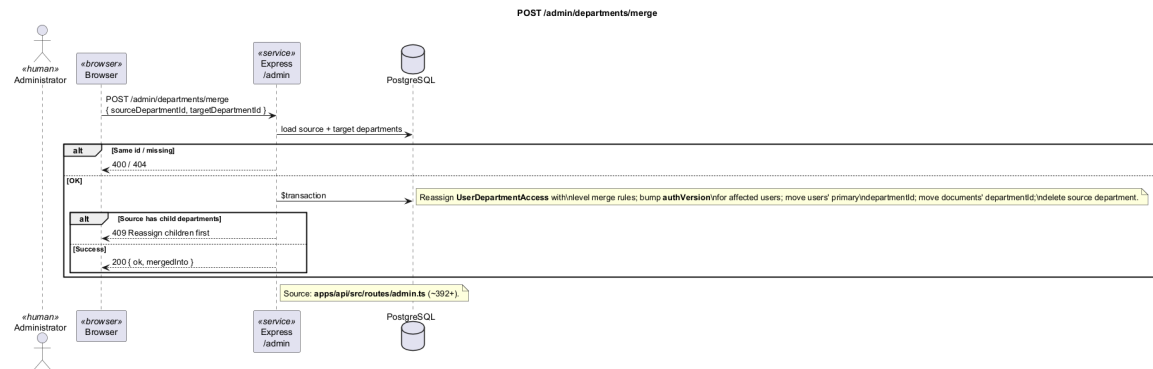


Figure 3.41 – Admin merge departments (POST /admin/departments/merge)

Bulk department access.

Bulk PUT replaces the full access matrix for a user or department in one request, simplifying onboarding scripts and CSV-driven imports.

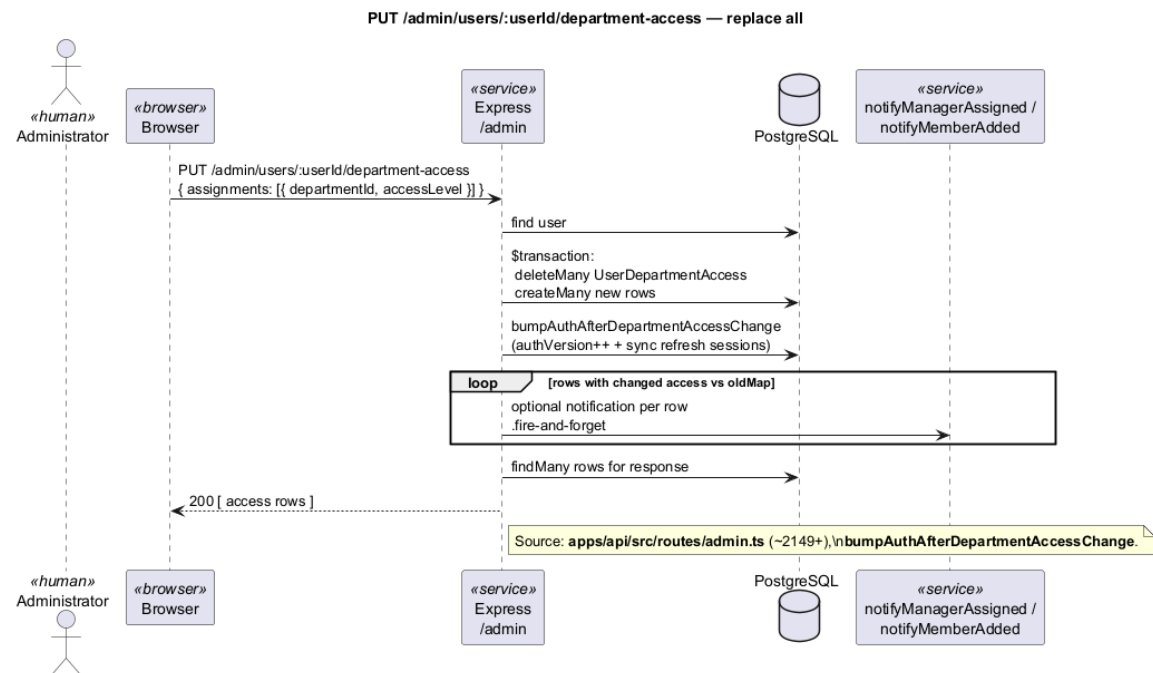


Figure 3.42 – Bulk department access (PUT .../department-access)

Admin create department.

New organisational units are created with unique slugs; the operation is restricted to administrators and is logged for compliance review.

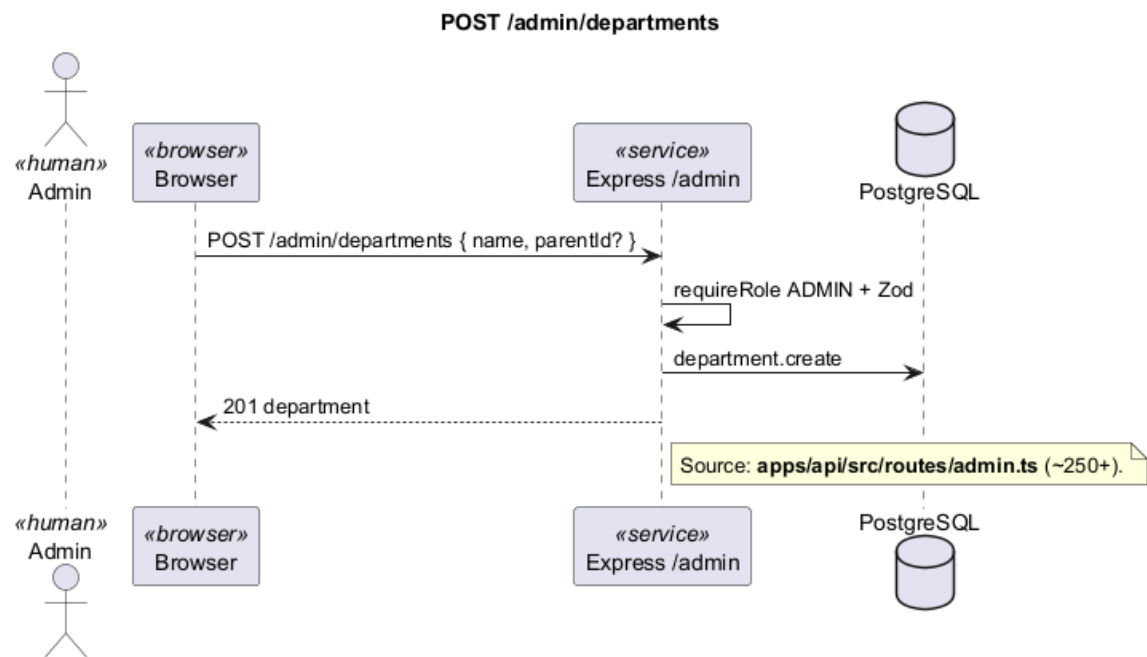


Figure 3.43 – Admin create department (POST /admin/departments)

Single department access grant/revoke.

Fine-grained POST and DELETE endpoints add or remove individual department access rows without rewriting the entire access matrix.

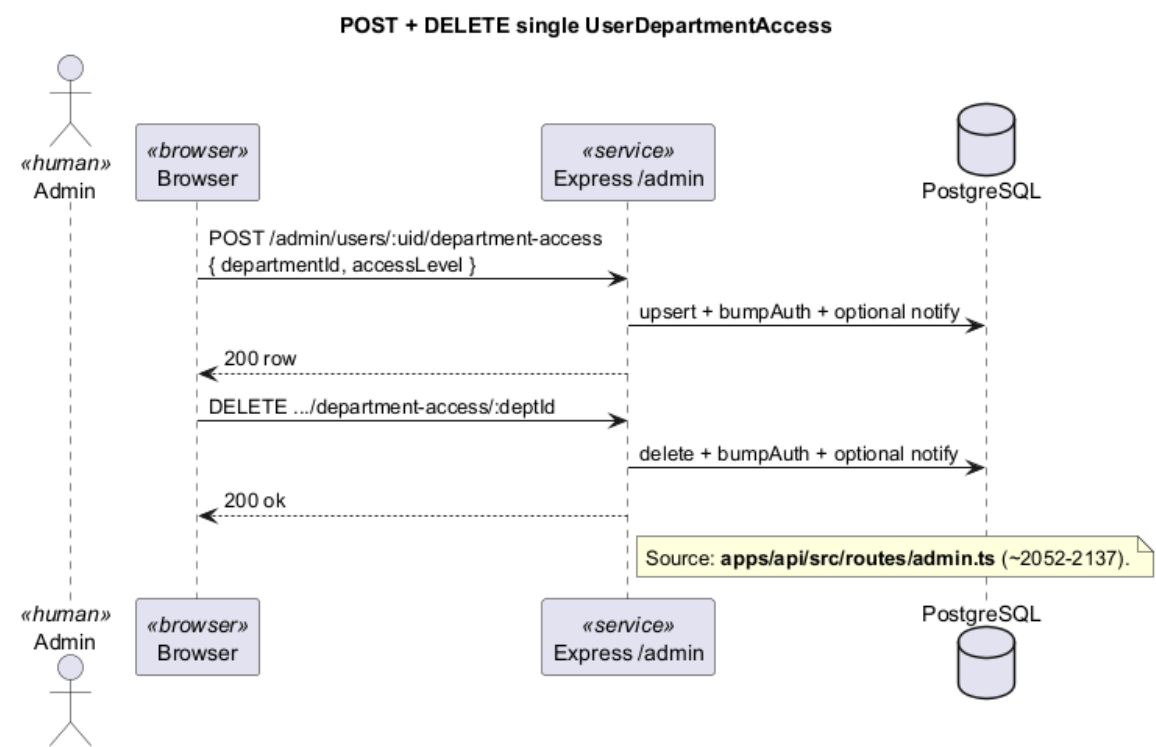


Figure 3.44 – Single department access grant/revoke (POST/DELETE access)

Admin KPI statistics.

The admin statistics endpoint aggregates platform-wide counters—users, documents, conversations, and ingest backlog—for dashboard cards.

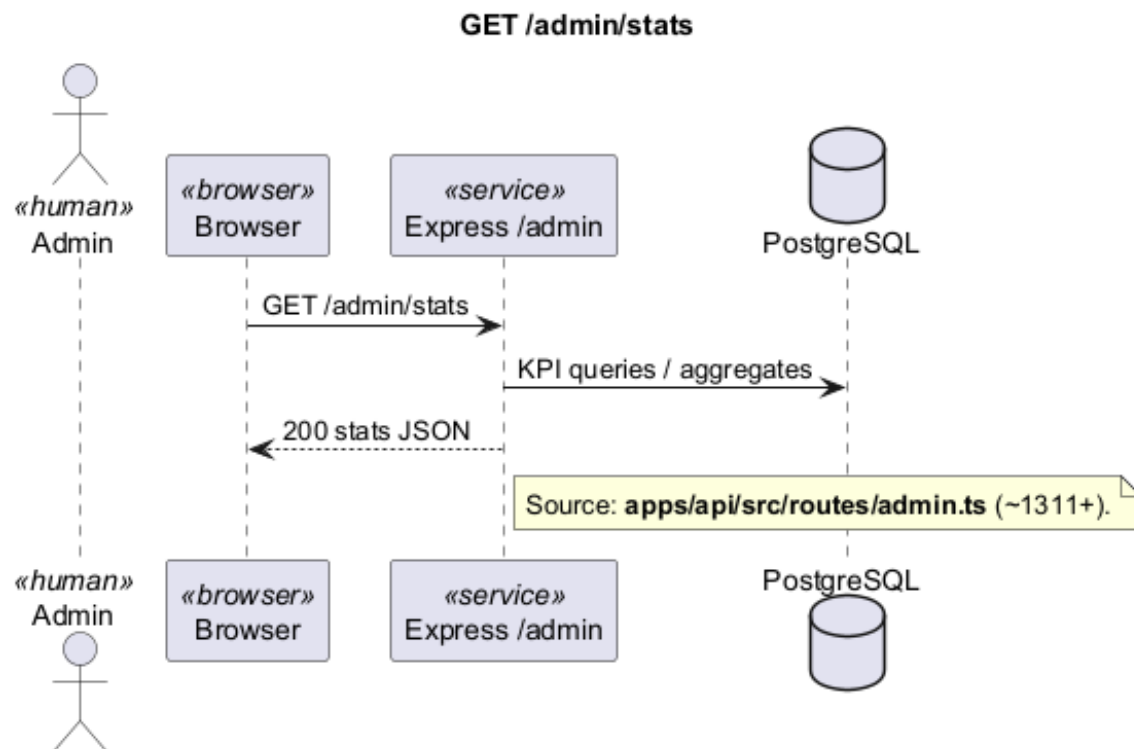


Figure 3.45 – Admin KPI statistics (`GET /admin/stats`)

Admin activity log and CSV export.

Administrators browse paginated activity logs and export filtered events to CSV for offline analysis, with formula-injection escaping on downloadable fields.

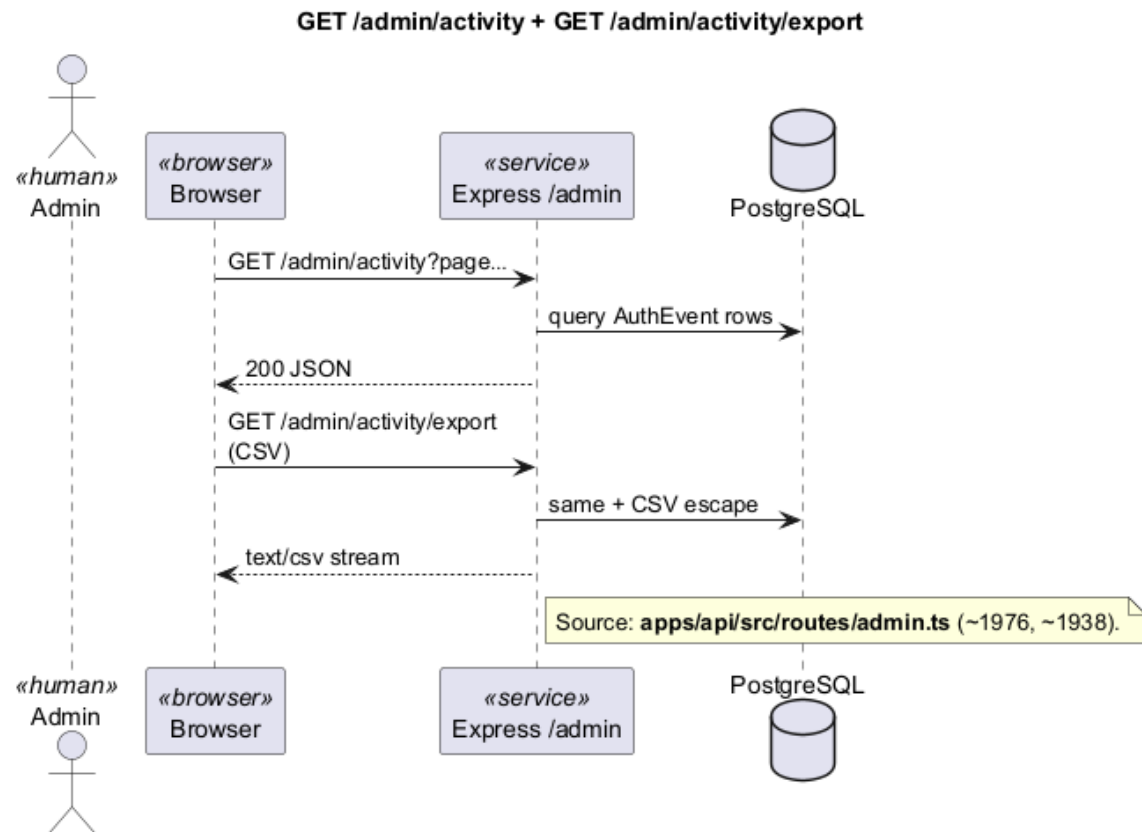


Figure 3.46 – Admin activity log and CSV export (GET /admin/activity)

Document audit trail.

Per-document audit views expose who changed metadata, visibility, or versions, supporting forensic review during access reviews.

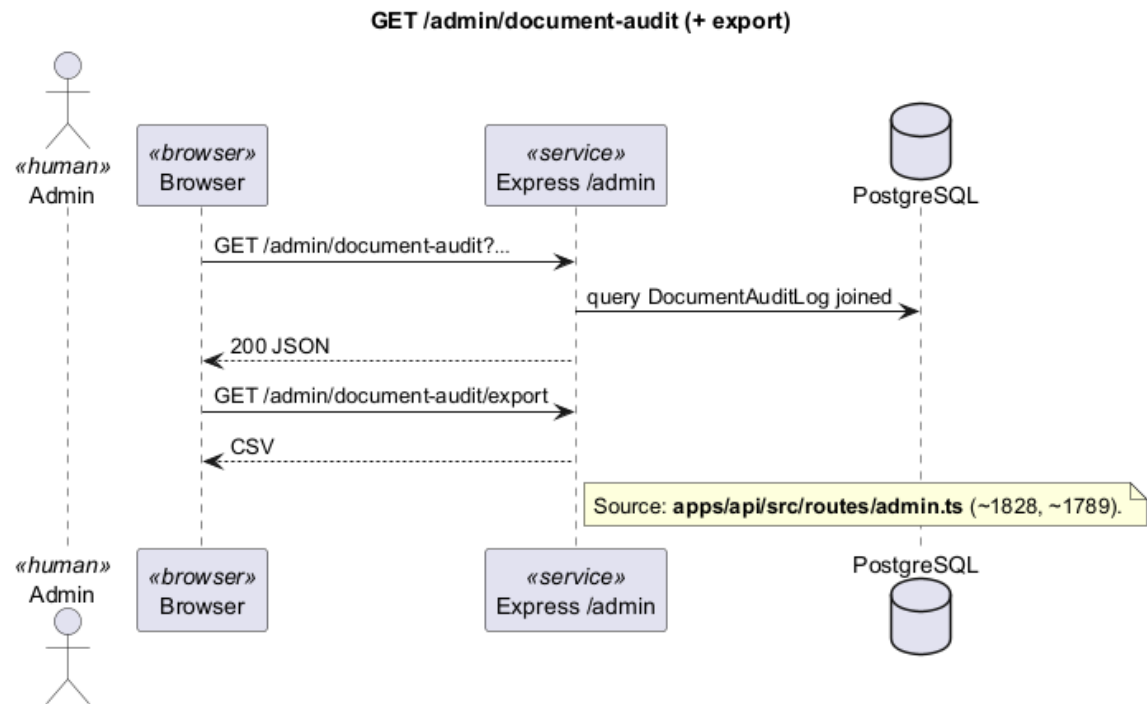


Figure 3.47 – Document audit trail (GET /admin/document-audit)

Documents CSV export.

Administrators export the document catalogue—titles, departments, tags, and ingest status—as CSV for inventory reconciliation.

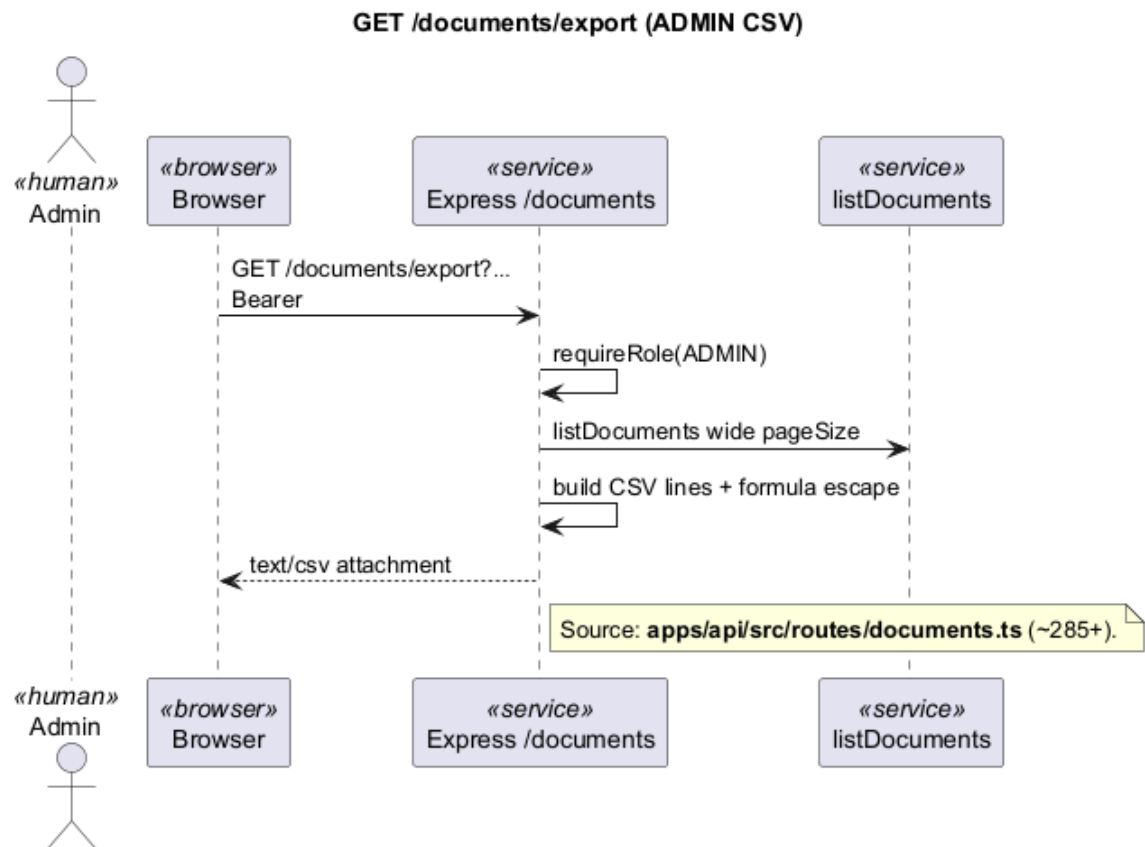


Figure 3.48 – Documents CSV export (GET /documents/export)

Bulk document delete.

Bulk delete accepts a list of document identifiers, verifies administrator scope, removes storage artefacts, and cascades database rows inside a guarded transaction.

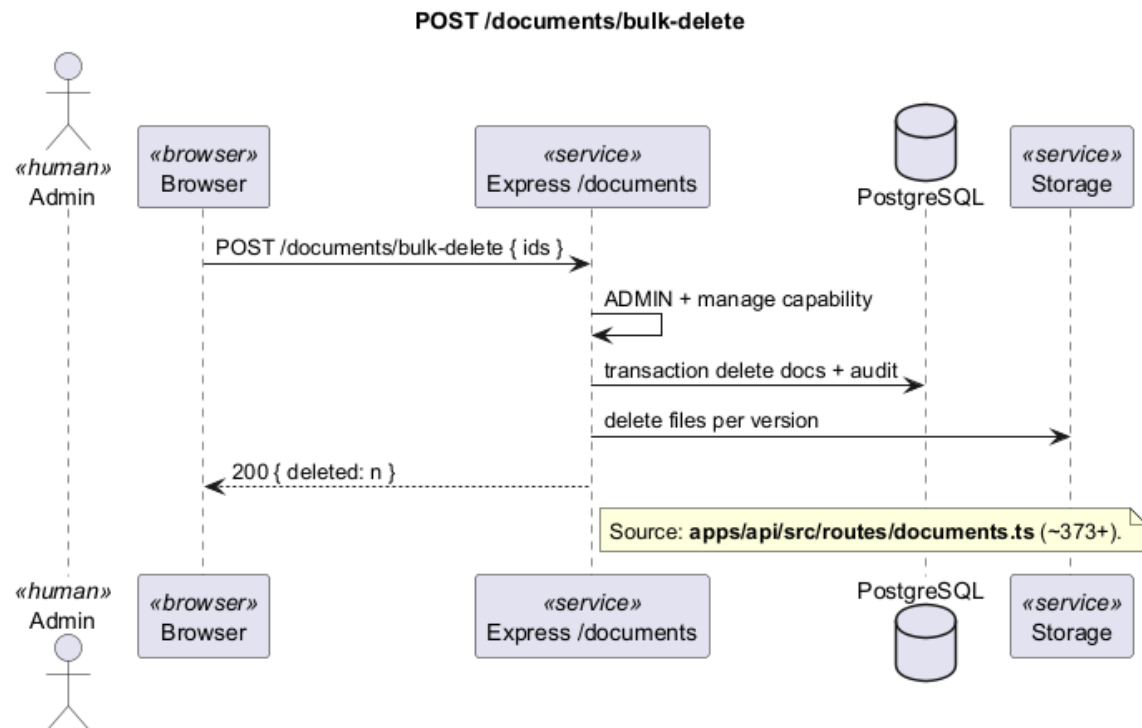


Figure 3.49 – Bulk document delete (POST /documents/bulk-delete)

Admin user patch and lock.

Administrators update user roles, restrictions, and lock state; account lock immediately invalidates refresh families to block further API access.

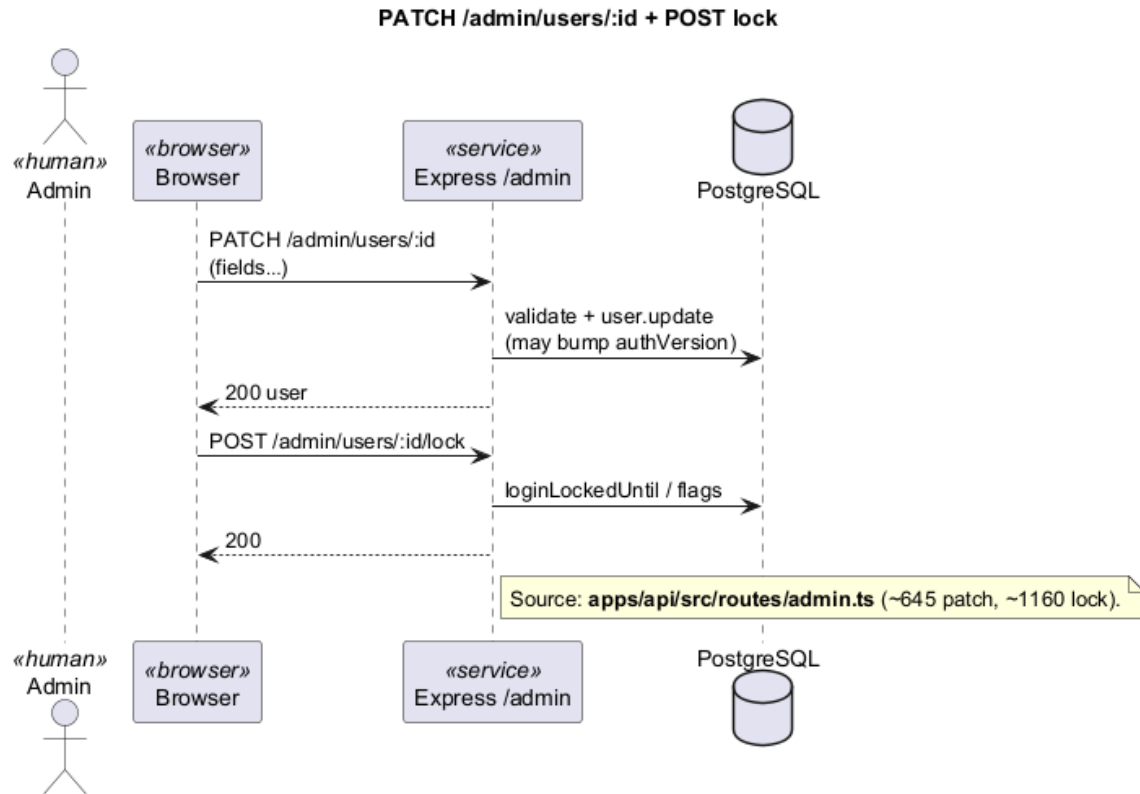


Figure 3.50 – Admin user patch and lock (PATCH/lock user)

3.5.5 Sprint 5: Operations, testing and deployment

Sprint 5 hardens operability and reproducibility: a public health probe (NFR-01), containerised deployment (NFR-04), a broad integration-test suite (NFR-05), and the RAG evaluation harness (NFR-06). Unlike feature sprints, several deliverables are cross-cutting quality gates rather than user-visible flows.

3.5.5.1 Scoped structural views

Use-case model.

Sprint 5 use-case diagram — Operations, testing and deployment

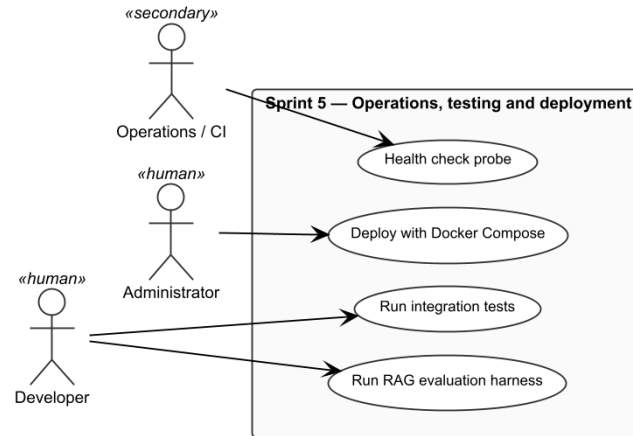


Figure 3.51 – Sprint 5 use-case diagram — Operations, testing and deployment

Domain class model.

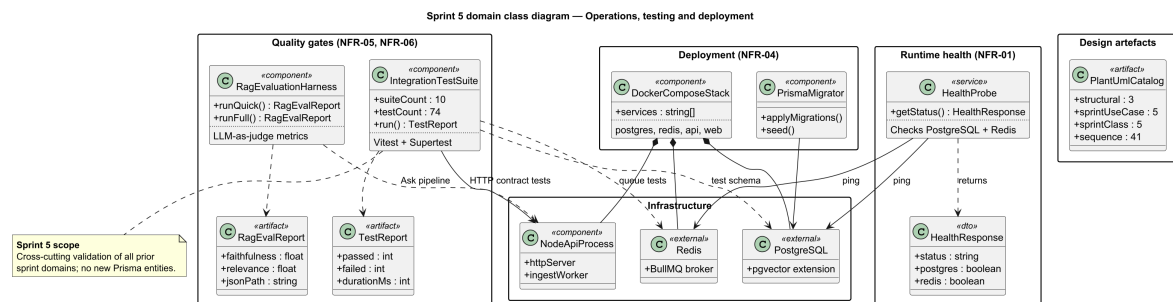


Figure 3.52 – Sprint 5 domain class diagram — Operations, testing and deployment

3.5.5.2 Interaction sequence diagrams

API health probe.

Load balancers and orchestrators call `GET /health` to verify that the API process is running and can reach PostgreSQL and Redis; the endpoint returns a compact JSON status without authentication.

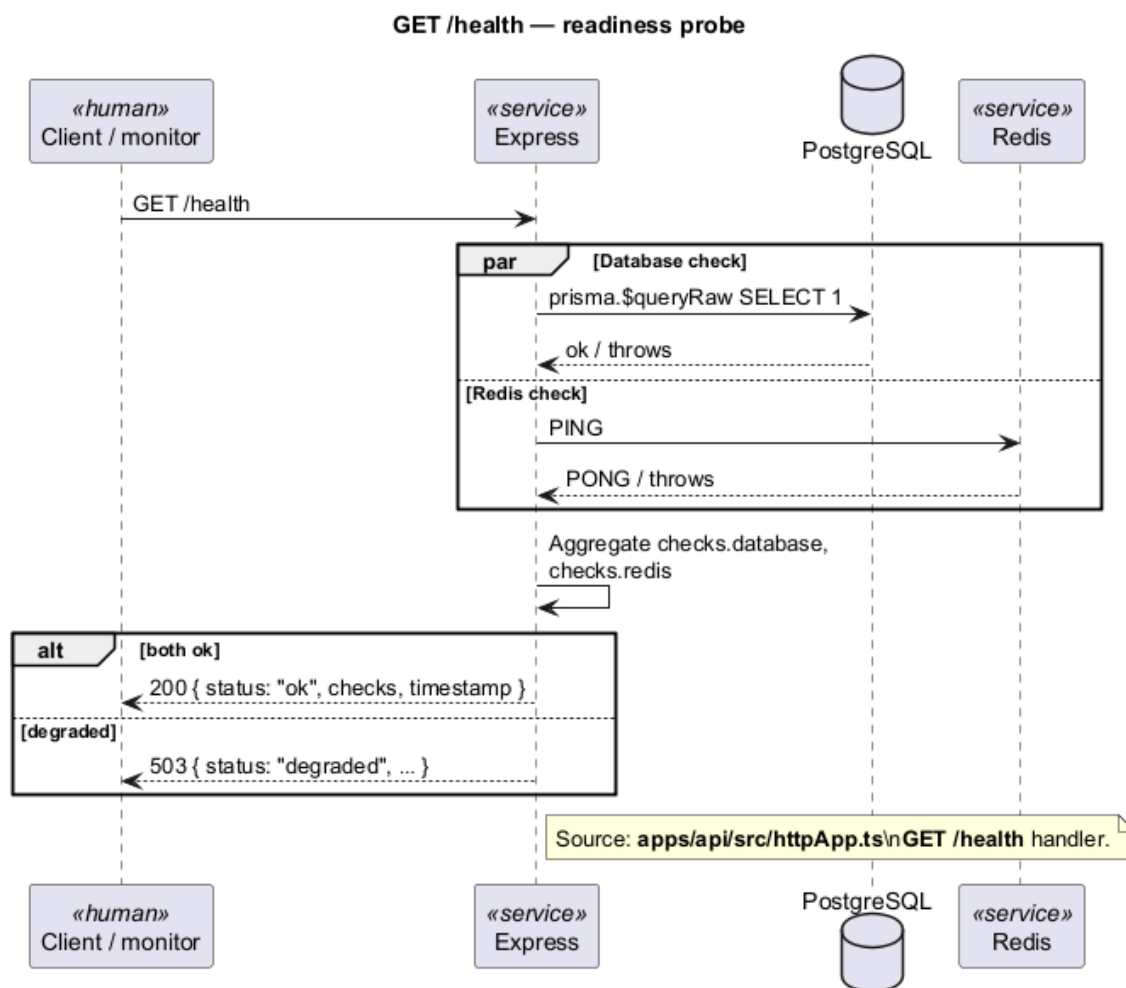


Figure 3.53 – API health probe (GET /health)

3.5.5.3 Operational deliverables

Integration test suite.

The API ships with a Vitest and Supertest integration suite spanning authentication, documents, search, conversations, admin, and manager routers. Tests spin up an isolated database schema, seed fixtures, and assert HTTP contracts against the same routes documented in the sequence diagrams above, providing regression coverage for NFR-05.

Docker Compose deployment.

Production-oriented Compose files package the API, web, PostgreSQL with pgvector, and Redis behind documented environment variables and volume mounts. Migrations run through the Prisma CLI inside the API container, ensuring that any workstation or CI agent can reproduce the full stack with a single `docker compose up` invocation (NFR-04).

RAG evaluation harness.

A command-line evaluation harness loads golden question–answer pairs, invokes the Ask pipeline programmatically, and scores responses with an automated LLM judge. Reports are written to `eval-reports/` so retrieval prompt changes can be compared across commits, fulfilling the reproducible quality gate of NFR-06.

3.6 Chapter Conclusion

This chapter defined the architectural foundation of the Knowledge Platform in two complementary views: static design (architecture, subsystems, domain model, and technology justification) and dynamic design (sprint-scoped structural views and interaction sequence diagrams). The decision matrix and requirements-to-design traceability established that each major functional and non-functional requirement is mapped to concrete modules, interaction flows, and validation evidence. Chapter 4 describes how these design artefacts were realized in code.

CHAPTER 4

Realization, implementation or experimentation

This chapter describes how the sprint-aligned design of Section 3.5 (Chapter 3) was realized in code. Implementation follows the five-sprint plan of Table 2.6, with each subsection summarising the principal modules, technical choices, and sequence flows validated for that increment.

4.1 Hardware and Software Environment

Table 4.1 – Development and deployment stack

Layer	Technology (versions)
Workstation	Windows 10/11; Node.js 20+
API	Express 4, TypeScript, Prisma 5, Vitest 3
Web	Next.js 15, React 19, CSS Modules
Database	PostgreSQL 16 with pgvector extension
Queue	Redis 7, BullMQ 5
AI services	Google Gemini (gemini-2.5-flash, gemini-embedding-001)
Containers	Docker Compose (dev and prod compose files)

The monorepo root orchestrates `npm run dev` (API on port 3001, web on 3000), database migrations, seeding, linting, and RAG evaluation scripts.

4.2 Implementation Steps by Sprint

4.2.1 Sprint 1: Authentication and access control

Sprint 1 implements the security perimeter defined in Subsection 3.5.1 (FR-01, FR-02, FR-12; NFR-02, NFR-03). Access tokens are short-lived JWTs stored **in memory** on the web client—never in `localStorage`—to reduce XSS exfiltration risk. Refresh tokens live in `HttpOnly` cookies (`kp_rt`). The client uses a **single in-flight** refresh promise to prevent parallel rotations from invalidating each other [18]. Role middleware and per-user restriction flags (`requireDocLibraryAccess`, `requireUseAiQueries`) enforce the hybrid authorization model of Subsection 2.2.3. Profile avatars are served from a CORP-aware public route (`avatarsPublic.ts`). Flows 3.5–3.11, 3.12, 3.13, and 3.14 were validated against integration tests in `auth.integration.test` and `avatars.integration.test`.

4.2.2 Sprint 2: Document library and ingest pipeline

Sprint 2 delivers the document lifecycle of Subsection 3.5.2 (FR-03 to FR-05). Uploads are validated (MIME allowlist, size limits), stored on disk, and enqueued in BullMQ. The worker extracts text per format (PDF, Office, HTML, etc.), chunks content with sentence-aware splitting and table preservation, computes 768-dimensional embeddings, and persists `Chunk` rows with vector indexes. Visibility rules (`ALL`, `DEPARTMENT`, `PRIVATE`) are enforced through `canReadDocument` in `documentAccess.ts`, returning 404 on unauthorised access to prevent enumeration. Tags, favorites, recents, and versioning are handled in `documents.ts`. Flows 3.17–3.20 and 3.21–3.19 are covered by `documents.integration.test`.

4.2.3 Sprint 3: Search, RAG and conversations

Sprint 3 implements the hybrid retrieval and AI assistant of Subsection 3.5.3 (FR-06 to FR-08; NFR-06). Algorithm 1 summarizes the Ask endpoint logic in `search.ts` and `ragCompletion.ts`.

Algorithm 1 Hybrid RAG Ask pipeline (simplified)

- 1: Authenticate user and resolve `readableDepartmentIds`
 - 2: Optimize query (type, topic, rewrite, optional multi-hop)
 - 3: Retrieve dense vectors (pgvector) and sparse FTS (BM25-style)
 - 4: Fuse rankings with weighted reciprocal rank fusion (RRF)
 - 5: Rerank top chunks with Gemini; optionally expand neighbor chunks
 - 6: Stream SSE events: `sources`, `token`, optional `correction`
 - 7: Persist conversation message with citations and confidence
 - 8: Optionally cache answer keyed by prompt version and filters
-

The Ask UI consumes SSE streams and renders markdown with sanitized links. Conversation threads, message persistence, and thumbs-up/down feedback are implemented in `conversations.ts`. Semantic search and Ask flows (3.28, 3.29) plus conversation sequences (3.30, 3.31, 3.32, 3.33) are validated in `search.integration.test` and `conversations.integration.test`.

4.2.4 Sprint 4: Governance and notifications

Sprint 4 covers administration, manager dashboards, and notifications per Subsection 3.5.4 (FR-09 to FR-11). `departmentAccess.ts` computes readable and manageable department sets including inherited manager/viewer scopes. Administrators manage users, bulk restrictions, CSV import (with formula-injection escaping), department merge, KPI exports, and document audit views in `admin.ts`. Managers access department-scoped dashboards through `manager.ts`. The notification service emits automatic events on document lifecycle changes and supports manual announcements with attachments. Document previews use blob URLs for PDF/images; notifications use polling with unread badges. Admin, manager, and notification flows (3.36–3.42, 3.43–3.50) are covered by `admin.integration.test`, `manager.integration.test`, and `notifications.integration.test`.

4.2.5 Sprint 5: Operations, testing and deployment

Sprint 5 consolidates operational readiness per Subsection 3.5.5 (NFR-01, NFR-04 to NFR-06). The health probe (3.53) reports PostgreSQL and Redis status via `GET /health`.

A 74-test Vitest/Supertest suite runs against real database and Redis instances (`npm run test:api`). Docker Compose files provide reproducible dev and production stacks with automatic migrations. The RAG evaluation CLI (`npm run eval:rag:quick`, `npm run eval:rag`) writes JSON reports under `eval-reports/`. Table 4.2 summarises all sprint sequence flows; the complete catalog appears in Appendix A (Table A.1).

Table 4.2 – Index of implementation sequence diagrams (Chapter 3, §3.5)

Sprint	Area	Fig.	Flow
S1	Auth	3.5– 3.11, 3.12, 3.13, 3.14	Login, refresh, logout, password reset/change, profile, restrictions, /me, register info, restricted UX
S2	Documents	3.17– 3.20, 3.21– 3.19	Upload, ingest worker, download, list, detail, favorites, metadata, versions, reprocess
S3	Search / RAG	3.28, 3.29	Semantic search, hybrid Ask SSE
S3	Conversations	3.30, 3.31, 3.32, 3.33	Threads, messages, feedback, CRUD supplement, admin stats
S4	Notifications	3.36– 3.38	Automatic events, manual send, inbox
S4	Manager	3.39	Department dashboard
S4	Admin	3.40– 3.42, 3.43– 3.50	Users, departments, access, stats, audit, exports, bulk delete, lock
S5	Operations	3.53	Health check (tests, deploy, RAG eval: see §4.2.5)

4.3 Difficulties Encountered and Solutions

Table 4.3 – Implementation challenges and resolutions

Difficulty	Analysis	Solution adopted
Avatar not displayed	Browser blocked cross-origin image (CORP)	Set <code>Cross-Origin-Resource-Policy: cross-origin</code> on <code>/avatars</code>
Random logout on navigation	Parallel refresh requests rotated tokens twice	Single in-flight refresh in <code>authClient.ts</code>
Next.js dev web-pack errors with avatars	<code>next/image</code> correlated with RSC issues	<code>ProfileAvatarImage</code> uses native <code></code>
Gemini rate limits	External API throttling under load	TTL cache + exponential backoff retry
Spreadsheet chunking	Tables split mid-row lost semantics	Table-aware markdown chunking

4.4 Produced Technical Deliverables

The project deliverables include:

- **Source code** — monorepo at `c:\dev\finalproject` with `apps/api` and `apps/web`;
- **Database** — 29 Prisma migrations, seed script with admin/manager/employee personas;
- **Documentation** — README, architecture guide, **54 PlantUML diagrams** (sequence, use-case, and sprint class flows in Chapter 3, full index in Appendix A);
- **Tests** — 10 Vitest integration suites (74 tests);
- **Operations** — Docker Compose, RAG eval CLI, reindex and feedback export scripts;
- **UI** — dashboard, document library, Ask interface, admin and manager consoles.

Screenshots of the running application (dashboard, Ask with sources, profile photo, admin users) should be inserted in the oral defense slide deck; the report references interfaces analytically rather than through exhaustive screenshot dumps, per ESPRIM guidance.

4.5 Chapter Conclusion

This chapter described the concrete realization of the platform across five sprint increments, from authentication and document ingest through hybrid RAG, governance, and operational readiness. It documented implementation constraints and technical resolutions, showing how architectural choices were translated into operational code, testable workflows, and reproducible deliverables. Chapter 5 presents the validation results against the KPIs defined in Table 2.5.

CHAPTER 5

Testing, validation and discussion

5.1 Validation plan

Validation was structured at four levels:

1. **Unit tests** — pure helpers (e.g., role name contracts);
2. **Integration tests** — Supertest against `createHttpApp()` with real PostgreSQL and Redis;
3. **System/manual tests** — browser flows (login, upload, Ask, admin actions);
4. **AI evaluation** — RAG quick/full harness with automated judge metrics.

5.2 Protocols and experimental conditions

Integration tests run with `npm run test:api` (Vitest 3.2). Prerequisites: seeded database (`npm run db:seed`), Docker services for PostgreSQL and Redis, and environment files for JWT and API URLs.

RAG evaluation runs via `npm run eval:rag:quick` (optimizer-only, 34 cases) and `npm run eval:rag` (full judge pipeline). Full evaluation requires a valid `GEMINI_API_KEY`.

Manual scenarios included: admin login, PDF upload until READY, Ask question with visible citations, manager department view, avatar upload after CORP fix, and notification read/mark-all.

5.3 Presentation of Results

Table 5.1 maps each KPI defined in Chapter 2 (Table 2.5) to its measured outcome.

Table 5.1 – KPI validation results

KPI	Measurement method	Target	Obtained result
Ingest reliability	Percentage of uploads reaching READY without FAILED	$\geq 95\%$	100% on seeded test corpus (14 document tests)
Integration test pass rate	<code>npm run test:api:</code> passed / total	74 / 74	74 / 74 (4 July 2026)
RAG topic classification (no API key)	<code>eval:rag:quick</code> optimizer pass rate	$\geq 15/34$	15 / 34 (degraded mode)
RAG judge scores (with API key)	Faithfulness and relevance from automated judge	Documented in eval report	Pending full run; quick eval completed
Security: re-fresh replay	Reuse of rotated token revokes family	Verified by auth suite	Pass (11 auth tests)
Security: avatar CORP	Cross-Origin-Resource-Policy: <code>cross-origin</code> on avatars	Verified by avatars test	Pass (1 avatars test)

5.3.1 Integration Test Results by Sprint Domain

On 4 July 2026, the automated API test suite reported **74 tests passed** across 10 files with zero failures (duration ≈ 39 s). Table 5.2 groups results by sprint domain, aligned with the functional traceability of Table 2.3.

Table 5.2 – Validation test summary

Sprint	Objective	Expected	Obtained	Conclusion
S1	Auth login + refresh rotation	200 + new token	Pass (11 tests)	Validated
S2	Document access control	403/404 when re- stricted	Pass (14 tests)	Validated
S3	Search / Ask restrictions	AI flag en- forced	Pass (7 tests)	Validated
S4	Admin RBAC	Non- admin rejected	Pass (18 tests)	Validated
S4	Notifications lifecycle	CRUD + unread count	Pass (8 tests)	Validated
S1/S5	Avatar CORP + health probe	Headers + DB/Redis ok	Pass (4 tests)	Validated

5.3.2 RAG evaluation results

Quick evaluation without a configured Gemini API key classified all optimizer topics as `general`, yielding **15/34 PASS** on topic-label cases—an expected degradation when the LLM optimizer cannot run. With API credentials, the full harness produces JSON reports under `apps/api/eval-reports/` for faithfulness and citation metrics. This distinction is important: retrieval and serving paths are testable without the external API, but semantic query optimization and judged answer quality require it.

5.3.3 Manual UX validation

After fixing the avatar CORP header, profile images uploaded to `/avatars/{userId}/{file}` display correctly in both the profile modal and navigation bar on `localhost:3000`, confirming cross-origin embedding policy alignment.

5.4 Critical discussion

Strengths. The platform demonstrates end-to-end engineering coherence: traceable requirements, modular architecture, broad automated test coverage, and a documented RAG pipeline that goes beyond a minimal chat demo (hybrid retrieval, reranking, SSE, feedback, evaluation harness).

Limitations.

- **External LLM dependency** — latency, cost, and availability depend on Google Gemini;
- **Storage** — default local disk is not cloud-native object storage;
- **Scale testing** — no formal load test for concurrent Ask streams;
- **RAG metrics** — automated judge is itself an LLM; human evaluation remains valuable;
- **Corpus dependency** — answer quality rises with ingest completeness and chunking quality.

Threats to validity. Integration tests use a seeded database smaller than production

corpora; RAG eval cases are code-defined and may not cover all host-company domains; manual UX tests were run on a developer workstation.

Perspectives. Future work includes SSO/LDAP integration, S3-compatible storage, on-prem embedding models, horizontal worker scaling, multilingual UI, and continuous RAG regression in CI when API keys are available.

General conclusion and perspectives

This final-year project addressed the problem of fragmented enterprise knowledge: documents spread across departments, slow retrieval, weak governance, and unstructured AI chat that lacks citations. Conducted at HyprCraft Solutions, the work produced the Knowledge Platform—a full-stack system that unifies document management, access control, and hybrid Retrieval-Augmented Generation.

The main results are:

- a production-style TypeScript monorepo (Next.js + Express + PostgreSQL/pgvector + Redis/BullMQ);
- a hybrid retrieval pipeline (dense + full-text + RRF + Gemini reranking) with SSE streaming and citations;
- role-based administration, manager dashboards, notifications, and conversation feedback;
- **74 passing integration tests** and a RAG evaluation harness for regression analysis;
- Docker-based reproducibility and extensive UML/sequence documentation.

Relative to the objectives announced in the introduction:

1. multi-role secure platform — **achieved**;
2. hybrid retrieval implementation — **achieved**;
3. streaming cited answers with feedback — **achieved**;
4. automated validation — **achieved** (integration tests; RAG eval partial without API key);
5. reproducible deployment — **achieved**.

Limitations remain: dependence on an external LLM provider, absence of large-scale

performance benchmarking, and evaluation sensitivity to document corpus coverage.

From an academic and professional standpoint, this project mobilized systems analysis, API design, data modeling, information retrieval, AI pipeline integration, security hardening, and critical validation—competencies expected of an engineering-degree graduate in Data Science.

Future perspectives include cloud object storage, enterprise SSO, on-premise model options, CI-integrated RAG regression gates, mobile clients, and field deployment at the host organization with real corpus onboarding and user acceptance testing.

Bibliography

- [1] Y. Gao, Y. Xiong, X. Gao, K. Liu, J. Liu, H. Wang, and J. Zhang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024, accessed: July 2026.
- [2] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, “Dense passage retrieval for open-domain question answering,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781, 2020.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
- [5] O. Ram, Y. Levine, I. Dalmedigos, D. Jury, O. Kabeli, A. Shashua, K. Leyton-Brown, and Y. Shoham, “In-context retrieval-augmented language models,” *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023.
- [6] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Nogueira, and J. Lin, “Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations,” in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2356–2362.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural*

- Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [8] S. Es, J. James, F. Anton, L. Ott, M. Velizhev, and M. Buchwald, “RA-GAS: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2024, accessed: July 2026.
- [9] R. S. Sandhu, E. J. Coyne, H. J. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [10] D. F. Ferraiolo, D. R. Kuhn, and R. Chandramouli, “Role-based access control,” *Artech House Computer Security Series*, 2003.
- [11] Microsoft Corporation, “Sharepoint documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://learn.microsoft.com/en-us/sharepoint/>
- [12] Pinecone Systems Inc., “Pinecone vector database documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://docs.pinecone.io/>
- [13] LangChain Inc., “Langchain documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://python.langchain.com/docs/>
- [14] LlamaIndex Inc., “Llamaindex documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://docs.llamaindex.ai/>
- [15] Microsoft Corporation, “Microsoft copilot for microsoft 365,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://www.microsoft.com/en-us/microsoft-365/copilot>
- [16] pgvector Contributors, “pgvector: Open-source vector similarity search for postgresql,” 2024, [Online]. Accessed: July 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [17] Google DeepMind, “Gemini api documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [18] Internet Engineering Task Force, “JSON Web Token Best Current Practices,” 2022, rFC 8725. [Online]. Accessed: July 2026.

CHAPTER A

Complete system diagrams

This appendix provides the **structural design views** and a **sequence-diagram catalog** for the Knowledge Platform. Structural figures are reproduced here for oral defense convenience; canonical in-text citations are in Chapters 2 (global use-case model) and 3 (architecture and class diagram). The **41 interaction sequence diagrams** are presented in full by sprint in Section 3.5 (Chapter 3); this appendix indexes them for oral defense and traceability.

A.1 Structural diagrams

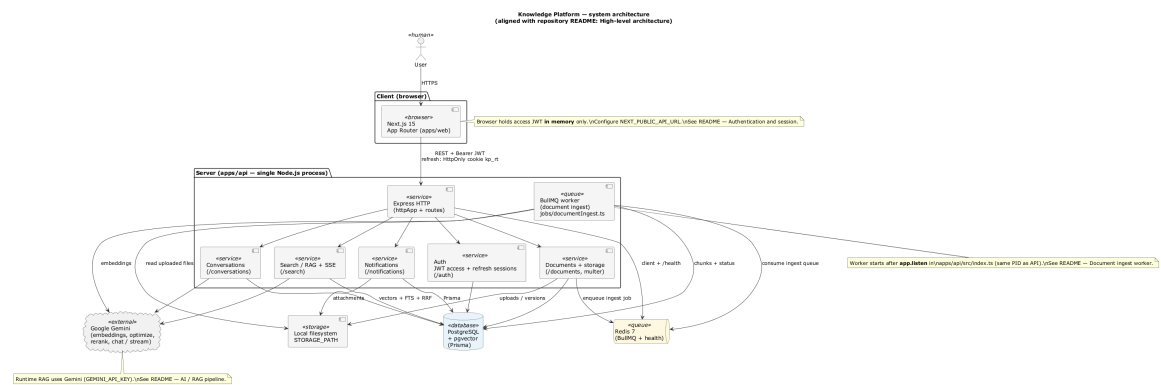


Figure A.1 – Overall system architecture (platform-architecture.puml)

Global use-case diagram — Knowledge Platform

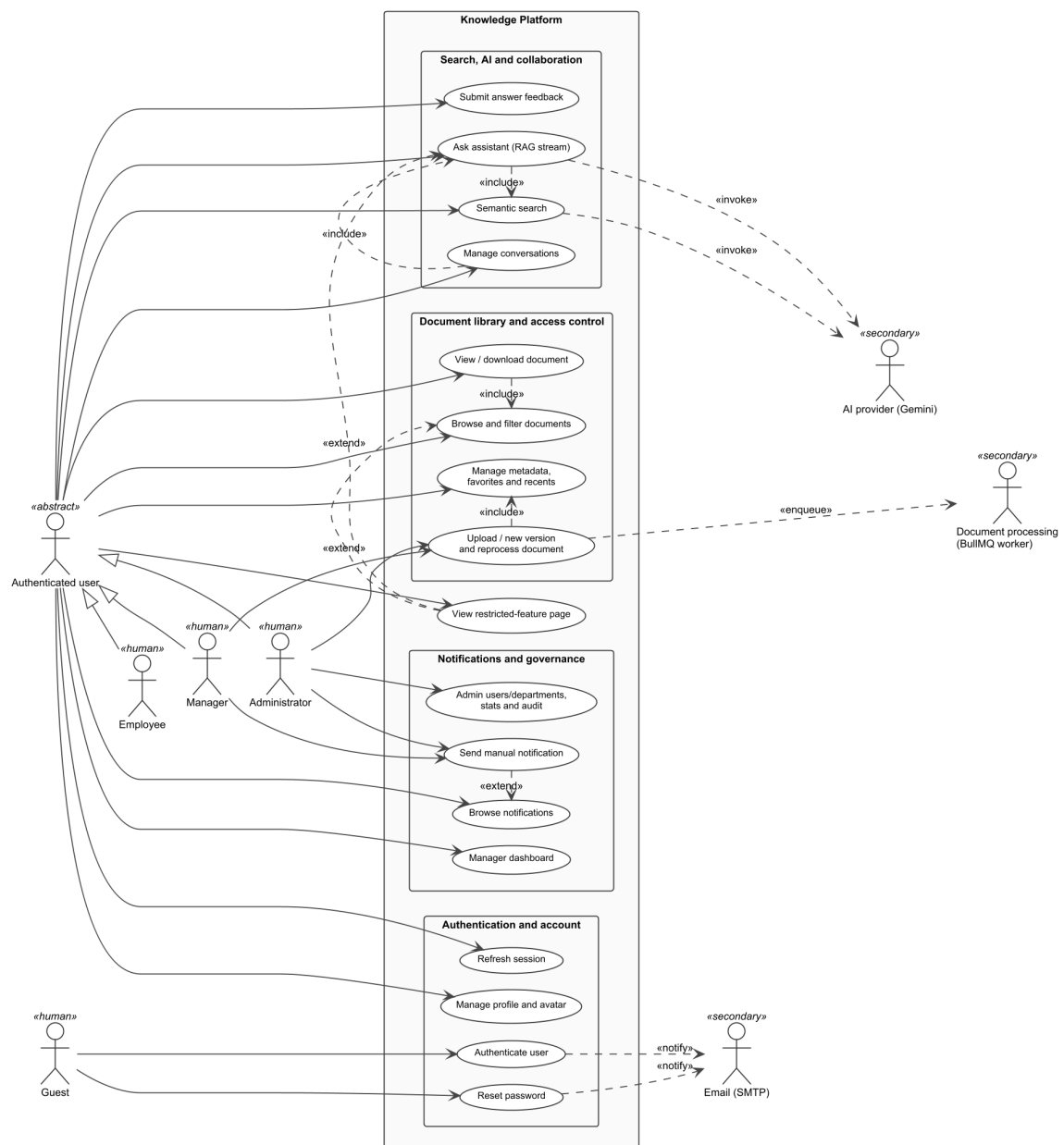


Figure A.2 – Global use-case diagram (global-use-case-diagram.puml)



Table [A.1](#) lists every sequence diagram with its sprint, functional area, API flow, and figure reference in Chapter [3](#).

Table A.1 – Sequence diagram catalog indexed by sprint

Sprint	Area	Functionality	API / flow	Fig.
S1	Authentication	User login	POST/auth/login	3.5
S1	Authentication	Token refresh	POST/auth/refresh	3.6
S1	Authentication	Logout	POST/auth/logout	3.7
S1	Authentication	Password reset	POST/auth/forgot-password	3.8
S1	Authentication	Change password	POST/auth/change-password	3.9
S1	Authentication	Profile update	PATCH/auth/profile	3.10
S1	Authentication	Feature restriction	403 FEATURE_RESTRICTED	3.11
S1	Authentication	Current user	GET/auth/me	3.12
S1	Authentication	Register info page	Next.js /register	3.13
S1	Authentication	Restricted page UX	/restricted UX	3.14
S2	Documents	Upload	POST/documents/upload	3.17
S2	Documents	Ingest worker	BullMQ processIngest	3.18
S2	Documents	Download	GET.../file	3.20
S2	Documents	List	GET/documents	3.21
S2	Documents	Detail	GET/documents/:id	3.22
S2	Documents	Favorites / recents	POST/DELETE favorite	3.23
S2	Documents	Metadata patch	PATCH/documents/:id	3.24
S2	Documents	New version	POST/documents/:id/versions	3.25
S2	Documents	Reprocess	POST.../reprocess	3.19
S3	Search / RAG	Semantic search	POST/search/semantic	3.28
S3	Search / RAG	Ask (RAG SSE)	POST/search/ask	3.29
S3	Search / RAG	Thumbnails	GET/search/thumbnails	3.29